WISE iRAG V2 Search Engine

Administrator & Developer's Guide

Version 1.0

(주)와이즈넛 경기도 성남시 분당구 대왕판교로644번길 49 DTC타워 5,6층

Copyright © WISEnut. All Rights Reserved

문서 개정 이력

버전	변경 일자	변경 내용	작성자
1.0	2025-02-04	최초 작성	임병수

목차

1. 제품 개요 **13**
1.1. 제품 특징 13
1.2. 모듈 구성 14
1.2.1. API 15
1.2.2. Cluster 15
1.2.3. Language Analyzer 16
1.2.4. Collection 16
1.2.5. Indexer 16
1.2.6. Searcher 17
1.2.7. Distributor 17
2. 설정 및 실행 **18**
2.1. 검색엔진 노드 설정 18
2.1.1. config.yaml 설정 18
2.2. 검색엔진 실행 및 종료 21
2.2.1. 엔진 실행 22
2.2.2. 엔진 종료 22
2.2.3. 엔진 상태 조회 22
3. 수집 **23**
3.1. Bridge(수집기) 소개 23
3.2. 수집 설정 26
3.2.1. DB Bridge 수집 설정 26
3.2.1.1. DataSource 설정 26
3.2.1.2. DataSource.Database 27
3.2.1.3. Config 설정 28
3.2.1.3.1. FtServer 설정 (첨부 파일 FilterServer) 29
3.2.1.3.2. Source 설정 30
3.2.1.4. 주요 항목 상세 설명 36

3.2.1.4.1. 날짜(Date)변환 설정 36
3.2.1.4.2. DSN 37
3.2.1.4.3. SubQuery 설정 38
3.2.1.4.4. SubMemory 설정 40
3.2.1.4.5. FilterInfo 설정 41
3.2.1.4.6. Query 구문 설정 44
3.2.1.4.7. Init 설정 46
3.2.1.4.8. Query 구문의 조건절 변경을 통한 동적 수집 48
3.2.1.4.9. 정적(Static) 문서 수집 49
3.2.1.4.10. 동적(Dynamic) 문서 수집 49
3.2.1.4.11. 추가(Insert)된 문서 수집 50
3.2.1.4.12. 수정(Update)된 문서 수집 51
3.2.1.4.13. 변경(Replace)된 Field 수집 53
3.2.1.4.14. 삭제(Delete)된 문서 수집 54
3.2.1.4.15. 관리자 테이블을 이용한(Master) 문서 수집 56
3.2.1.4.16. PrimaryKey의 설정 57
3.2.2. Web Bridge 수집 설정 58
3.2.2.1. Config 설정 58
3.2.2.2. Web Bridge의 URL Seed File 작성 방법 62
3.2.3. File Bridge 수집 설정 64
3.2.3.1. Config 설정 64
3.2.3.2. Source 설정 65
3.2.3.3. Source.RefColl 설정 (옵션) 66
3.2.3.4. Source.SCDdir 설정 66
3.2.3.5. Source.CategoInfo.Mapping 67
3.2.3.6. Source.TargetFile 68
3.2.3.7. Source.History 68
3.2.3.8. Source.DirAcceptRule 68
3.2.3.9. Source.FileAcceptRule 68
3.2.4. 기타 수집 모듈 69
3.2.4.1. 문서 Filter 와 FilterServer 69
3.2.4.1.1. FilterServer의 등록 69

3.2.4.1.2. FilterServer의 실행 69
3.2.4.1.3. 문서 Filter 실행 파일 70
3.2.4.1.4. file.allow파일의 설정 71
3.2.4.1.5. FilterExt의 설정 72
3.2.4.2. WNEncrypt 73
3.2.4.3. SCDRecvServer(SCD 수신 서버) 74
3.2.4.4. Bridge 설정 추가 74
3.2.4.5. SCDRecvServer의 실행 75
3.3. 수집기 실행 77
3.3.1. DB Bridge 실행 77
3.3.2. Web Bridge 실행 79
3.3.3. File Bridge 실행 81
4. 사전 84
4.1. 사전 등록 84
4.1.1. 사전 ID 84
4.1.2. 사전 타입 84
4.1.3. 사전 등록 (초기화) 85
4.1.4. 사전 키워드 업데이트 및 키워드 삭제 85
4.1.5. 사전 삭제 87
4.1.6. 사전 조회 87
4.2. 사용자 명사 사전 88
4.2.1. 단일 명사 분석 정보 89
4.2.2. 복합 명사 분석 정보 90
4.3. 동의어 사전 90
4.3.1. 동의어: 하위어 92
4.3.2. 동의어 등록 시 주의사항 93
4.3.2.1. 동일 단어를 동의어로 등록하지 않는다. 93
4.3.2.2. 하위어 개념으로 추가할 경우에는 표제어로 등록한다. 93
4.3.2.3. 동의어 사전에 등록한 단어를 금칙어 사전에 등록하지 않는다. 94
4.3.3. 동의어 확장 룰 (Synonym Expansion Rule) 94
4.3.3.1. Rule 94
4.3.3.2. 사전 예시 94

4.3.3.3. 검색 결과 예시 96
4.4. 금칙어 사전 99
4.5. 애널라이저 및 컬렉션에 사전 적용 100
4.5.1. 사용자 명사 사전 적용 100
4.5.2. 동의어 사전 적용 101
4.5.3. 금칙어 사전 적용 102
5. ModelServer 103
5.1. ModelServer 등록 103
5.1.1. ModelServer server 103
5.1.2. ModelServer name 103
5.1.3. ModelServer 등록 (초기화) 103
5.1.4. ModelServer 업데이트 및 특정 model, server 해제 104
5.1.5. ModelServer 조회 105
6. 언어 분석 106
6.1. 지원문자 집합 106
6.2. 색인어 및 검색어 분석 106
6.2.1. 레귤레이터 - 구분자 및 병합자 지정 107
6.2.2. 고급 레귤레이션 111
6.2.2.1. 다중 method : 동시 적용 111
6.2.2.2. 다중 method: 각각 적용 116
6.2.3. 색인어 및 검색어 추출 121
6.2.3.1. 언어 독립 분석 121
6.2.3.1.1. token 121
6.2.3.1.2. ngram 122
6.2.3.1.3. like 122
6.2.3.1.4. matrix 123
6.2.3.2. 언어 종속 분석 125
6.2.3.2.1. 형태소 분석 125
6.2.3.2.2. stemmer 125
6.3. 형태소 분석 126
6.3.1. 한국어 형태소 분석 (KMA) 126
6.3.1.1. 토큰 색인 옵션(A) 126

6.3.1.2. 접사 관련 옵션 (F) 127
6.3.1.3. 분석 결과 개수 옵션 (R) 128
6.3.1.4. 숫자 형식 결합 옵션 (N) 129
6.3.1.5. 숫자 추출 옵션 (N) 130
6.3.1.6. 용언 색인 옵션 (V) 131
6.3.1.7. 단위성 의존명사 결합 옵션 (B) 131
6.3.1.8. 기호/알파벳/숫자 자동 조합 옵션 (X) 132
6.3.1.9. 복합명사 분해 옵션 (C) 133
6.3.1.10. 영어 스테밍 옵션 (S) 134
6.3.2. 영어 형태소 분석 134
6.3.2.1. 명사 원형 색인 옵션 (M) 134
6.3.2.2. 동사 원형 색인 옵션 (V) 135
6.3.3. 일본어 형태소 분석 (JMA) 135
6.3.3.1. 기본 옵션(Basic option) 135
6.3.3.1.1. 단위 명사 결합 옵션 (B) 135
6.3.3.1.2. 후리가나 색인 옵션 (D) 136
6.3.3.1.3. 접사 결합 옵션 (F) 136
6.3.3.1.4. 영어 스테밍 옵션(S) 137
6.3.3.1.5. 용언 색인 옵션(V) 137
6.3.3.2. 확장 옵션(Extension option) 138
6.3.3.2.1. 히라가나 변환 옵션(H) 138
6.3.3.2.2. 가타카나 변환 옵션(K) 138
6.3.3.2.3. 일본어 흔들림 표기 변환 옵션(L) 139
6.3.3.2.4. 작은 가나 변환 옵션(S) 139
6.3.4. 중국어 형태소 분석 (CMA) 140
6.3.4.1. 기본 옵션(Basic option) 140
6.3.4.1.1. 단위 명사 결합 옵션 (B) 140
6.3.4.1.2. 병음 변환 옵션 (P) 140
6.3.4.1.3. 번체 변환 옵션 (T) 141
6.3.4.1.4. 간체 변환 옵션 (S) 141
6.3.4.2. 확장 옵션(Extension option) 142
6.3.4.2.1. 영어 스테밍 옵션(E) 142

6.3.4.2.2. 복합 명사 분해 옵션(R) 142
6.3.5. 동의어 분석 143
6.4. 애널라이저 관리 143
6.4.1. 애널라이저 관리 내용 144
6.4.2. 애널라이저 관리 방법 및 예시 150
6.4.3. 일본어, 중국어 형태소 분석기 애널라이저 등록 방법 및 예시 151
7. 컬렉션 153
7.1. 컬렉션 정의 153
7.1.1. Collection(컬렉션) 153
7.1.2. Shard(샤드) 153
7.1.3. Replica(레플리카) 154
7.2. 컬렉션 관리 155
7.2.1. 초기화 156
7.2.1.1. 삭제 160
7.2.1.2. 조회 160
7.2.1.3. 컬렉션 리스트 조회 160
7.2.2. 컬렉션 업데이트 161
7.3. 컬렉션 상태 163
7.3.1. 컬렉션 Shard 상태 163
7.4. 컬렉션 새로고침 166
7.4.1. 컬렉션 새로고침 166
8. 색인 및 배포 167
8.1. 색인 운영 방식 167
8.1.1. 색인 방식 - Gateway 169
8.1.1.1. Config 설정 170
8.1.1.2. Gateway 실행 172
8.1.1.3. TEA 연동 가이드 174
8.1.2. JSON 색인 방식 178
8.1.2.1. 색인 (Index) 178
8.1.3. 색인어 보기 (Index Morpheme) 180
8.1.4. 색인 문서 업데이트 182
8.1.4.1. 색인 문서 업데이트 (Index Update) 182

8.1.5. 색인 삭제 184
8.1.5.1. DOCID 명시 삭제 방법 184
8.1.6. 색인 데이터 정보 185
8.2. 배포 187
8.2.1. 배포 운영 방식 187
8.2.1.1. 색인 노드 재할당 187
8.2.1.2. 정적 색인 시작 187
9. 검색 189
9.1. [예제 실행을 위한 설정] 189
9.2. 기본 검색 쿼리 (Basic Query) 198
9.2.1. 개요 198
9.2.2. 기본 검색 198
9.2.2.1. API 호출 198
9.2.2.2. 예제 198
9.2.2.2.1. 검색 기본 구조 198
9.2.2.2.1.1. [기본 검색 예시] 198
9.2.2.2.1.2. [통합 검색 예시] 199
9.2.2.2.2. 검색 옵션 200
9.2.2.2.3. 쿼리 내부 옵션 201
9.2.2.2.4. 결과 203
9.2.2.3. 쿼리 확장 옵션 예제 203
9.2.2.3.1. 쿼리 확장 옵션 기본 구조 204
9.2.2.3.2. 쿼리 옵션 204
9.2.2.3.3. 검색 옵션 별 쿼리 확장 205
9.2.2.3.3.1. useLa, useSynonym 을 사용하지 않을 경우 205
9.2.2.3.3.2. 형태소 분석기만 적용하는 경우 206
9.2.2.3.3.3. 형태소 분석기, 동의어사전을 모두 사용하는 경우 208
9.2.2.3.3.4. originalTerm 에 대해서만 확장하는 경우 209
9.2.2.3.4. 부가 기능 211
9.2.3. 벡터 검색 212
9.2.3.1. API 호출 212
9.2.3.2. 예제 212

9.2.3.2.1. 벡터 검색 구조 212
9.2.3.2.2. 벡터 쿼리 옵션 214
9.2.3.2.3. 벡터 검색 결과 215
9.3. 필터 쿼리 (Filter Query) 219
9.3.1. 개요 219
9.3.2. Type별 연산자 정리 219
9.3.2.1. Text Type operator 219
9.3.2.2. Long Type operator 220
9.3.2.3. filterQuery 기본구조 221
9.3.2.4. 옵션설명 221
9.3.3. MATCH 연산자 221
9.3.4. NOT_MATCH 연산자 222
9.3.5. IN 연산자 222
9.3.6. NOT_IN 연산자 223
9.3.7. START 연산자 224
9.3.8. END 연산자 224
9.3.9. SUBSTRING 연산자 225
9.3.10. GT, GTE, LT, LTE 연산자 225
9.3.11. RANGE 연산 226
9.3.11.1. API 호출 226
9.3.11.2. range 연산 기본구조 226
9.3.11.3. 옵션설명 226
9.3.11.4. 예시 227
9.3.12. 날짜 제한 검색 쿼리 (Date Range Query) 227
9.3.12.1. API 호출 228
9.3.12.2. date_range 연산 기본구조 228
9.3.12.3. 옵션설명 228
9.3.12.4. 예시 229
9.3.13. 카테고리 검색 쿼리 (Category Search Query) 229
9.3.13.1. API 호출 230
9.3.13.2. categorySearch 연산 기본구조 230
9.3.13.3. 옵션설명 230

9.3.13.4. 예시 230
9.4. 권한 쿼리 (Authority Query) 231
9.4.1. 개요 231
9.4.1.1. 권한 컬렉션 API 호출 231
9.4.1.2. 권한 쿼리 API 호출 232
9.4.1.3. 권한 조회 232
9.4.2. 권한 컬렉션 (Authority Collection) 232
9.4.2.1. 컬렉션 내부 옵션 232
9.4.3. 권한 쿼리 (Authority Query) 233
9.4.3.1. 권한 쿼리 기본 구조 233
9.4.3.2. 쿼리 내부 옵션 234
9.4.3.2.1. 예제 234
9.4.3.2.2. 권한 컬렉션 설정 234
9.4.3.2.3. 검색 234
9.5. 부가 기능 235
9.5.1. 소개 235
9.5.2. 정렬 (Sorting) 235
9.5.2.1. API 호출 235
9.5.2.2. 정렬 기본 구조 235
9.5.2.3. 쿼리 내부 옵션 236
9.5.2.4. 단일 정렬 예제 236
9.5.2.5. 멀티 정렬 예제 237
9.5.3. 형태소 분석 조회 기능 (Morpheme Analysis) 238
9.5.3.1. API 호출 238
9.5.3.2. 쿼리 내부 옵션 239
9.5.3.3. 예제 240
9.5.4. 최근 검색어 조회 (Recent Query List) 240
9.5.4.1. API 호출 240
9.5.4.2. 기본 구조 241
9.5.4.3. 쿼리 내부 옵션 241
9.5.4.4. 예제 242
9.5.5. 머지 컬렉션 (Merged Collection) 243

9.5.5.1. API 호출 244
9.5.5.2. 머지 컬렉션 검색 기본 구조 244
9.5.5.3. 쿼리 내부 옵션 245
9.5.5.4. 예제 246
9.5.6. 거리 제한 검색 (Geo Distance Search) 248
9.5.6.1. API 호출 248
9.5.6.2. 거리 검색 일반 검색 기본 구조 248
9.5.6.3. 쿼리 내부 옵션 249
9.5.6.4. 예제 249
9.6. 랭킹 250
9.6.1. 랭킹 요소 250
9.6.2. 기본 구조 251
9.6.3. 랭킹 내부 옵션 251
9.6.4. 키워드 부스팅 쿼리 주의 사항 252
9.6.5. 기본 랭킹 252
9.6.6. 커스텀 랭킹 260
10. 로그 관리 262
10.1. 로그 설정 262
10.1.1. 로그 파일 경로 262
10.1.2. 로그 언어 지원 263
10.2. 시스템 로그 263
10.3. 색인 로그 264
10.4. 검색 로그 265

1. 제품 개요

본 제품은 데이터 빅뱅 시대에 초대용량 검색을 요구하는 고객 니즈가 점차 늘어남에 따른 선제적 대응으로 개발된 Java 기반 클라우드 레벨 분산형 검색엔진이다. 1억건 이상의 초대용량 검색이 가능하도록 개발되었다.

1.1. 제품 특징

iRAG Search Engine은 Java 기반의 분산형 검색엔진으로 클라우드 환경에 특화되어 있는 초대용량 분산형 검색엔진이다. 본 제품은 클러스터 구조로 설계되어 부하분산, 고가용성 확보는 물론 단일 검색엔진보다 뛰어난 성능과 안정성을 제공하며, 최적화된 멀티스레딩 구현을 통해 색인과 동시에 바로 검색이 가능하다. 또한 REST API 지원, 분산 색인 및 분산 검색을 통해 구축 및 운영 과정이 간편화되었다.

초대용량의 검색/색인, 높은 수준의 성능과 품질의 검색결과 제공을 요구하는 고객 니즈를 만족시키기 위해서는 무엇보다 구축 환경에 최적화된 클러스터 구조 설계가 매우 중요하다. 이에 본 제품은 클러스터 구성 시 여러 설정 파일들을 구성하면서 반복되는 업무를 단순화시켜 설정 시간을 단축하고 Human-Error를 최소화하여 원활한 클러스터 구조 설계가 가능하도록 개발되었다. 본 제품의 주요 특징은 다음 표와 같다.

[표 1.1 iRAG Search Engine의 주요 특징]

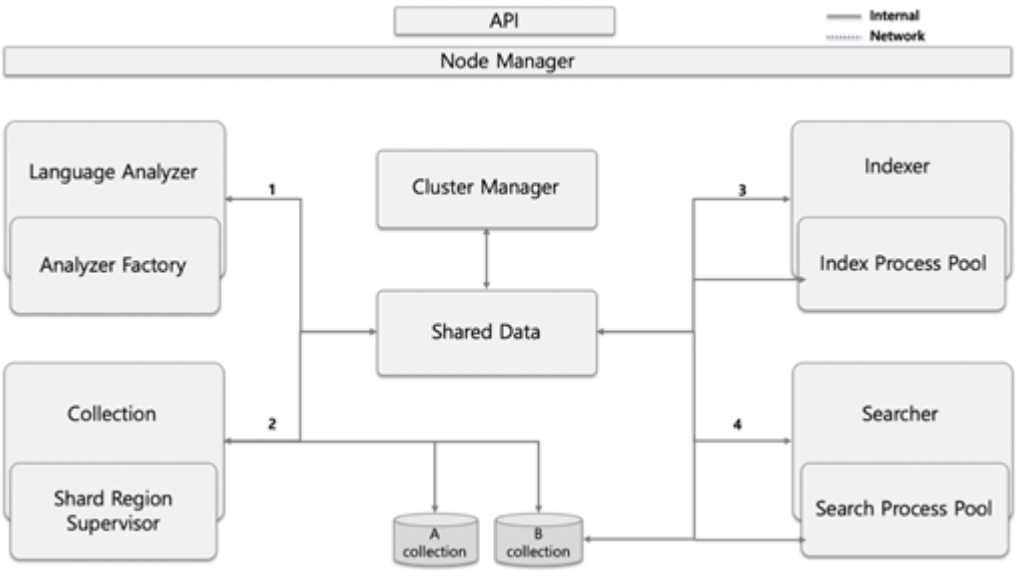
주요특징	**설 명**
안정성 및 호환성 확보	iRAG Search Engine 엔진은 장애가 있는 검색기를 자동으로 탐지하고, 다른 검색기가 가지고 있는 백업 컬렉션을 통해 안정적인 검색이 가능함
준실시간 색인 및 멀티스레딩	멀티스레딩의 최적화가 진행되어 빠르게 색인 작업을 실행할 수 있음 색인과 동시에 바로 검색이 가능한 시스템 구조임
REST API 지원	기존 Query API 통신 구조를 엔진이 직접 REST API 호출이 가능해짐 간단한 구성은 API 호출을 통해 구성이 가능해짐

분산검색	iRAG Search Engine 엔진은 모든 노드가 검색 요청을 받을 수 있는 주체로서 부하를 줄일 수 있음
구축 및 운영 과정의 반복 업무 단순화&개선	설정 파일을 최소화하여 설정파일을 여러 개 구성해서 만드는 데 발생되는 시간을 단축 시킴 위 장점을 통해 엔진 구축 중 발생될 수 있는 Human-Error를 최소화함
벡터검색	기존의 키워드 검색과 다르게 단어의 의미와 문맥을 이해하여 검색 결과 제공이 가능하며 사용자가 원하는 정보와 유사한 내용을 가진 문서를 출력 가능함

1.2. 모듈 구성

iRAG Search Engine은 API, Cluster, Language Analyzer, Collection, Indexer, Searcher가 하나의 엔진으로 구성되어있다. 하나의 binary가 복합적인 일을 수행하도록 구성되어 있다.

[그림 1.1 iRAG Search Engine의 모듈 구성]



1.2.1. API

iRAG Search Engine은 기본적으로 Query API를 통해 엔진과 통신할 수 있으며, REST API를 통해 엔진 내부의 모든 모듈과 통신이 가능하다. REST API는 메시지를 읽는 것 만으로도 메시지가 의도하는 바를 명확하게 파악할 수 있는 장점을 가지고 있어 가독성을 확보하였으며, REST API 사용을 위한 별도 인프라 구축을 요구하지 않기 때문에 원하는 API를 쉽고 빠르게 호출이 가능하다.

REST API는 기본적으로 Body 데이터와 함께 사용되는 POST와 PUT, Body 데이터를 사용하지 않는 DELETE, GET 총 4가지의 Method를 지원한다. 각 API는 Http 표준에 맞게 사용되어야 하며, 등록, 업데이트, 삭제에 사용되는 POST와 조회에 사용되는 GET으로 구분 지을 수 있다.

[표 1.2 REST API 구성]

Method 명	**설명**	**Body 데이터 사용 여부**
POST	데이터 삽입, 삭제 및 업데이트에 사용됨	O
GET	데이터 조회에 사용됨	X

1.2.2. Cluster

Cluster 모듈은 분산 구조 구성을 위한 모듈이다. Cluster(클러스터)는 하나 이상의 Node(노드: 하나의 서버(엔진), 검색/색인 구동)들의 집합으로 모듈안에는 여러 개의 노드들로 이루어져 있다. 또한 여러 노드들을 추가/제거할 수 있으며 멀티적으로 구동할 수 있도록 지원하는 Seed node(씨드 노드)로 구성되어 있다. Cluster 모듈은 현재 서버의 상태, 클러스터의 상태 정보 등을 제공 받을 수 있다.

1.2.3. Language Analyzer

Language Analyzer 모듈은 형태소 분석, 언어 분석을 담당하는 모듈이다. 이 모듈은 기본적으로 오픈소스 형태소 분석기를 커스텀하여 연동할 수 있도록 구조화 되었으며, 커스텀 형태소 분석기를 자유롭게 추가할 수 있는 기능은 추후 제공될 예정이다.현재 iRAG Search Engine에서 사용되는 형태소 분석기는 자사에서 개발한 WISE KMA-Black으로 한국어를 지원하고 있다.

1.2.4. Collection

Collection 모듈은 iRAG Search Engine에서 제공하는 Collection 설정, Collection 구성과 관련된 기능을 제공하는 모듈이다. 여기서 Collection이란 검색 대상이 되는 데이터 세트 단위를 의미한다.

iRAG Search Engine은 클라우드 환경에서 유연한 구성을 제공하기 위해 Shard/Replica 개념을 도입했다. Shard란 컬렉션을 분할한 파티션 단위를 의미한다. Collection을 두꺼운 책 한권으로 비유하면, Shard는 두꺼운 책을 챕터별로 나눈 단위라고 생각할 수 있다. Replica는 Shard의 복제본을 의미한다. 자세한 내용은 6장을 참고한다.

1.2.5. Indexer

Indexer 모듈은 수집한 문서를 Language Analyzer 모듈을 이용하여 검색하기 용이하도록 텍스트 형태로 추출/가공하는 전처리와 색인 또는 검색 시 해당 문서의 정보를 바로 제공하기 위하여 특정 형태로 저장하는 역할을 수행한다. 또한 검색어가 들어왔을 때 해당 검색어가 포함된 문서를 빠르게 찾아주기 위해 각각의 검색어가 들어있는 문서의 리스트(색인 데이터)를 구축한다.

1.2.6. Searcher

Searcher 모듈은 Indexer 모듈이 만든 색인 데이터를 참고하여 사용자가 요청하는 검색을 수행한다. “필터 쿼리”, “카테고리 쿼리” 등 다양한 검색 옵션을 지원한다.

1.2.7. Distributor

Distributor 모듈은 색인 노드로부터 색인된 데이터를 검색을 수행할 노드의 Searcher를 통해 검색할 수 있도록 색인 데이터 배포를 수행한다

2. 설정 및 실행

iRAG Search Engine의 설정 및 실행 방법은 구축 환경에 필요한 노드를 config.yaml 파일을 통해 설정하고 $IRAG_HOME/batch 디렉터리안에 있는 search-engine.sh 스크립트를 이용해 실행 및 종료시킬 수 있다.

2.1. 검색엔진 노드 설정

2.1.1. config.yaml 설정

config.yaml 파일은 iRAG Search Engine의 기본 설정 파일로 엔진에 대한 기본 정보 및 클러스터 구성 정보 등을 기입한다. 설정 파일에 기입하는 내용은 다음과 같다. security의 경우 사용하지 않을 시 기입하지 않아도 기본값으로 동작하게 된다.

[표 2.1 config.yaml 설정]

속성 (Attribute)	**설명**
environment.homePath	$IRAG_HOME path를 기입해준다. 상대 경로 또는 절대 경로 모두 사용이 가능하다. 경로 설정 시에는 큰 따옴표 없이 입력하는 것을 권장한다. - 기본경로 : 엔진 실행 위치
environment.dataPath	색인 데이터가 저장될 경로를 설정할 수 있다. - 기본경로 : homePath/data/%NODE_NAME% NAS(NFS) 디스크를 마운트해서 사용하는 경우 모든 엔진의 dataPath를 마운트된 디스크의 경로로 일치해야한다.
environment.sharedDataPath	shared 데이터가 저장될 경로를 설정할 수 있다. - 기본경로 : homePath/shared/%NODE_NAME%

environment.knowledgePath	knowledge 데이터가 저장될 경로를 지정할 수 있다. knowledgePath 하위에는 지원하는 언어분석과 관련된 korean, japanese, english, chinese 폴더가 필요하다. - 기본경로 : homePath/knowledge
system.ip	현재 서버의 IP를 의미한다. 사용하는 서버의 **실제 ip**로 사용해야 한다.
system.httpPort	API 통신 포트를 의미한다. 동일 서버에 여러 개의 노드를 사용할 경우, 포트 충돌을 막기위해 각각 다른 포트를 기입해야한다.
system.nodePort	클러스터 구성 시, 노드 간 통신에 사용하는 포트를 의미한다. 동일 서버에 여러 개의 노드를 사용할 경우, 내부 서버의 포트 충돌을 방지하기 각각 다른 포트를 기입해야한다.
system.distServerPort	Collection 전송 시 사용되는 송신 포트를 의미한다. 동일 서버에 여러 개의 노드를 사용할 경우, 내부 서버의 포트 충돌을 방지하기 위해 각각 다른 포트를 기입해야한다.
system.distClientPort	Collection 전송 시 사용되는 수신 포트를 의미한다. 동일 서버에 여러 개의 노드를 사용할 경우, 내부 서버의 포트 충돌을 방지하기 위해 각각 다른 포트를 기입해야한다.
system.searchTimeout	엔진-API 통신의 timeout 을 지정한다. 해당 시간이 넘어갈 경우, 자동으로 엔진과 API 사이에서 발생하는 통신을 취소한다.
system.clusterTimeout	node-node 통신의 timeout 을 지정한다. 해당 시간이 넘어갈 경우, 자동으로 node와 node 사이에서 발생하는 통신을 취소한다.
system.seed	클러스터 구성시, 클러스터에 최초로 접속하는 게이트 서버의 주소를 기입한다. 클러스터 구성시 2개이상의 seed node를 설정하는 것을 권장한다. 노드를 추가할 경우, seed node를 통해 클러스터로 붙게되는데, seed node를 1개로 설정할 경우, seed node가 통신이 불가능할 경우 추가 노드가 클러스터로 붙지 못하기 때문이다. 추가적으로 최초 기동시 맨 처음에 있는 seed가 뜨기 전까지 클러스터링을 구성하지 않는다. 이후는 맨 처음에 있는게 꺼져도 정상적으로 작동한다. **On-premise 환경의 경우, 최초 구성에 사용되는 모든 서버의 Address값을 를 seed 리스트에 추가 하는 것을 권장함.**

system.nodeType	engine: 검색 수행, (indexer가 없는 경우로 조건부 색인) indexer : 색인만 수행
system.sharedIndexes	NFS(NAS) 등의 디스크 환경에서 색인 파일을 공유하는 공유 색인 기능을 활성화 유무 이다. **seed값을 기준으로 앞에 있는 엔진과 설정값이 다를 경우 뒤에 있는 엔진은 종료된다.** environment.dataPath 옵션과 같이 사용하는 것을 권장한다.
system.searchCacheSize	1. 통합 검색에 포함된 alias 를 기준으로 캐시에 저장된다. 예를 들어 통합쿼리 1개에 13개의 컬렉션 쿼리가 포함시, 캐시에는 13개의 request 쿼리 결과값이 저장된다. (Default = 1000) 2. 컬렉션 별 totalHitsThreshold 값을 넘는 totalHits를 가질 때 저장한다. ● 주의 - 엔진이 사용할 수 있는 메모리에 따라 동적으로 잘 선택해줘야 한다. 너무 많은 결과나 큰 결과셋을 처리할 때는 캐시에 저장되는 데이터 양을 제한하여 메모리 사용을 최적화하는 작업이 필요하다. - 캐시에 저장된 값은 **검색 결과가 변경될 수 있는** 사전, analyzer 업데이트 및 색인 commit 작업 이후에 초기화된다.
maximunFrameSize	API를 통해 한번에 보낼수 있는 최대 데이터 사이즈를 의미한다. (Default = 10,240 KiB)
maximumLargeFrameSize	노드간 주고 받을 수 있는 최대 데이터 사이즈를 의미한다. (Default = 81,920 KiB)
numOfSearchers	샤드당 최대 검색 스레드 수 (Default = 20)
security.enableAuthorization	API 제한 사용 여부, true로 설정할 경우 밑에 옵션 2개 중 하나는 설정해야 한다. (Default = false)
security.secret	API 요청 시 header에 포함해야 할 key value를 기입한다. (secret만 있을 경우 key value만 체크한다.)
security.whitelist	API 제한으로부터 제외할 IP 목록을 기입한다. (whilelist만 있을 경우 허용 ip만 체크한다.)

[config.yaml 설정 예제]

```
  environment:
    #상대 경로와 절대 경로 모두 가능
    homePath: home1

  system:
    ip: "x.x.x.x"
    httpPort: 9001
    nodePort: 2551
    distServerPort: 2221
    distClientPort: 2222
    seed: [
      "x.x.x.x:2551",
      "x.x.x.x:2552"
     ]

 security:
    enableAuthorization: "true"
    secret: "secretKey"
     whitelist: [ "x.x.x.x" ]

  #  maximumFrameSize: 20480 #KiB (default 10,240)
  #  maximumLargeFrameSize: 163840 #KiB (default 81,920)
  # #numOfSearchers: 40 (default 20)
```

2.2. 검색엔진 실행 및 종료

본 제품에 포함된 모듈 실행을 위해서는 환경 변수 'IRAG_HOME'에 제품이 설치된 ROOT의 절대 경로가 필요하다.

2.2.1. 엔진 실행

엔진의 구동은 $IRAG_HOME/batch/search-engine.sh 을 통해 구동이 가능하며, 실행할 노드 이름을 임의로 기입한다. 구동 방법은 아래와 같다.

[engine 노드 실행 예제]

```
cd $IRAG_HOME/batch
./search-engine.sh ../config/config.yaml node1 start
```

2.2.2. 엔진 종료

엔진의 구동과 동일하게 $IRAG_HOME/batch/search-engine.sh 을 통해 구동이 가능하며, 구동 방법은 아래와 같다.

[engine 노드 중지 예제]

```
cd $IRAG_HOME/batch
./search-engine.sh node1 stop
```

엔진 종료 후, 바로 실행 스크립트를 구동시키면, http port connection 에러가 발생할 수가 있다. 이는 OS상에서 포트를 정상적으로 종료하는데 시간이 다소 걸리기 때문에 발생되는 현상으로, 조금 시간을 두고 다시 실행시키면 정상적으로 엔진이 구동되는 것을 확인 할 수 있다.

2.2.3. 엔진 상태 조회

엔진의 구동과 동일하게 $IRAG_HOME/batch/search-engine.sh 을 통해 구동이 가능하며, 구동 방법은 아래와 같다.

[engine 노드 상태 조회 예제]

```
cd $IRAG_HOME/batch
./search-engine.sh node1 status
```

3. 수집

3.1. Bridge(수집기) 소개

Bridge(수집기)는 검색의 대상이 되는 자료를 수집하는 역할을 수행한다. 또한 그 기능은 크게 자료 수집, 자료 변환, 자료 저장, 상황 별 수집 기능으로 나눌 수 있으며, 이에 따라 여러 종류의 Bridge(수집기)가 존재한다. 즉, 하나의 Bridge(수집기)가 모든 검색 대상 자료를 수집할 수 없다.

자료 수집의 경우 자료를 저장하고 있는 시스템이 어떤 종류인가에 따라 Bridge(수집기)의 종류가 다르다. 예를 들어 시스템이 Oracle, MS-SQL등의 DBMS, 일반 웹문서, 일반적인 파일서버에 저장되어 있는 각종 파일들인 경우는 Standard Bridge(표준 수집기)를 사용하고 IBM Lotus Notes Domino 솔루션인 경우는 Notes Bridge(노츠 수집기)를 사용하여 검색 대상 자료를 수집한다. 즉 검색 대상 자료가 있는 각 시스템에 적합한 각기 다른 Bridge(수집기)로 접속을 시도하여 자료를 수집해야 한다.

Standard Bridge(표준 수집기)는 기본적으로 제공이 되며 나머지 Bridge(수집기)는 별도로 구매를 필요로 한다.

[표 3.1 Bridge(수집기)의 종류]

Bridge(수집기)	**수집 대상**
Standard Bridge (표준 수집기)	상용 RDBMS/웹 문서/파일 시스템의 비정형 문서 수집 JDBC Driver를 지원하고 SQL문으로 접근 가능한 모든 종류의 시스템(그룹웨어, EDMS, KMS 등) 수집
LDD Bridge (노츠 수집기)	IBM Lotus Notes Domino 시스템 수집 (별도 구매)
Moss Bridge (MS Share Point 수집기)	Microsoft Share Point Portal 시스템 수집 (별도 구매)

AD Bridge (MS Active Directory 파일 수집기)	Microsoft Active Directory로 관리되는 공유파일 수집 (별도 구매)
LDAP Bridge (LDAP 파일 수집기)	LDAP으로 관리되는 공유파일 수집 (별도 구매)
XML Bridge (XML 파일 수집기)	XML 파일 수집 (별도 구매)
I-ON Bridge (아이온 수집기)	I-ON CMS 시스템 수집 (별도 구매)
ON-Nara Bridge (온나라 수집기)	정부 업무 처리 전산화 시스템인 On-Nara BPS 수집 (별도 구매)
Exchange Bridge (익스체인지 수집기)	Microsoft Exchange Server 메일 수집 (별도 구매)

자료 변환의 경우 각 시스템에서 수집한 데이터는 각각의 시스템 환경에 따라서 매우 다를 수 있기 때문에 표준화된 형태로 변환이 필요하다. 또한 첨부 파일 형태의 비정형 문서들도 존재할 수 있기 때문에 Document Filter(문서 필터) 모듈이 Bridge(수집기)와 함께 사용된다.

Note.
Document Filter(문서 필터)가 지원하는 문서 종류 ? MS Word (DOC, DOCX), MS PowerPoint (PPT, PPTX), MS Excel (XLS, XLSX), 아래아한글 (HWP), Adobe Acrobat (PDF), Open Office, Autodesk Drawing File (DWG), Flash Movie File (SWF), 압축파일 (ZIP, TAR), XML/HTML, MHT, CHM, EML, MIME, MP3 TAG, Rich Text Format (RTF), 핸디소프트 아리랑 (HWD), 일태랑 (JTD), Microsoft Document Imaging (MDI), Microsoft Outlook Message (MSG), 워드퍼펙트 (WPD) 4.x 13.x, 유니코드(UCS-2, UCS-4, UTF-8, BIG-ENDIAN, LITTLE-ENDIAN) 텍스트 파일, 훈민정음 (GUL) ※ 보라워드 (BWP)와 정음글로벌(GUL)파일은 별도의 문서 필터 구매가 필요하다. ※ 위의 언급된 훈민정음(GUL)은 정음글로벌 이전 버전이며, 해당 문서 필터링을 위해서는 기본 제공되는 Document Filter(문서 필터) 모듈 이외에 별도의 필터 모듈(DocFilter)이 필요하며, 이는 필요 시에 제공된다. 단, 다음의 서버 OS환경에만 제공한다. ● Windows 2000/2003/2008 32bit (64bit OS에서는 32bit로 동작) ● IBM AIX 4.3 ● Sun Solaris 5.8/5.9 ● HP-UX 11.00

- Linux Redhat gcc 2.9x ~ 3.2 (Redhat 7/8/9)

자료 저장의 경우 위와 같이 수집 변환된 데이터를 Indexer(색인기)가 인식할 수 있는 파일 구조의 형태로 저장이 되어야 한다. irag의 Indexer(색인기)는 별도의 SCD(Structured Crawled Document)라고 하는 형태의 파일 구조를 지원한다.

Note

SCD는 Structured Crawled Document를 의미하며 Element, Content의 쌍으로 구성된 텍스트 파일이다. SCD는 Bridge(수집기)에서 색인 시 필요한 Type(Insert, Delete, Update, Replace)구분되어 저장된다.

- ✔ Insert: 신규 데이터
- ✔ Delete: 삭제 데이터
- ✔ Update: 변경 데이터
- ✔ Replace: 숫자 및 간단한 문자에 해당하는 상시 변경 데이터(예:상품가격, 조회수 등)

[SCD 파일의 예]

```
<DOCID>0001
<DATE>201720170605093000
<TITLE>정보검색 엔진
<BODY>코리아와이즈넛의 검색 엔진 iRAG Search Engine은 매우 뛰어난 성능의
엔진으로 최고의 품질을 자랑한다.
<AUTHOR>홍길동
```

마지막으로 초기에 검색 시스템을 구축하는 경우와 운영 중인 경우, 수집 방식이 상이할 수 있다. 보통에 경우 초기 시스템 구축 시에는 정적 수집(Static Crawl)이라는 방식으로 전체 자료를 수집한다. 그리고 운영 중 사용자들의 자료 추가, 수정, 삭제에 따른 변화 부분만을 수집할 수 있도록 하는 동적 수집(Dynamic Crawl)으로 구분된다.

3.2. 수집 설정

3.2.1. DB Bridge 수집 설정

DB Bridge는 가장 일반적인 Bridge로 상용 RDBMS로의 연결을 지원한다. 또한 수집 Data에 첨부 파일이 포함되어 있는 경우, 파일의 내용까지 수집하므로 첨부 파일 검색이 가능하다. DB Bridge는 JDK(Java Development Kit) 1.5 이상 및 JDBC Interface를 사용하여 동작된다.

3.2.1.1. DataSource 설정

DB Bridge에서는 DBMS 접속 정보를 DataSource라는 별도의 설정 파일로 분리하여 관리한다. DBMS 접속 정보가 변경되는 경우에는 datasource.xml만 수정하면 된다. DataSource가 여러 개일 경우에는 *<Database/>*항목을 반복해서 설정함으로써 지원이 가능하며, 용자는 원하는 ID를 입력함으로써 해당 DataSource를 사용할 수 있다. 또한, DBMS 접속 정보를 암호화하는 기능을 사용하면 접속 정보의 유출을 방지할 수 있다. 암호화 모듈의 실행 방법은 “3.2.4.2. WNEncrypt”에서 설명한다.

datasource.xml의 설정은 다음 항목들로 구성되어 있다.

주의.
Data Source는 반드시 $IRAG_HOME/config/datasource.xml 파일로 존재해야 한다.

[DataSource 설정파일의 설정 예제]

```
<xml version="1.0" encoding="UTF-8">
<DataSource>
<Database>
<Id>db1</Id>
<CharSet>8859_1,euc-kr </CharSet>
<Vendor>mssql</Vendor>
<SID></SID>
<DatabaseName>mknews</DatabaseName>
<User>sa</User>
```

```
<Password>sqladmin</Password>
<PortNumber>1433</PortNumber>
<ServerName>192.168.10.1</ServerName>
<Driver>sqljdbc.jar</Driver>
<ClassName>com.microsoft.sqlserver.jdbc.SQLServerDriver</ClassName>
</Database>
<Database>
<Id>db2</Id>
 … 생략 …
</Database>
</DataSource>
```

3.2.1.2. DataSource.Database

[표 3.2 DataSource.Database 설정]

속성 (Attribute)	**설명**
Id	Data Source를 구별하는 ID 값. 브리지가 참조하는 설정 파일의 DSN에서 사용하는 id가 된다.
CharSet(옵션)	DataBase의 문자집합을 변경하고자 하는 문자집합으로 설정. 8859_1,euc-kr와 같이 설정하면 DataBase로부터 전달받은 문자열의 데이터 인코딩을 iso-8859-1에서 euc-kr로 변경한다.
SID(옵션)	Oracle의 경우에만 사용한다. Oracle의 SID를 의미한다.
Vendor	DBMS Vendor를 말함. 지원하는 Vendor는 다음과 같다. ● oracle ● oracle_oci ● mssql ● informix ● db2 ● sybase ● mysql ● cubrid ● as400 ● access ● postgre ● altibase

	● tibero ● derby
DatabaseName (옵션)	사용할 DataBase 이름이다. Oracle의 경우 표기하지 않는다. ● Informix : (ex) gw65 : informixserver = ids940)
User	사용자 계정
Password	사용자 암호
Port Number	DBMS에서 사용하는 PORT
ServerName	DBMS Server IP 또는 URL
Driver	Bridge 실행에 필요한 JDBC 드라이버 파일명
Url (옵션)	JDBC 접속을 URL방식으로 사용하고자 할 때 설정한다. ● 사용 예 : jdbc:oracle:thin:@hostname:1521:dbname
ClassName	DBMS가 제공하는 JDBC 드라이버 명을 아래와 같이 명시한다. ● Oracle : oracle.jdbc.driver.OracleDriver ● oracle_oci : oracle.jdbc.driver.OracleDriver ● mssql : net.sourceforge.jtds.jdbc.Driver ● Informix : com.informix.jdbc.IfxDriver ● db2 : com.ibm.db2.jcc.DB2Driver ● sybase : net.sourceforge.jtds.jdbc.Driver ● mysql : com.mysql.jdbc.Driver ● cubrid : cubrid.jdbc.driver.CUBRIDDriver ● as400 : com.ibm.as400.access.AS400JDBCDriver ● access : sun.jdbc.odbc.JdbcOdbcDriver ● postgre : org.postgresql.Driver ● altibase : Altibase.jdbc.driver.AltibaseDriver ● tibero : com.tmax.tibero.jdbc.TbDriver ● derby : org.apache.derby.jdbc.ClientDriver

3.2.1.3. Config 설정

DB Bridge의 설정 파일은 모두 Source Node에서 이루어진다.

3.2.1.3.1. FtServer 설정 (첨부 파일 FilterServer)

FilterServer(문서 필터 서버)는 수집 대상이 되는 Data의 첨부 문서가 원격지 디렉터리에 존재할 경우(일반적으로 웹 서버가 설치된 장비), 첨부 문서를 Filtering하여 검색 서버에서 실행 중인 Bridge로 Filtering된 텍스트 내용을 전달해주는 역할을 한다. Data Source의 설정과 마찬가지로 FilterServer 설정의 경우도 여러 대의 FilterServer를 미리 설정해놓고 *<Source>*의 Filter 설정에서는 FilterServer ID와 FilterServer의 정보를 매칭시키면 된다. 따라서 FilterServer에 설정된 IP나 PORT 정보가 변경되더라도 각각의 Source에 설정된 Filter 설정에는 영향을 미치지 않는다. 또한 DB Bridge에서는 첨부 문서가 위치한 System이 이중화되어 있는 경우여도 각각의 서버에 설치된 FilterServer로의 연결이 가능하다.

[설정파일의 FilterServer 설정 예제]

```
...
<FtServer>
<server>
<Id>Filter1</Id>
<ServerName>192.0.10.1</ServerName>
<ServerName>192.0.10.2</ServerName>
<PortNumber>21020</PortNumber>
</server>
        <server>
<Id>Filter2</Id>
<ServerName>192.0.10.3</ServerName>
<PortNumber>21020</PortNumber>
</server>

</FtServer>
<Source>
        ...
<Filter order="3" filter="Filter1" ... />
        ...
</Source>
...
```

[표 3.3 FilterServer의 접속 설정]

항목 (Element)	값 (Value)	설명
server.Id	문자열	FilterServer를 구분하는 값을 지정
server.ServerName	문자열	FilterServer가 설치된 장비의 IP 주소를 지정 FilterServer가 이중화되어 있는 경우 다중으로 설정이 가능한 항목이다.
server.PortNumber	숫자	FilterServer가 DB Bridge의 접속을 대기하는 포트 다중으로 설정할 수 없다.

3.2.1.3.2. Source 설정

Bridge 실행에 필요한 설정은 설정 파일의 *Source* 에 작성한다. *Source* 설정은 SubQuery, Filtering, FtServer, Dynamic 등 Bridge 실행에 중요한 인자들을 포함하고 있기 때문에 작성 시 주의가 필요하다. *Source* 는 아래와 같은 형태로 구성된다.

[설정 파일의 Source 설정 예제]

```
<Source id="news" type="DB" sourcealias="daily" extension="json">

<RefColl id="news" prefix="#01"/>
<SCDdir path="C:\irag\collection\news\scd\"/>
<Date dbformat="none" order="1"/>

<DSN type="target" dsn="db1"/><!--Taget DataBase Info-->
<DSN type="log"    dsn="db1"/><!-- Log DataBase Info -->

<DateTime/>
<CatalogInfo/>
<SubMemory/>
<SubQuery/>
<FilterInfo/>
<Init/>
<Static/>
<Dynamic/>
<MasterTable/>
<PrimaryKey/>
```

</Source>

[표 3.4 Source 설정]

항목 (Element)	속성 (Attribute)	값 (Value)	설명
Source	id	문자열	수집 설정을 구분하는 값을 지정. DB Bridge를 실행할 때 사용하는 옵션인 srcid이기도 함.
	type (옵션)	db web file	해당 Source가 DB에서 정보를 가져온다는 것을 의미
	sourcealias (옵션)	문자열	SCD 파일에 해당 source의 출처가 어디인지 구분하기 위해 sourcealias를 넣어 줄 때 사용.
	charset (옵션)	utf-8 euc-kr shift-jis euc-jp gb2312	SCD 파일을 구성하고 있는 문자 집합을 설정. 기본값은 utf-8이며 euc-kr, shift-jis와 같이 설정한다.
	extension (옵션)	json scd	수집된 파일 포맷을 JSON 또는 SCD로 설정한다. (default : json)
Refcoll (옵션)	id	문자열	해당 Source가 참조하는 Collection ID를 지정. 여러 개의 Source가 하나의 Collection을 참조할 수 있음.
	prefix	문자열	Collection에 참조하는 Source가 여러 개일 경우 유일한 문서 구분 ID인 DOCID가 중복될 수 있음. 중복을 방지하기 위해서 #t#과 같은 문자를 사용.
Date (옵션)	dbformat	문자열 (날짜 변환 설정 참고)	DataBase의 Date Column을 색인기가 인식하는 형태(Format)으로 변경시켜주는 역할을 함.
	order	숫자	SCD 항목의 index 번호를 나타냄.
DSN	type	target log	수집[또는 검색]대상(Target) DataBase와 수집된 정보(Log)를 기록하는 DataBase의 접속 정보를 맵핑(Mapping)하는 정보를 가지고 있음. type은 target, log 두 가지 type이 올 수 있음

	dsn	문자열	dsn은 Data Source에서 설정한 Id을 적어줌으로써 datasource.xml의 DataBase 정보와 맵핑(Mapping)된 정보를 얻어옴.
CatalogInfo			SCD를 구성하는 필드를 설정하는 노드
SCDdir	path	문자열	SCD가 저장되는 디렉터리 경로를 절대 경로로 지정 예) /home/wisenut/irag/collection/news/scd
SubMemory (옵션)			서브쿼리에 데이터를 미리 메모리에 올려서 메모리 조회를 통해 데이터를 수집하는 설정. 자세한 설명은 SubMemory Part 참조.
SubQuery (옵션)			SubQuery 및 재귀 호출 Query에 대한 설정을 하는 부분. 자세한 설명은 SubQuery Part 참조.
FilterInfo (옵션)			Filtering 과정 관련 설정을 하는 부분과 FilterServer를 통한 Filtering에 대한 자세한 설명은 FilterInfo 참조.
MasterTable (옵션)			통합게시판과 같이 수집대상 데이터가 일괄적으로 관리되는 경우에 설정. 즉, 각 개별 Table을 관리하는 목적 Table에 Table명이 저장되는 필드를 설정.
Init (옵션)			DB Bridge가 init 모드로 실행될 때 수행될 Query를 작성.
Static			DB Bridge가 Static 모드로 실행될 때 Log Table 또는 Queue Table을 사용하지 않고 대상 Data를 모두 수집하는 Query를 작성.
Dynamic (옵션)	mode	logdb queue	추가/수정/삭제된 Data를 수집하기 위한 Log Table 또는 Queue Table을 설정하는 부분. 자세한 설명은 Dynamic Node Part 참조.
	seqno	y/n	Queue Table을 사용할 때 설정하는 옵션으로 sequence number를 사용할 것인지 여부를 seqno="y"와 같이 설정
PrimaryKey			SCD에 작성된 <DOCID>에 해당하는 수집되는 대상 Table의 Primary Key Field를 설정.

DateTime	path	문자열	동적 수집용 SQL Query의 날짜조회 기준 값을 저장하기 위한 properties파일의 절대 경로를 *<DateTime path="">*에 설정. 자세한 설명은 Dynamic Node Part 참조

Bridge는 *Source.CatalogInfo*를 통해 SCD구조를 생성하고 DB의 Data를 수집한다. 다음은 *Source.CatalogInfo*의 설정 예제이다.

[설정 파일의 *Source.CatalogInfo* 설정 예제]

```
<CatalogInfo>
<Mapping colnum="0" tagname="DOCID"/>
<Mapping colnum="1" tagname="source"/>
<Mapping colnum="2" tagname="seed"/>
<Mapping colnum="3" tagname="category" replace="y"/>
<Mapping colnum="4" tagname="Date"/>
<Mapping colnum="5" tagname="url"/>
<Mapping colnum="6" tagname="title" append=""/>
<Mapping colnum="7" tagname="title_keys" convert=""/>
<Mapping colnum="8" tagname="content" html="y" className="0x000"/>
<Mapping colnum="9" tagname="name"/>
<Mapping colnum="10" tagname="alias" alias="y"/>
</CatalogInfo>
```

[표 3.5 Source.CatalogInfo설정]

항목 (Element)	**속성 (Attribute)**	**값 (Value)**	**설명**
Mapping	colnum	숫자	SCD 항목의 index 번호. 0부터 시작.
	tagname	문자열	SCD 항목의 이름.
	alias (옵션)	y/n	alias="y"인 경우 *<Source sourcealias="">*로 설정한 값을 SCD의 해당 Field에 저장. 즉, 예를 들면 *<Source id = "news" sourcealias= "daily">*로 설정하였다면, 이 경우 'daily라는 값을 SCD의 alias필드에 저장.

	html (옵션)	y/n	html="y"로 지정. SCD 항목 중 index 번호에 해당하는 db컬럼의 데이터가 HTML 문서일 경우 HTML Parsing을 하여 SCD의 해당 Field에 저장.
	append (옵션)	숫자	append="2"로 지정. DB테이블의 컬럼과 컬럼을 합쳐서 하나의 SCD Field 값으로 저장해야 할 경우, SQL Query에서 결합 문자를 사용하지 않고 append="2" 라는 옵션을 추가함으로써 해당 Colnum부터 숫자만큼 append한 값을 합침
	convert (옵션)	문자열	수집하는 Field의 Data 문자 집합을 변환할 수 있는 설정 옵션 (예) convert="8859_1, EUC-KR"으로 지정했다면, 8859_1 Data를 EUC-KR로 문자 집합을 변경.
	replace (옵션)	y/n	replace="y"로 지정. R Type으로 색인하기 위하여 SCD에 <DOCID> 값과 replace로 지정된 Field의 값만을 기록. 단, 동적 수집을 위한 table에 R type에 해당하는 이벤트 값이 저장 되도록 환경을 준비.
	className (옵션)	문자열	커스터마이징이 필요한 부분을 정의하여 사용하는 클래스를 className="kr.co.wisenut.bridge3.job.custom.WSParseXML "과 같이 설정.
	maxLen (옵션)	숫자	수집하는 데이터의 최대 길이를 지정.
	md5 (옵션)	y/n	<DOCID>필드의 값을 MD5로 암호화 할 때 지정.

다음은 앞의 *Source.CatalogInfo* 설정 예제를 이용하여 DB Bridge를 실행시켰을 때 생성된 SCD의 내용이다.

[SCD 파일 내용]

```
<DOCID>#01116645
<source>1005
<seed>7665
<category>ZZ J3
<Date>20110701045950
<url> http://www.moneytoday.co.kr/sokbo/2004/07/31/all2004073119172301301.html
<title> 태풍 '남테우른' 북상…… 오늘밤부터 남부지방 영향권
<title_keys> 태풍 남테우른 북상 남부 지방 영향권
```

```
<content>일본 오사카 지방에 상륙한 제 10호 태풍 '남테우른'이 한반도로 북상 중 이어서 31일 밤부터 남해 동부와 동해 남부 해상이 태풍의 영향권에 들고 경상남도 지방부터 비가 오겠다.
<name>와이즈넛 뉴스
<alias>daily
```

3.2.1.4. 주요 항목 상세 설명

주요 항목 상세 설명에서는 *<Source>* 하위의 설정을 보다 자세히 설명한다.

3.2.1.4.1. 날짜(Date)변환 설정

[예제]

```
<Date dbformat="yyyyMMddHHmmss" order="1"/>
```

*<Source/>*에 위와 같은 설정을 할 수 있다. Date는 Table의 Column 값 중에 날짜(date)형식으로 들어 있는 값을 색인기가 인식할 수 있는 형태(Format)으로 변경하는 역할을 하게 된다.

[표 3.6 날짜 변환 설정 설명]

속성 (Attribute)	**값 (Value)**	**설명**
order	scd column index	SCD Field에서 Date의 index 번호.
dbformat	Date Format (표 3.7 참조)	이 설정은 DB Bridge에서 설정한 특성 값을 none_time_t 형식으로 변경을 해주는 역할을 하게됨. 예를 들어 날짜 Field에 2011-01-01으로 값이 들어 있을 경우 db형태(Format)="yyyy-mm-dd" 형식으로 주면 2011-01-01값을 20110101000000 형식으로 변환함.

다음 표는 DB Date 형태(Format)에서 지원되는 형식이다.

표에 있는 DB형태(Format)과 Table Data형식이 일치하지 않는 경우, Query문을 통해 지원하는 형식으로 변환하면 된다.

[표 3.7 날짜 변환 설정]

속성 (Attribute)	Data 형식	변환 후 형식
dbformat	none	Type 변환을 하지 않고 Table에서 읽은 값을 그대로 SCD에 기록.
	yyyy OR YYYY	yyyyMMddHHmmss로 변환.
	yyyymmdd	
	yyyymmddhhmm	
	yyyymmddhhmi	
	yyyymmdd:hh:mm:ss	
	yyyymmdd:hh:mi:ss	
	yyyy-mm-dd	
	yyyy.mm.dd	
	yyyy/mm/dd	
	yyyy-mm-dd hh:mm:ss	
	yyyy-mm-dd hh:mi:ss	
	yyyy/mm/dd hh:mm:ss	
	yyyy/mm/dd hh:mi:ss	

3.2.1.4.2. DSN

DSN 설정은 datasource.xml의 ID을 적어줌으로써 수집대상 Data Base Connection 정보를 얻어오게 된다. dsn="data_source_id" 설정을 하면 된다.

type = "target"은 반드시 있어야 하며, 주의할 점으로는 type="log"는 LogTable을 이용하여 Dynamic을 구현할 경우와 MySQL인 경우에만 작성하도록 한다.

[예제]

```
<DSN type = “target” dsn = “data_source_id” /><!- 반드시 설정되어야 한다.>
<DSN type = “log” dsn = “ data _source_id” /><!- LogTable을 이용하여 Dynamic을 구현할 경우에만 적어준다.>
```

3.2.1.4.3. SubQuery 설정

*SubQuery*는 다른 하나의 sql문장의 절에 중첩된 select문장을 사용하여 SCD의 특정필드를 생성하는데 큰 역할을 한다. 이 기능은 일반적으로 SQL Query문을 통해서는 여러 개의 결과 값을 기록하는 열(Row)을 생성하고 그 모든 값을 SCD 필드에 저장하는 경우에 *SubQuery*를 사용한다. *SubQuery*의 조건절(WHERE)은 *Static/Dynamic*의 SQL에서 전달해주는 Field의 결과 값을 이용한다.다음은 *SubQuery* 설정의 예이다.

[표 3.8 SubQuery 설정]

항목 (Element)	**속성 (Attribute)**	**값 (Value)**	**설명**
Sub	order	SCD Index 번호	ResultSet에서 읽어 들인 값이 들어가는 SCD Index 번호.
	type	subQuery reQuery connectBy append	SubQuery를 수행하는 것을 명시. ▪ subQuery : 일반적인 subquery로 동작. ▪ reQuery : 재귀 Query로 동작. ▪ connectBy : 계층 Query로 동작. ▪ append : 1개 이상의 Field를 수집할 때 동작.
	seperator	구분자	여러 개의 결과가 있을 경우 구분자로 구분이 가능하도록 구분자를 얻음.
	maxCount	결과 개수	이 connectBy일 경우 결과 최대 개수를 지정.
	mode	static dynamic	해당 SubQuery가 DB Bridge 실행 모드에 따라서 적용.
	wherecnt	조건절에 사용되는 값	where=“1”으로 지정. SQL의 where 조건의 개수.
	className	문자열	커스터마이징이 필요한 부분을 정의하여 사용하는 클래스를

			className="kr.co.wisenut.bridge3.job.custom.WSParseXML "과 같이 설정.
Sub.sql	statement	y/n	statement ="y"로 지정. Bind Variable를 이용하지 않고 동작.
		SQL Query문	SubQuery 할 Query를 입력.

[예제]

```
<SubQuery>
<Sub order="3" type="subQuery" wherecnt="1" seperator="++"
    mode="" maxCount="">
<sql statement="n">
        select categoryname from code_category where categorycode=?
</sql>
</Sub>
</SubQuery>
```

위 Query문을 통해 작성 방법을 알아보도록 하겠다.

order ="3" 은 CatalogInfo의 column ="3"에 SubQuery한 값을 넣는다는 의미이다. type ="subQuery" 는 네 가지 type중 subQuery를 수행 한다는 의미이다. DB Bridge에서의 모든 Query문의 조건 절은 Bind Variable를 사용한다. 즉 Statement를 사용하지 않고 Prepared Statement를 사용한다. Prepared Statement 사용시엔 Statement를 사용할 경우와 비교해 DBMS의 자원 효율 및 성능을 향상시킬 수 있다.

[예제]

```
select user_id, user_name from user_info where pid=?
```

Bind Variable를 사용한다는 것은 위 예제와 같이 Query 작성 시, 조건절에 들어가는 변수로 물음표(?)를 사용하여 해당 값을 설정하는 것이다. 따라서 SubQuery의 조건절로 들어가는 부분의 값도 물음표(?)로 사용해야 한다. 하지만 Statement만 사용해야 하는 경우에는 statement="y" 옵션을 활성화하고 다음의 예제처럼 Query를 변경하도록 한다.

[예제]

```
select user_id, user_name from user_info where pid=%s1
```

3.2.1.4.4. SubMemory 설정

*SubMemory*는 서브쿼리 메모리에 데이터를 미리 올려서 메모리 조회를 통해 수집 속도를 개선할 수 있는 설정이다. 다음은 *SubMemory* 설정의 예이다.

[예제]

```
<SubMemory>
    <MemoryTable id="mt1" keycount="2" usecount="2" keyseperator="^" fileyn="y">
<sql><![CDATA[
        select seed_id,seed_id, seed_name, seed_url from submemory_SUB1
        ]]>
        </sql>
</MemoryTable>
<MemoryTable id="mt2" keycount="2" usecount="2" keyseperator="^" fileyn="y">
<sql><![CDATA[
        select seed_id,seed_id, seed_name, seed_url from submemory_SUB2
        ]]>
          </sql>
</MemoryTable>
<MemorySelect order="2" refmemorytableid="mt1" useidx="1"
classname="wisenut.CustomData" seperator="!"/>
<MemorySelect order="4" refmemorytableid="mt2" useidx="1" seperator="^"/>
</SubMemory>
```

MemoryTable 은 메모리에 올릴 데이터를 설정한다. 여러 개의 *MemoryTable* 설정이 가능하며 id 값으로 구분한다.

[표 3.9 MemoryTable 설정]

항목 (Element)	**속성 (Attribute)**	**값 (Value)**	**설명**
MemoryTable	id	문자열	*MemoryTable*을 구분하는 고유한 값 지정
	keycount	필드 개수	키로 사용되는 필드 개수 (쿼리에서 맨 처음의 필드부터의 순서개수)
	usecount	필드 개수	데이터로 사용되는 필드 개수

			(쿼리에서 키로 사용되는 필드 이후의 나머지 필드의 순서 개수)
	keyseperator (옵션)	구분자	key 에 사용되는 필드가 2개 이상일 경우 필드 병합자 (key 로 사용되는 필드를 합쳐서 메모리의 key로 설정)
	fileyn (옵션)	y/n	메모리에 로드된 데이터를 파일로 저장하는 기능 디버깅 용도로 사용함
MemoryTable.sql		SQL Query문	Query 문 입력.

MemorySelect 는 메모리 데이터와 SCD 항목을 매핑한다. 이 설정으로 SCD 항목에 메모리 데이터를 수집할 수 있다.

[표 3.10 MemorySelect 설정]

항목 (Element)	**속성 (Attribute)**	**값 (Value)**	**설명**
MemorySelect	order	SCD Index 번호	ResultSet에서 읽어 들인 값이 들어가는 SCD Index 번호.
	refmemorytable id	문자열	참조되는 MemoryTable id 값. 해당 메모리를 사용하겠다는 의미.
	useidx	숫자	MemoryTable 에서 가져올 데이터의 위치 값
	seperator	구분자	가져올 데이터가 여러 row 일 경우 구분자
	classname (옵션)	클래스명	가져온 데이터를 커스텀 클래스를 이용해 재가공함

3.2.1.4.5. FilterInfo 설정

일반적으로 첨부 파일명은 특정 Table의 Field값으로 존재하며, 파일의 위치는 로컬 System에 있는 경우와 원격지 System에 있는 경우, URL 형식인 경우, 바이너리 Data 형식인 경우의 네 가지로 나뉜다. 이들은 retrieve="" 항목에서 설정한다.

1. retrieve="local" : 첨부 파일이 Bridge와 같은 local에 저장되어 있는 경우이다.
2. retrieve="FtServer" : 첨부 파일이 원격지에 있는 경우. FtServer를 이용하여 Filtering을 실시하고 추출된 문서 내용을 전달 받는다.
3. retrieve="url" : URL을 통해 첨부 파일에 접근하여 Bridge와 같은 local에 저장한 후 Filtering한다.
4. retrieve="blob" : 첨부 파일이 특정 Table의 필드에 바이너리 Data값으로 저장 되어있는 경우로 필드 타입이 blob 또는 image로 되어 있어야 한다.

[설정 파일의 *FilterInfo* 설정 예제]

```
<FilterInfo>
<Filter order="8" filter="all" retrieve="local" condition="none-chk-ext"
        split="^"  dir="d:\webapps\" wherecnt="1" usecnt="">
        <sql statement="n">
        select file_name^filename from filter where file_id = ?
        </sql>
  </Filter>
</FilterInfo>

*. BLOB(Binary LOB Type) 과 IMAGE 적용 방법
<FilterInfo>
<Filter order="20" filter="all" retrieve="blob" condition="source-delete"
        split="" dir="/nawas1/users/clickq/search/temp">
          <sql statement="n">
        select file_name, blob_file  from sj_wddata where docid = ?
</sql>
</Filter>
</FilterInfo>
```

파일명 Field와 Bridge에서 인식하는 Type이 BLOB 혹은 IMAGE인 Field는 반드시 순서대로 선택해야 한다. *<Filter/>*에 사용할 수 있는 항목은 아래 표와 같다.

[표 3.11 FilterInfo 설정]

항목 (Element)	**속성 (Attribute)**	**값 (Value)**	**설명**

Filter	order	SCD Index	Filtering 후, Data가 들어갈 SCD Index 번호.
	filter	all sn3f sn2f	첨부 문서를 Filtering할 Filter를 선택. 첨부 문서 파일명이 훈민정음 파일(확장자가 .gul)인 경우에는 지원 필터 외의 Filter를 지정하고 Custom Class를 구현. ▪ all: sn3f를 사용하여 Filtering ▪ sn2f는 하위 모듈 호환
	retrieve	local	필터링 대상 첨부 문서가 Bridge가 실행되는 로컬 디렉터리에 존재하는 경우에 설정.
		FtServer (FilterServer)	첨부 문서가 Bridge가 실행되는 System이 아닌 원격지에 System에 존재하는 경우(FilterServer가 실행되고 있는 System)에 설정.
		blob	첨부문서가 특정 Table의 컬럼에 바이너리 데이터로 저장되어 있는 경우에 설정. 단, Bridge가 인식가능한 컬럼 타입이어야 함. (BLOB, IMAGE) 이 경우 Bridge가 실행되는 서버의 로컬 디렉터리에 파일을 임시로 생성한 후 필터링 수행.
		url	수집 대상 Table의 첨부문서가 저장되어 있는 값이 URL형식으로 되어 있는 경우 지정한다. 이 경우 Bridge가 실행되는 서버의 로컬 System에 파일을 임시로 생성한 후 필터링하므로 수집서버에 DNS주소가 설정되어 있어만 웹서버의 첨부문서가 다운로드 가능. [주의] className="kr.co.wisenut.common.filter.custom.Http2FilterData"를 설정한다.
	split	구분 문자	첨부 문서가 하나 이상의 Field에서 결과를 얻어야 할 경우에 해당 Query의 결과 Field들을 연결하는 문자로 설정. 예) SELECT file1+'^'+file2 From Attach
	dir	디렉터리 경로	첨부 파일 경로가 Table에 들어 있는 경우 첨부 파일의 전체 경로가 아닌 특정 경로 이후부터 입력. 이 경우 앞의 특정 부분을 적어줌.
	Usecnt (옵션)	Field 개수	usecnt="2"로 지정. Filtering에 필요한 정보 Field 개수를 지정. (기본값 1)

	Wherecnt (옵션)	조건절에 사용되는 값	where="1"로 지정. SQL의 where 조건의 개수를 지정. (기본값 1)
	serverid	FtServer ID값	retrieve가 FtServer인 경우에 설정해주어야 하며, *FtServer* Node에 설정된 id값을 넣어 주어야 함.
	condition	조건 값	▪ none-chk-ext : Filtering 하기 전에 확장자 체크를 하지 않음 ▪ chk-ext-indexof : Filtering 하기 전에 확장자 체크 ▪ source-delete : [주의] Filtering 대상인 원본 문서를 삭제 ▪ delete-tmp : 핸디소프트의 문서를 필터링 할 경우 지정하는 옵션으로 Filtering 원문파일을 삭제
	className	Java class	커스터마이징이 필요한 부분을 정의하여 사용하는 클래스를 className="kr.co.wisenut.common.filter.custom.Http2 FilterData"과 같이 설정.
	seperator (옵션)	구분 문자	실제 SCD에 데이터를 쓸때 설정된 문자를 구분자로 사용 (기본값 " ")
Filter .sql	statement	y/n	statement ="y"로 지정. Bind Variable를 이용하지 않고 동작.
		Query문	Filtering 정보를 가져오는 Query의 값이 없는 경우는 *Static/Dynamic*의 Query문에서 Filtering 할 수 있는 정보를 넣어 주어야 함.

3.2.1.4.6. Query 구문 설정

Data 수집에 필요한 설정은 *<Init/>, <Static/>, <Dynamic/>, <MasterTable/>*에서 하게 된다. 각 Element에서 설정된 Query 구문은 DB Bridge의 실행모드(mode)인 init, static, dynamic에서 사용된다.

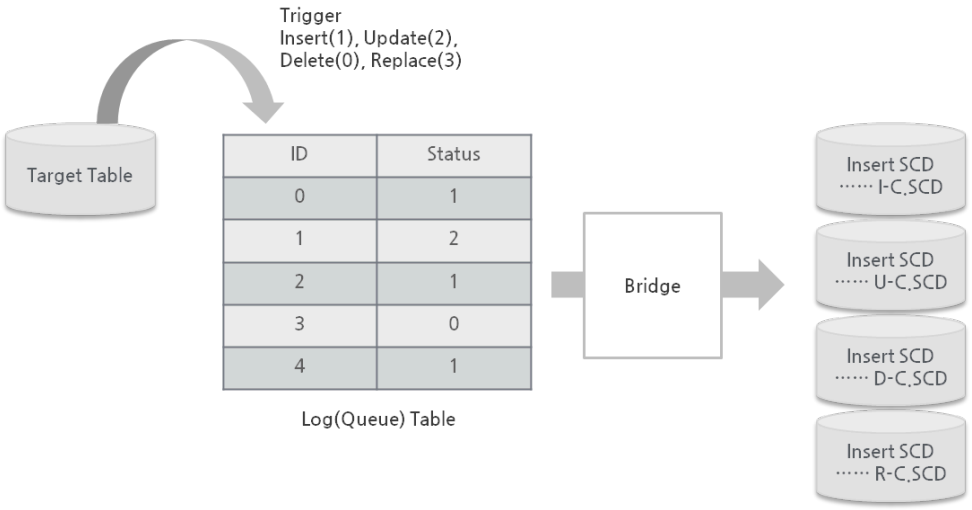
Init 설정 시엔 동적 Data를 수집하기 위해 필요한 테이블 및 인덱스 등을 생성하는 Query구문을 Execute Element 하위에 작성한다.

Static 설정 시엔 수집 대상 Data를 모두 수집하는 Query 구문을 작성한다.

Dynamic 설정 시엔 Select Element 하위에, 삽입(Insert), 수정(Update) 및 특정 Field만 수정(Replace)된 Data를 가져오는 Query 구문을 작성한다. 동적 Data를 가져오기 위해서는 로그(Log Table) 또는 큐(Queue Table)와 조인(Join) 하는 Query를 작성하거나 수집데이터의 등록일이나 수정일을 기준으로 데이터를 수집을 할 수 있다.

프로젝트의 운영 및 구현 환경에 따라 다를 수 있지만 DB Bridge는 static 모드로 Bridge가 실행되는 도중에 접속이 끊어질 경우나 수집이 완료된 정보를 남겨두어야 할 경우가 발생할 수 있다. 이러한 경우에는 문제가 발생된 시점까지 수집해 온 것을 제외한 나머지 부분들에 대해서 수집을 할 수 있어야 한다. 그리고 Dynamic 모드로 수집할 때 해당 Table의 INSERT, UPDATE, DELETE, REPLACE 사항들이 수집에 정확히 반영되어야 한다. 이와 같은 경우 정적 수집 중 발생할 수 있는 문제를 해결하고 동적(신규 추가, 수정, 삭제) Data를 수집하기 위해서는, 대상 Table에 트리거(Trigger)를 사용하고 별도의 로그(Log Table)/큐(Queue Table)를 생성하여 Data 수집을 관리해야 한다.

[그림 3.1 Dynamic 수집]



[표 3.12 실행 모드(Mode)별 설정]

항목 (Element)	속성 (Attribute)	설명
Init.Execute.sql		로그(Log Table)/큐(Queue Table)를 생성하는 Query 구문을 설정.
Static.Init.Execute	sql	정적 Data를 수집하기 전에 수행이 필요한 Query구문을 설정.
Static.Select.sql		수집대상 Data를 모두 select하는 Query구문을 설정.
Dynamic	mode	queue 또는 logdb라는 값을 지정.
	seqno	mode를 queue로 지정한 경우에 y 또는 n을 지정.
Dynamic.Init.Execute.sql		동적 Data를 수집하기 전에 수행이 필요한 Query구문을 설정. 한 개 이상의 값 설정 가능.
Dynamic.Select	type	SCD 종류인 I/U/D/R 의 4가지 값을 지정한다. I와 U는 IU와 같이 지정.
Dynamic.Select.sql		동적 Data를 select하는 Query구문을 설정.
Dynamic.Execute (옵션)	type	SCD 종류인 I/U/D/R 의 4가지 값을 지정. type이 지정된 경우 해당 type에서만 sql의 Query구문이 수행됨.
Dynamic.Execute.sql		동적 Data를 수집한 후 queue의 Data를 삭제하는 등의 Query구문을 설정.

[Query 작성 시 DOCID 생성 규칙]

DB Bridge에서 DOCID는 Table의 P.K.(Primary Key) 값으로 생성된다. 하지만 P.K 값이 여러 Field가 합쳐져서 생성되는 경우 Bridge는 여러 개의 Field를 조합하여 DOCID를 생성해야 한다. 이 경우 Bridge는 P.K Field를 지정하는 <PrimaryKey/>의 정보를 참조하여, Query로 가져오는 첫 번째 결과부터 순차적으로, PrimaryKey에 정의된 <ColName name=""/> 개수만큼 조합하여 DOCID를 생성한다.

[예] SELECT id1,id2 ,subject, content FROM bbs

3.2.1.4.7. Init 설정

[Queue Table을 이용한 예제]

```
<Init>
    <Execute>
      <sql>DROP TABLE QUEUE_TABLE</sql>
      <sql>CREATE TABLE QUEUE_TABLE (
                SEQNO CHAR(20) NOT NULL
                ,TABLE_NAME VARCHAR(30) NOT NULL
                ,SRCHKEY VARCHAR(128) NOT NULL
                ,ACTION CHAR(1) NOT NULL
               )
      </sql>
      <sql>CREATE  UNIQUE INDEX XPKQUEUE_TABLE
                        ON QUEUE_TABLE ( SEQNO )
                  </sql>
      <sql>DROP  SEQUENCE S_QUEUETABLE</sql>
      <sql>CREATE SEQUENCE S_QUEUETABLE START WITH 1
                        INCREMENT BY 1
                  </sql>
    </Execute>
</Init>
```

[Log Table을 이용한 예제]

```
<Init>
    <Execute>
      <sql>DROP TABLE IRAG_LOG</sql>
      <sql>CREATE TABLE QUEUE_TABLE (
                SRCHKEY CHAR(20) NOT NULL
                ,BRG_STC CHAR(1) NOT NULL
               )
      </sql>
      <sql>CREATE  UNIQUE INDEX IRAG_LOG_IDX
                        ON IRAG_LOG ( SEQNO )
                  </sql>
    </Execute>
</Init>
```

3.2.1.4.8. Query 구문의 조건절 변경을 통한 동적 수집

Bridge 실행 시 -p 옵션을 통해 추가 파라미터 값을 입력하면 <Select><sql> 조건절에 들어가는 값을 변경할 수 있다. -p[:digit:] <value>의 형태로 사용하며 <Static|Dynamic><Select><sql>문 조건절의 p[:digit:] 값으로 입력된다.

[실행 예]

```
C:\irag\bin>java -DIRAG_HOME=C:\irag -Dirag.ver=6 -classpath
C:\irag\bridge\bridge390.jar;C:\irag\bridge\lib\jdom.jar;C:\irag\bridge\lib\xecres.jar
kr.co.wisenut.bridge3.JBridge -conf C:\irag\config\config.xml -srcid sample_goods -mode test
-filterdel true -log stdout -debug 3 -indexdir -p1 2014 -p2 2015
```

[예제]

```
<Source id="sample" type="DB">
...
    <Static>
      <Select>
        <sql>
            select title,year,content from table
            where year >= %p1                <!-- p1의 값으로 2014 입력 -->
            where year < %p2                 <!-- p2의 값으로 2015 입력 -->
        </sql>
      </Select>
    </Static>
</Source>
```

[결과]

```
2014~2015 컨텐츠 수집
```

3.2.1.4.9. 정적(Static) 문서 수집

정적 문서 수집은 원본 Data 전부를 수집하는 방법이다. *<Static><Select><sql>*에 작성된 Query로 Bridge가 Data를 수집하게 되며, 이 경우 Data 전부를 수집해야 하므로 Log Table이나 Queue Table을 Join하지 않아야 한다.

[예제]

```
<Static>
    <Init>
  <Execute>
  <sql/>
  </Execute>
    </Init>
    <Select>
      <sql>
      <![CDATA[
                    SELECT docid,writer,createdate,subject,content FROM bbs
              ]]>
      </sql>
    </Select>
</Static>
```

3.2.1.4.10. 동적(Dynamic) 문서 수집

수집대상 데이터의 날짜 조회 조건으로 동적(Dynamic)문서를 수집하기 위한 설정이다.

[설정 예제]

```
<!-- 시간 값은 각 sql 조건에 사용될 datetime 형식을 맞춰야 함.
 저장된 값은 %DATETIME%과 치환되는데 사용됨 /-->
<DateTime path="c:\irag\collection\bbs\properties\datetime.propeties">
<Select>
<sql>SELECT now()</sql><!-- MySQL 예 -->
</Select>
</DateTime>
```

동적 수집용 SQL Query의 날짜조회 기준 값을 저장하기 위한 properties파일의 절대 경로를 *<DateTime path="">*에 설정한다. *<DateTime>*의 *<SELECT><sql>*설정은 동적 수집용 SQL Query의 날짜조회 기준값으로 사용되므로 조회(WHERE) 조건의 형식과 일치하도록 Query구문을 작성해야 한다.

3.2.1.4.11. 추가(Insert)된 문서 수집

Data의 증가분만 수집하여 변경 사항을 반영하는 방법이다. *<Dynamic><Select type="IU"><sql>*에서는 증가분의 Data를 수집하는 Query가 작성되고 Bridge는 이에 따라 수집을 수행하게 된다. 따라서 Log Table 또는 Queue Table과 Join을 해야 하며, 상태가 추가된 경우만 Query 조건에 포함시켜야 한다.

[Queue Table을 이용한 예제]

```
<Dynamic mode="queue" seqno="y">
  <Init>
  <Execute>
  <sql/>
  </Execute>
  </Init>
  <Select type="IU">
      <sql>
      SELECT b.docid,b.writer,b.createdate,b.subject,b.content,q.seq
            FROM bbs b, queue q
      </sql>
      <condition type="I">
      WHERE b.docid = q.srchkey and q.action = 'I'
      </condition>
   </Select>
</Dynamic>
```

[Log Table을 이용한 예제]

```
<Dynamic mode="logdb" seqno="n">
  <Init>
  <Execute>
  <sql/>
```

```
  </Execute>
  </Init>
  <Select type="IU">
      <sql>
      SELECT b.docid,b.writer,b.createdate,b.subject,b.content
            FROM bbs b
      </sql>
      <condition type="I">
    WHERE EXISTS
    (SELECT 'X' FROM IRAG_LOG sl WHERE sl.srchkey = b.docid and sl.brg_stc='1')
      </condition>
    </Select>
</Dynamic>
```

[날짜조회 조건 중에서 등록일을 이용한 예제]

```
<Dynamic mode="datetime" seqno="n">
  <Init>
  <Execute>
  <sql/>
  </Execute>
  </Init>
  <Select type="IU">
      <sql>
      SELECT b.docid,b.writer,b.createdate,b.subject,b.content
            FROM bbs b
            WHERE delete_falg='N'
      </sql>
      <condition type="I" >
          AND createdate >= '%DATETIME%'
      </condition>
    </Select>
</Dynamic>
```

3.2.1.4.12. 수정(Update)된 문서 수집

수정된 Data만 수집하여 변경 사항을 반영하는 방법이다. *<Dynamic><Select type="IU"><sql>*에서는 수정된(Update) Data를 수집하는 Query가 작성되고 Bridge는 이에

따라 수집을 수행하게 된다. 따라서 Log Table 또는 Queue Table과 Join을 해야 하며, 상태가 변경된 경우만 Query 조건에 포함시켜야 한다.

[Queue Table을 이용한 예제]

```
<Dynamic mode="queue" seqno="y">
  <Init>
  <Execute>
  <sql/>
  </Execute>
  </Init>
  <Select type="IU">
      <sql>
      SELECT b.docid,b.writer,b.createdate,b.subject,b.content,q.seq
          FROM bbs b, queue q
      </sql>
      <condition type="U">
      WHERE b.docid = q.srchkey and q.action = 'U'
      </condition>
</Select>
</Dynamic>
```

[Log Table을 이용한 예제]

```
<Dynamic mode="logdb" seqno="n">
  <Init>
  <Execute>
  <sql/>
  </Execute>
  </Init>
  <Select type="IU">
      <sql>
      SELECT b.docid,b.writer,b.createdate,b.subject,b.content
          FROM bbs b
      </sql>
      <condition type="U">
      WHERE EXISTS
      (SELECT 'X' FROM IRAG_LOG sl WHERE sl.srchkey = b.docid and sl.brg_stc='2' )
      </condition>
  </Select>
</Dynamic>
```

[날짜조회 조건 중에서 수정일을 이용한 예제]

```
<Dynamic mode="datetime" seqno="n">
  <Init>
  <Execute>
  <sql/>
  </Execute>
  </Init>
  <Select type="IU">
      <sql>
      SELECT b.docid,b.writer,b.createdate,b.subject,b.content
            FROM bbs b
            WHERE delete_falg='N'
      </sql>
      <condition type="U">
          AND modifydate >= '%DATETIME%'
</condition>
    </Select>
</Dynamic>
```

3.2.1.4.13. 변경(Replace)된 Field 수집

수정된 Data의 모든 결과 Field를 수집하는 것이 아니라 변경된 특정 Field만 수집하여 변경 사항을 반영하는 방법이다. *<Dynamic><Select type="R"><sql>*에는 변경된(Replace) Field와 <DOCID>에 해당하는 Field를 수집하는 Query를 작성하고 Bridge는 이에 따라 수집을 수행한다. 따라서 Log Table 또는 Queue Table과 Join을 해야 하며, 특정 Field의 상태가 변경된 경우만 수집할 수 있도록 Query 조건에 포함시켜야 한다.

[Queue Table을 이용한 예제]

```
<Select type="R">
 <sql>
      SELECT b.docid,b.writer,q.seq FROM bbs b, queue q
      WHERE b.docid = q. srchkey and q.action = 'R'
 </sql>
</Select>
```

[Log Table을 이용한 예제]

```
<Select type="R">
 <sql>
      SELECT b.docid,b.writer,q.seq FROM bbs b
      WHERE  EXISTS
      (SELECT 'X' FROM IRAG_LOG sl WHERE sl.srchkey = b.docid and sl.brg_stc='4' )
 </sql>
</Select>
```

3.2.1.4.14. 삭제(Delete)된 문서 수집

삭제된 Data만 수집하여 변경 사항을 반영하는 방법이다. *<Dynamic><Select type="D"><sql>*에서는 Log Table 또는 Queue Table의 상태 값을 참고하여 삭제된(Delete) Data를 수집하는 Query가 작성되고 Bridge는 이에 따라 수집을 수행하게 된다.

[Queue Table을 이용한 예제]

```
<Select type="D">
   <sql>
      SELECT srchkey, seqno FROM queue_table WHERE table_name='bbs' and action='D'
   </sql>
</Select>
```

[Log Table을 이용한 예제]

```
<Select type="D">
   <sql>
      SELECT srchkey FROM IRAG_LOG WHERE brg_stc='0'
   </sql>
</Select>
```

[삭제 Flag값을 이용한 예제]

```
<Dynamic mode="datetime" seqno="n">
  <Select type="D">
      <sql>
  SELECT
      extdata_id
```

```
        FROM if_extdata
        WHERE delete_flag=1
        AND extdata_cdatetime >= '%DATETIME%'
        </sql>
    </Select>
</Dynamic>
```

주의
*<Dynamic mode="queue" seqno="y">*로 설정하고 sequence 번호 Field로 비교하는 구문이 포함될 경우, *<Select><sql>*의 Query 구문에서 결과Field들의 마지막에 항상 Queue Table의 sequence 번호 Field 값을 얻을 수 있도록 추가 설정되어야 한다.

Log Table을 사용하여 동적 Data를 수집한 경우에는 추가, 수정된 Data의 상태 값은 변경하고 삭제된 Data는 Log Table에서 삭제되어야 한다. Queue Table을 사용하여 동적 Data를 수집한 경우에는 수집된 문서의 상태 값을 가진 Data를 삭제해야 한다.

다음은 Queue Table을 이용하여 동적 Data를 수집한 후 실행해야 하는 Query 구문의 예이다.

[Queue Table을 이용한 예제]

```
<Execute>
  <sql statement="n">
      <![CDATA[
        DELETE FROM queue_table WHERE trim(SRCHKEY)= ? and trim(seqno) <= ?
      ]]>
  </sql>
</Execute>
```

다음은 Log Table을 이용하여 동적 Data를 수집한 후 실행해야 하는 Query구문의 예이다.

[Log Table을 이용한 예제]

```
<Execute type="D">
  <sql statement="n">
            DELETE FROM IRAG_LOG WHERE trim(SRCHKEY)= ?
  </sql>
```

```
</Execute>
<Execute>
  <sql statement="n">
            UPDATE IRAG_LOG SET brg_stc='3' WHERE trim(SRCHKEY)= ?
  </sql>
</Execute>
```

3.2.1.4.15. 관리자 테이블을 이용한(Master) 문서 수집

관리자 테이블을 이용한 문서 수집은 통합게시판과 같이 관리자(Master 또는 Admin) 테이블에 생성된 게시판의 이름을 이용하여 특정 조건에 부합하는 Table의 Data 전부를 수집하는 방법이다. *<Master><Select><sql>*에 작성된 Query로 Bridge가 Data를 수집하게 된다. 본 기능을 사용한 경우도 정적 수집과 동적 수집은 모두 지원한다.

게시판 이름을 수집대상 SQL의 FROM절에 지정해야 하는데 이 때 *<Static>*과 *<Dynamic>*의 sql 구문의 FROM절 예약어로 %s1을 사용해야 한다.

[*<Static>* 예제]

```
<Static>
  <Select>
      <sql>
      <![CDATA[
                SELECT docid,writer,createdate,subject,content FROM %s1
            ]]>
      </sql>
  </Select>
</Static>
```

[*<Master>* 예제]

```
<MasterTable>
  <Select>
      <sql>
      <![CDATA[
                SELECT table_name FROM masterT WHERE delete='N'
            ]]>
      </sql>
```

```
  </Select>
</MasterTable>
```

3.2.1.4.16. PrimaryKey의 설정

PrimaryKey는 수집 대상 Table의 P.K(Primary Key)를 *<Column name="">*에 설정하고 DOCID로 생성하는 방식을 결정한다. 하나 이상의 Column이 Primary key로 지정된다면 DOCID에 해당하는 Table의 Field도 하나 이상을 얻어야 한다. 이러한 설정은 모든 *<sql>*의 Query 구문에 반영되어야 한다.

[설정파일의 PrimaryKey 설정 예제]

```
<PrimaryKey>
  <Column name="seq"/>
<Column name="id"/>
</PrimaryKey>
```

위와 같이 2개의 Field(Column)을 Primary Key로 선택한 경우 다음과 같이 Select Query문을 작성해야 한다.

[예제]

```
<sql> Select seq, id, title, content, writer FROM [TABLE] </sql>
```

동적 Data수집에 필요한 Table을 생성하는 *<Init><Execute><sql>*의 Query구문에도 *<PrimaryKey>*의 ColName 2개의 Field(Column)를 생성할 수 있도록 한다.

[예제]

```
<Init>
    <Execute>
            <sql> CREATE TABLE IRAG_LOG
                  (
                   srchkey char(20)
```

```
                        , id char(10)
                        , brg_stc char(1)
                        )
                </sql>
    </Execute>
</Init>
```

3.2.2. Web Bridge 수집 설정

일반적으로 Web Bridge는 사이트 내부에 존재하는 웹 문서를 수집하여, 색인기가 인식 가능한 SCD를 만들어내는 일을 한다. Web Bridge는 static한 HTML 페이지는 물론, host와 href tag를 사용하여 링크되어 있는 문서들(HTML 페이지 외의 일반문서 포함)까지 수집할 수 있다.

Web Bridge는 JDK(Java development kit) 1.5 이상의 버전에서 동작한다.

3.2.2.1. Config 설정

다음은 설정 파일(Config)에서 Web Bridge와 관련된 부분이다.

[설정파일의 예제]

```
<Source id="web" type="web" sourcealias="web" extension="json">
<RefColl id="web" prefix="#a#"/>
<SCDdir path="C:\irag\collection\web\scd\"/>
<CatalogInfo>
<Mapping colnum="0"  tagname="DOCID"/>
<Mapping colnum="1"  tagname="subject"  type="subject" usertag=""/>
<Mapping colnum="2"  tagname="content"  type="content" usertag =""/>
<Mapping colnum="3"  tagname="url"     type="url"/>
<Mapping colnum="4"  tagname="Date"    type="date"/>
<Mapping colnum="5"  tagname="base_url" type="baseurl"/>
<Mapping colnum="6"  tagname="type"    type="form" />
<Mapping colnum="7"  tagname="source"  type="source" />
<Mapping colnum="8"  tagname="writer"  type="extend"  meta="" value="홍길동"/>
<Mapping colnum="9"  tagname="img_name" type="thumbnail"/>
<Mapping colnum="10" tagname="img_url " type="imageurl" />
```

```
<Mapping colnum="11" tagname="img_width" type="imagewidth" />
<Mapping colnum="12" tagname="img_height" type="imageheight" />
<Mapping colnum="13" tagname="fsize" type=" filesize " />
<Mapping colnum="14" tagname="ftype" type=" filetype " />
<Mapping colnum="15" tagname="alias"        alias="y"/>
</CatalogInfo>
<!-- wisenut seed file /-->
<SeedUrlFile path="C:\irag\config\seed.txt"/>
<!--방문할 URL List Queue를 관리하는 DB file  Path /-->
<DBFile path="C:\irag\config\urldb"/>
<!--썸네일 이미지가 저장될 파일 Path 및 이미지 설정 /-->
<Thumbnail path="" width="300" height="300" include="y" />
</Source>
```

주의
config 설정은 대소문자를 구분하고, XML 문법에 맞게 작성해야 한다.

[표 3.13 Web Bridge - Config 설정]

항목 (Element)	속성 (Attribute)	값 (Value)	설명
Source	id	문자열	수집 설정을 구분하는 값을 지정. DB Bridge를 실행할 때 사용하는 옵션인 srcid이기도 함.
	type (옵션)	db web file	해당 Source가 DB에서 정보를 가져온다는 것을 의미 (참고) type 속성은 명시적 표현으로서 수집모듈의 동작에 영향을 미치지 않음. 실행시 type를 제외하여도 동작 함.
	sourcealias (옵션)	문자열	SCD 파일에 해당 source의 출처가 어디인지 구분하기 위해 sourcealias를 넣어 줄 때 사용.
	extension (옵션)	json scd	수집된 파일 포맷을 JSON 또는 SCD로 설정한다. (default : json)
	imgAltTag (옵션)	y/n	HTML 내용의 모든 이미지 설명(<img alt>)을 본문에 추가.
	charset (옵션)	utf-8 euc-kr	SCD 파일을 구성하고 있는 문자 집합을 설정. 기본값은 utf-8이며 euc-kr, shift-jis와 같이 설정.

		shift-jis euc-jp gb2312	
Refcoll (옵션)	id	문자열	해당 Source가 참조하는 Collection ID를 지정. 여러 개의 Source가 하나의 Collection을 참조할 수 있음.
	prefix	문자열	Collection에 참조하는 Source가 여러 개일 경우 유일한 문서 구분 ID인 DOCID가 중복될 수 있음. 중복을 방지하기 위해서 #t#과 같은 문자를 사용.
SCDdir	path	SCD파일 디렉터리	SCD 파일이 생성되는 디렉터리 지정.
CatalogInfo.Mapping	colnum	0,1, …	SCD 항목의 index값 지정.
	tagname	DOCID	SCD 항목의 이름 지정.
	alias (옵션)	y/n	alias="y"인 경우 *<Source sourcealias=" ">*로 설정한 값을 SCD의 해당 Field에 저장. 즉, 예를 들면 *<Source id = "news" sourcealias= "daily">*로 설정하였다면, 이 경우 'daily라는 값을 SCD의 alias필드에 저장.
	meta (옵션)	Metatag의 속성값	meta tag의 속성 값을 설정하면 content의 value를 모두 얻어올 수 있음.
	usertag (옵션)	Meta_Title	수집대상 HTML 내용의 tag로 Meta_Title, Meta_Body와 같이 정의하면 content의 value를 얻어올 수 있음. 예) *<Meta_Body content="페이지를 설명하는 내용">*
	useTitleTag (옵션)	y/n	type의값이 subject 인 경우 옵션. anchortext를 사용여부를 설정. default : n y : anchortext 를 사용 n : HTML 문서의 <TITLE>태그의 값을 사용
	maxLen(옵션)	숫자	수집하는 데이터의 최대 길이를 지정.
	className (옵션)	Java Class	커스터마이징이 필요한 부분을 정의하여 사용하는 클래스를

			className="kr.co.wisenut.common.filter.custom.Http2 FilterData"과 같이 설정
	type	Web Bridge Data Type	▪ subject : Web 문서의 제목. ▪ content : Web 문서의 본문. ▪ url : 현재 문서의 전체URL. ▪ baseurl : 현재 문서의 Home URL. ▪ form : Web 또는 File을 구분. ▪ section : Seed 파일에 설정한 section값. ▪ source : Seed 파일에 설정한 source값. ▪ writer : Seed 파일에서 userdefine으로 작성자를 설정한 경우에 지정. ▪ date : 현재 문서에서 처음으로 찾은 Date 값을 사용. ▪ 지원되는 Date Type : yyyy-mm-dd or yyyy/mm/dd or yyyy.mm.dd ▪ extend : 확장 Type으로 CatalogInfo에 Field명이 필요한 경우에 지정. ▪ thumbnail : 지정할 경우 썸네일 이미지 이름이 필드에 저장. ▪ imageurl : 지정할 경우 이미지의 원본 URL을 저장. ▪ imagewidth : 썸네일 원본 이미지의 가로 사이즈를 저장. ▪ imageheight : 썸네일 원본 이미지의 세로 사이즈를 저장. filesize : 수집한 문서의 파일 사이즈를 저장. ▪ filetype : 수집한 문서의 파일 확장자를 저장. ▪ value : 임의의 값을 SCD파일에 저장할 때 지정 ▪ collectdivd: Seed 파일에서 collectDivd에 정의된 Div id 태그의 내용 ▪ collectclass: Seed 파일에서 collectClass에 정의된 class 태그의 내용
SeedUrlFile	path	Seed File Path	Web Bridge에서 사용하는 Seed 파일의 전체 경로를 지정.
Thumbnail	path	문자열	썸네일 이미지의 생성 경로를 설정. ""로 지정할 경우 $IRAG_HOME/thumbnail 디렉토리에 생성한다.
	width	숫자	썸네일 이미지 가로 사이즈를 설정.

	height	숫자	썸네일 이미지 세로 사이즈를 설정.
	include	y/n	썸네일 이미지 데이터를 SCD에 포함할 것인지 여부를 설정. y : SCD 에 base64인코딩 된 text값으로 저장. n : local경로에 이미지 파일로 생성

썸네일 이미지 생성시 주의할 점은 'Mapping노드'에 thumbnail 지정과 'Thumbnail 노드'선언을 반드시 해야 한다는 것이다. 또한 썸네일 이미지를 생성할 때 대상으로 선택되는 원본이미지는 그 문서에 포함된 대표 이미지이다. 대표 이미지의 선택 기준은 다음과 같다.

- 이미지 확장자는 jpg
- 원본 이미지의 높이가 150px 이상
- HTML에서 첫 번째 <img src>에 해당하는 이미지

3.2.2.2. Web Bridge의 URL Seed File 작성 방법

[예제]

```
[source]와이즈넛
[homeurl]http://www.wisenut.co.kr
[maxdepth]0
[maxurl]0
[url]http://www.wisenut.co.kr/index.asp
```

[표 3.14 Seed 항목 설정]

항목 (Element)	설명
source	Site Name을 설정한다.
homeurl	Site의 Base URL 주소를 설정한다. (Host Name)
maxdepth	수집 할 URL의 최대 깊이를 설정한다. Default : 0 (제한 없음)
maxurl	수집 할 URL의 최대 개수를 설정한다. Default : 0 (제한 없음)

Allowurl (옵션)	Site URL의 Link를 추출할 때 허용할 Site를 설정한다. (다중 설정)
Skipurl (옵션)	Site URL의 Link를 추출할 때 Skip 할 Site를 설정한다. (다중 설정)
Skipmenu (옵션)	Site에서 중복된 메뉴 부분을 SCD에 쓰지 않을 때 설정한다. (다중 설정)
Tailmenu (옵션)	Site에서 중복된 하단 내용을 SCD에 쓰지 않을 때 설정한다. (다중 설정)
Tailsrc (옵션)	HTML의 Div id를 지정하면 해당 id 태그부터 Html End Tag까지 모든 값을 SCD에 저장하지 않는다.
url	수집 대상이 되는 URL 주소를 설정한다.
Section (옵션)	config 작성 시, mapping부분에서 type을 section으로 설정했을 경우 이 값을 사용한다.
Charset (옵션)	HTML문서의 문자집합을 지정한다. HTML문서에 문자집합이 정의되지 않은 경우 이 값을 사용한다. 값으로는 euc-kr, shfit-jis, 8859-1 등이다.
Locale (옵션)	web 문서의 인코딩이 한국어가 아닌 다른 나라 언어일 때 사용한다. Default 값은 korean이다. 값으로는 Japan, china, hongkong, latin을 사용 할 수 있다. Locale 값을 설정했을 때 사용하는 인코딩은 Japan(shift-jis), china(gb2312), hongkong(bing5), latin(iso-8859-1)이다.
onlyviewurl (옵션)	onlyviewurl 에 정의 된 url은, [tailsrc] 설정 값에 관계 없이 수집이 된다. 예) [onlyviewurl]http://www.wisenut.co.kr/solutions/Classifier[tailsrc]footer 수집 되는 URL들 모두 footer 태그 하위의 내용들은 제거하여 수집되지만, "http://www.wisenut.co.kr/solutions/Classifier" URL을 수집 할 경우에는 footer 메뉴를 포함하여 수집한다.
islist (옵션)	목록페이지(링크만 있는 페이지)를 seed 로 지정하여 seed page에 등록된 URL만 추출하는 설정이다. 단, 링크페이지의 내용은 추출하지 않음.
denyId (옵션)	HTML의 Div id를 지정하면 해당 id에 포함되는 모든 값을 SCD에 저장하지 않는다. 2개 이상의 id를 정의하려면 콤마(,)를 이용한다. 예) [denyId] div1,div2,div3,…
denyClass (옵션)	HTML의 class 이름을 지정하면 해당 class에 포함되는 모든 값을 SCD에 저장하지 않는다. 2개 이상의 id를 정의하려면 콤마(,)를 이용한다.

	예) [denyClass] class1,class2,class3,…
collectDivId (옵션)	HTML의 Div id를 지정하면 해당 id에 포함되는 값만을 SCD에 저장한다. 2개 이상의 id를 정의하려면 콤마(,)를 이용한다. 예) [denyId] div1,div2,div3,…
collectClass (옵션)	HTML의 class 이름을 지정하면 해당 class에 포함되는 값만을 SCD에 저장한다. 2개 이상의 id를 정의하려면 콤마(,)를 이용한다. 예) [denyClass] class1,class2,class3,…

3.2.3. File Bridge 수집 설정

File Bridge는 local 저장소에 존재하는 문서의 내용을 수집하는 수집기이다. File Bridge는 파일 제목, 최종 수정일, 본문 내용 등 파일이 포함하고 있는 기본 정보들을 수집할 수 있다. File Bridge는 파일의 변경 이력 관리를 통해서 동적(신규 추가, 수정, 삭제) Data의 수집을 수행한다.

File Bridge는 JDK(Java development kit) 1.5 기반에서 동작한다.

3.2.3.1. Config 설정

다음은 설정 파일(Config)에서 File Bridge와 관련된 부분이다.

[설정 파일의 예제]

```
<Source id="file" type="file" sourcealias="" extension="json">
<RefColl id="file"/>
<SCDdir path="C:\irag\collection\file\scd\"/>
<Date format="none_time_t" order="4"/>

<CatalogInfo>
<Mapping colnum="0" tagname="DOCID" />
<Mapping colnum="1" tagname="subject"  type="filename" convert="" />
<Mapping colnum="2" tagname="writer"   type="extend"   convert=""  value="홍길동"/>
<Mapping colnum="3" tagname="content"  type="content"  convert="" html=""/>
<Mapping colnum="4" tagname="Date"         type="date"      convert="" />
<Mapping colnum="5" tagname="url"          type="fullpath" convert="" />
<Mapping colnum="6" tagname="alias"        type="gubun"   convert="" />
```

```
</CatalogInfo>

<TargetFile path="C:\document\" depth="0" dir="exclude" />
<History path="C:\irag\config\history\" />

<DirAcceptRule>
<Deny  rule="C:\document\temp" />
<Deny  rule="C:\document\system " />
<Allow rule="*" />
</DirAcceptRule>

<FileAcceptRule>
<Allow rule="*.doc" />
<Allow rule="*.ppt" />
<Allow rule="*.xls" />
<Allow rule="*.hwp" />
<Allow rule="*.pdf" />
<Allow rule="*.docx" />
<Allow rule="*.pptx" />
<Allow rule="*.xlsx" />
<Allow rule="*.txt" />
<Allow rule="*.hwd" />
<Allow rule="*.mht" />
<Allow rule="*.txt" />
<Deny  rule="*" />
</FileAcceptRule>
</Source>
```

주의
config 설정은 대소문자를 구분하고, XML 문법에 맞게 작성해야 한다.

3.2.3.2. Source 설정

[표 3.15 Source 설정]

속성 (Attribute)	값 (Value)	설명

id	문자열	수집 설정을 구분하는 값을 지정. DB Bridge를 실행할 때 사용하는 옵션인 srcid이기도 함.
type (옵션)	db web file	해당 Source가 DB에서 정보를 가져온다는 것을 의미 (참고) type 속성은 명시적 표현으로서 수집모듈의 동작에 영향을 미치지 않음. 실행시 type를 제외하여도 동작 함.
sourcealias (옵션)	문자열	SCD 파일에 해당 source의 출처가 어디인지 구분하기 위해 sourcealias를 넣어 줄 때 사용.
extension (옵션)	json scd	수집된 파일 포맷을 JSON 또는 SCD로 설정한다. (default : json)
charset (옵션)	utf-8 euc-kr shift-jis euc-jp gb2312	SCD 파일을 구성하고 있는 문자 집합을 설정. 기본값은 utf-8이며 euc-kr, shift-jis와 같이 설정.

3.2.3.3. Source.RefColl 설정 (옵션)

[표 3.16 Source.RefColl 설정]

속성 (Attribute)	**값 (Value)**	**설명**
id	문자열	해당 Source가 참조하는 Collection ID를 지정. 여러 개의 Source가 하나의 Collection을 참조할 수 있음.
prefix	문자열	Collection에 참조하는 Source가 여러 개일 경우 유일한 문서 구분 ID인 DOCID가 중복될 수 있음. 중복을 방지하기 위해서 #t#과 같은 문자를 사용.

3.2.3.4. Source.SCDdir 설정

[표 3.17 Source.SCDdir설정]

속성 (Attribute)	**값 (Value)**	**설명**
path	SCD 파일 디렉터리	SCD 파일이 생성되는 디렉터리를 지정.

3.2.3.5. Source.CategoInfo.Mapping

[표 3.18 Source. CategoInfo.Mapping설정]

속성 (Attribute)	값 (Value)	설명
colnum	0,1, …	SCD 항목의 index값을 나타냄.
fieldname	DOCID	SCD 항목의 이름을 지정.
alias(옵션)	y/n	alias="y"인 경우 *<Source sourcealias=" ">*로 설정한 값을 SCD의 해당 Field에 저장. 즉, 예를 들면 *<Source id = "news" sourcealias= "daily">*로 설정하였다면, 이 경우 'daily라는 값을 SCD의 alias필드에 저장..
type	File Bridge Data Type	▪ filename: File 문서의 이름. ▪ content : File 문서의 본문. ▪ fileext : File 문서의 확장자. ▪ fullpath : File 문서의 전체 경로. ▪ gubun : File 또는 디렉터리를 구분. ▪ dirpath : File 문서의 디렉터리 경로. ▪ filesize : File 문서의 크기. ▪ date : File 문서의 수정일. (YYYYMMDDhhmmss) ▪ modified date : File 문서의 수정일. (YYYY-MM-DD hh:mm:ss) ▪ extend : 확장 type으로 CatalogInfo에 Field 명이 필요한 경우에 지정. ▪ value : 임의의 값을 SCD파일에 저장할 때 지정
tagname		SCD 항목의 이름.
className (옵션)	Java Class	커스터마이징이 필요한 부분을 정의하여 사용하는 클래스를 className="kr.co.wisenut.common.filter.custom.Http2FilterData"과 같이 설정
convert (옵션)	문서의 charset 이름, 새로 저장 할 charset 이름	수집하는 Field의 Data 문자 집합을 변환할 수 있는 기능. 예) convert="8859_1,EUC-KR"으로 지정하였다면, 8859_1 Data를 EUC-KR로 문자 집합을 변경.

3.2.3.6. Source.TargetFile

[표 3.19 Source.TargetFile 설정]

속성 (Attribute)	값 (Value)	설명
path depth dir	디렉터리	수집 대상 디렉터리 또는 파일의 PATH 값.

3.2.3.7. Source.History

[표 3.20 Source.History설정]

속성 (Attribute)	값 (Value)	설명
path	디렉터리	수집한 파일의 이력 관리 파일의 PATH 값.

3.2.3.8. Source.DirAcceptRule

[표 3.21 Source.DirAcceptRule설정]

속성 (Attribute)	값 (Value)	설명
Deny/Allow	rule	디렉터리 PATH값으로 Allow와 Deny 규칙을 설정.

3.2.3.9. Source.FileAcceptRule

[표 3.22 Source. FileAcceptRule 설정]

속성 (Attribute)	값 (Value)	설명
Deny/ Allow	rule	파일 PATH 값으로 Allow와 Deny규칙을 설정.

3.2.4. 기타 수집 모듈

3.2.4.1. 문서 Filter 와 FilterServer

3.2.4.1.1. FilterServer의 등록

첨부 문서가 검색 엔진이 설치된 장비에 존재하지 않고 원격지의 Windows 혹은 Unix 서버 장비에 존재할 경우, FilterServer(문서 필터 서버)를 해당 서버 장비에 설치해야 한다. FilterServer를 백그라운드로 실행시키는 설정은 CManager의 서비스로 등록하도록 한다. 다음은 CManager의 설정파일에 FilterServer를 등록한 예이다.

[예제]

```
<Server id="filterserver1" type="etc" onstart="y" autorestart="y">
<StartCmd value="C:\irag\batch\filterServer.cmd start"/>
<StopCmd  value="C:\irag\batch\filterServer.cmd stop"/>
<Monitor   mode="port" port="7700" email="y"/>
</Server>
```

3.2.4.1.2. FilterServer의 실행

문서 필터를 Console에서는 c:\irag\bridge 디렉터리에서 아래의 예제와 같이 실행하도록 한다.

[예제]

```
C:\irag\bin>java -DIRAG_HOME=C:\irag -Dirag.ver=6 -Dthreadno=2 -classpath
C:\irag\bridge\bridge390.jar kr.co.wisenut.bridge3.ftserver.FtServer -port 7600
        [option]
          -log day or stdout
          -logpath set log file path
          -debug <1~4>
```

[표 3.23 FilterServer 실행 인자]

인자	값	설명

-DIRAG_HOME	<path>	VM 파라미터. irag의 home 디렉터리를 설정. IRAG_HOME 환경 변수를 overwrite함.
-Dirag.ver	6	irag의 버전을 의미.
-port	숫자	FilterServer가 서비스를 대기할 포트 번호
-start		FilterServer를 실행하는 인자
-stop		FilterServer를 중지하는 인자
-log	<stdout or day>	Standard out 과 Log File로 구분.
-logpath	문자열	로그파일이 저장될 절대경로를 지정
-debug	1~4	1 : ERROR 2: WARNING 3: INFO 4 : DEBUG

3.2.4.1.3. 문서 Filter 실행 파일

Bridge를 이용하여 수집되는 모든 파일은 문서 Filter를 통해 Filtering 후 수집된다. 문서 Filter는 Bridge가 설치된 경로 아래의 bin 디렉터리에 존재해야 한다. OS와 문서 종류에 따라 Filter의 종류는 다를 수 있지만 일반적으로 다음과 같이 실행하여 정상적으로 동작하는지 확인한다.

[예제]

```
C:\irag\bin>sn3f.exe
Synap Next 3 Document Filter v3.17.20
Copyright (c) since 2001 by SynapSoft.
Release : 20172017/11/22
DefaultEncoding : cp949

sn3f
 - shows synap next 3 filter usage

Usage: sn3f [OPTION]... <input_file> [output_file]
Filters input_file to stdout or [output_file]

[Filter Option]
 -O          filters embeded OLE Document, too
 -S          extract summary info
```

```
  -E Encoding  default encoding
[Output Encoding]
  -C Encoding  output encoding
  -U           UCS2 output encoding(deprecated)
  -U8          UTF8 output encoding(deprecated)
[Encoding List] : utf8, cp949, sjis, eucjp, big5, gb2312
```

[실행 예]

```
C:\irag\bin>sn3f.exe sample.doc sample.txt
```

3.2.4.1.4. file.allow파일의 설정

file.allow는 필터서버에서 파일 확장자에 따라서 필터링 제외를 판단할 수 있도록 설정하는 파일이다.

[사용법]

```
1) $IRAG_HOME/config/file.allow 파일 설치
참조 : FilterServerService.sh/cmd 에 정의된 변수 '-DIRAG_HOME' 경로를 참조함.
2) 수집 제외 하고자하는 파일타입을 file.allow 에서 주석처리 또는 제거한다.
```

[예]

```
# file extension properties
# MS Office File
allow=doc
allow=docx
allow=pptx

… 중략…

# Other file
allow=xml
allow=html
allow=chm
allow=eml
```

3.2.4.1.5. FilterExt의 설정

FilterExt는 표준브리지(DB, WEB, FILE)에서 필터링을 수행하고자 하는 파일 확장자를 설정한다.

확장자들은 쉼표(,)으로 구분해서 지정하며 FilterExt 노드를 정의하지 않으면 Default 로 정의된 확장자들을 필터링한다.

[기본 확장자]

doc, rtf, mht, pdf, ppt, xls, hwp, hwd, xml, txt, gul, jtd, zip, tar, gz, docx, xlsx, pptx, wpd, dwg

[사용법]

1) $IRAG_HOME/config/bridge설정 파일 확인
2) 수집 하고자하는 파일타입을 설정한다.

[예]

```
<FilterExt>
<ext desc="ms office">doc,docx,ppt,pptx,xls,xlsx</ext>
<ext desc="hangul">hwp</ext>
<ext desc="pdf 1.x file">pdf</ext>
<ext desc="rich text format">rtf</ext>
<ext desc="handy gw">hwx,hwn,hwd</ext>
<ext desc="iltaerang">jtd</ext>
<ext desc="ms document image">mdi</ext>
<ext desc="ms outlook">msg</ext>
<ext desc="word perfect">wpd</ext>
<ext desc="autodesk drawing">dwg</ext>
<ext desc="flash movie">swf</ext>
<ext desc="compress file">zip,tar,gzip,alzip,bzip</ext>
<ext desc="other">xml,htm,html,mht,chm,eml,mime,text,text,txt</ext>
</FilterExt>
```

[표 3.24 FilterExt 설정상세]

항목 (Element)	속성 (Attribute)	값 (Value)	설명

FilterExt.ext			필터링을 허용할 파일 확장자를 지정한다.
	desc	문자열	필터링을 허용할 파일 확장자에 대한 설정을 작성한다.

3.2.4.2. WNEncrypt

Bridge는 보안 상의 문제가 발생할 수 있는 DBMS접속에 필요한 계정 정보를 보호하는 모듈로 WNEncrypt라는 프로그램을 별도로 제공한다. WNEncrypt 프로그램은 datasorce.xml에 설정된 아이디와 패스워드를 아래와 같이 실행하여 암호화 한다.

[예제]

```
Usage : java -DIRAG_HOME=< IRAG_HOME> -Denc.id= <all  id name> kr.co.wisenut.WNEncrypt
```

[실행 예]

```
C:\irag\bin>java -DIRAG_HOME=C:\irag -classpath
C:\irag\bridge\encrypt\xmlutils.jar;C:\irag\bridge\encrypt\wnencrypt110.jar;. -Denc.id=all
kr.co.wisenut.WNEncrypt
```

위 실행 예의 명령을 실행하면, 검색 엔진 홈 디렉터리는 C:\irag로 설정되고, C:\irag\config\datasource.xml을 찾아서 datasource의 모든 아이디와 패스워드가 암호화된다. 암호화 되기 전의 파일은 datasource.bak.년월일시간 형태로 백업된다. 실행 인자에 대한 설명은 다음과 같다.

[표 3.25 wnencrypt 실행 인자]

인자	값	설명
-DIRAG_HOME	<path>	VM 파라미터. irag의 home 디렉터리를 설정. IRAG_HOME 환경 변수를 overwrite함.
-Denc.id	all or id	암호화할 계정이 설정된 Data Source의 ID를 지정하거나 all을 지정한다.

		all : datasource.xml의 모든 id의 접속정보를 암호화. id : datasource.xml의 해당 id의 접속정보만을 암호화

3.2.4.3. SCDRecvServer(SCD 수신 서버)

수집서버와 색인서버가 분리되어 설치된 경우 Bridge에서는 색인서버에서 SCD를 전송 받을 수 있는 SCDRecvServer라는 프로그램을 별도로 제공한다.

SCDRecvServer를 백그라운드로 실행시키기 위해서는 CManager의 서비스로 등록해야 하며 다음은 CManager의 설정파일에 SCDRecvServer를 등록한 예이다.

[예제]

```
<Server id="scdreceiver1" type="etc" onstart="y" autorestart="y">
<StartCmd value="C:\irag\batch\SCDRecvServer.cmd start"/>
<StopCmd  value="C:\irag\batch\SCDRecvServer.cmd stop"/>
<Monitor   mode="port" port="7700" email="y"/>
</Server>
```

3.2.4.4. Bridge 설정 추가

Bridge(DB, WEB, FILE)의 설정파일의 Source 항목 설정에 다음과 같이 설정하도록 한다.

[설정 파일의 예제]

```
<Source id="file" type="file" charset="euc-kr" sourcealias="">
<RefColl id="file"/>

<RemoteSCDdir path="C:\irag\collection\file\scd\"/>
<Remote ip="192.168.0.11" port="7801" deleteSCD="n"/>
<Remote ip="192.168.0.12" port="7801" deleteSCD="n"/>

</Source>
```

[표 3.26 SCDRecvServer – Bridge의 설정]

항목(Element)	속성(Attribute)	값(Value)	설명
RemoteSCDdir	path	문자열	SCDReceiver가 대기중인 서버의 경로를 지정한다. RemoteSCDdir 설정 없이 원격지로 파일을 전송할 경우 SCDdir 경로와 같은 위치로 전송된다.
Remote	ip	문자열	SCDRecvServer가 대기중인 서버의 IP주소를 설정한다.
	port	숫자	SCDRecvServer가 대기중인 서버의 PORT를 설정한다.
	deleteSCD	y/n	n : SCD파일을 원격지로 전송하고 backup 디렉터리로 이동시킨다 (기본값) y : SCD파일을 원격지로 전송하고 전송이 끝난 SCD 파일을 삭제한다.

3.2.4.5. SCDRecvServer의 실행

SCDRecvServer를 Console에서는 아래의 예제와 같이 실행하도록 한다.

[예제]

```
C:\irag\bin>java  -DIRAG_HOME=C:\irag  -Dirag.ver=6  -classpath  C:\irag\bridge\bridge390.jar
kr.co.wisenut.common.scdreceiver.SCDReceiver -port 7700 -start
option
-log  day or stdout
-logpath  set log file path
-debug <1~4>
```

실행 인자에 대한 설명은 다음과 같다.

[표 3.27 SCDRecvServer 실행 인자]

인자	값	설명
-DIRAG_HOME	<path>	VM 파라미터. irag의 home 디렉터리를 설정. IRAG_HOME 환경 변수를 overwrite함.

-Dirag.ver	6	irag의 버전을 의미.
-port	숫자	SCDRecvServer가 서비스를 대기할 포트 번호
-start		SCDRecvServer를 실행하는 인자
-stop		SCDRecvServer를 중지하는 인자
-log	<stdout or day>	Standard out 과 Log File로 구분.
-logpath	문자열	로그파일이 저장될 절대경로를 지정
-debug	1~4	1 : ERROR 2: WARNING 3: INFO 4 : DEBUG

3.3. 수집기 실행

3.3.1. DB Bridge 실행

[배치파일 실행 예시]

```
C:\irag\bin>bridge.cmd {config path} db {source-id} test -filterdel true -log stdout -debug 3
-indexdir

set global environment: irag.sh.cmd

config-file:
   config file path

avaliable bridge-types:
   db      crawl from database
   web     crawl from web
   file    crawl from file

source-id:
   source id

mode:
   test        test mode
   init        initialize data
   static      crawl to static data
   dynamic     crawl to dynamic data
   replace     crawl to replace data (db type only)

avaliable options:
   -indexdir              SCD generate location to scd/index directory
   -debug <1~4>           set debug level from 1 to 4
   -filterdel [true|false]   set filtered text file remove status
```

[바이너리 실행 예시]

```
C:\irag\bin> java -DIRAG_HOME=C:\irag -Dirag.ver=5 -classpath C:\irag\bridge\bridge402.jar;C:\irag\bridge\lib\jdom.jar;C:\irag\bridge\lib\xecres.jar kr.co.wisenut.bridge3.JBridge -conf C:\irag\config\config.xml -srcid sample_goods -mode test -filterdel true -log stdout -debug 3 -indexdir
```

Note

Unix 계열에서 실행하는 경우에는 디렉터리 및 classpath 를 Unix 계열의 형식에 맞게 변경해야 할 필요가 있다. 예를 들어, Windows 계열의 디렉터리 표현방식의 C:\irag 대신 /home/wisenut/irag 와 같이 사용해야 하며, classpath 의 구분자인 ; 대신 : 를 사용해야 한다.

Unix 계열에서는 아래와 같은 명령어를 사용하여 Bridge(브릿지)를 실행할 수 있다.

```
$ java -DIRAG_HOME=/home/wisenut/irag -Dirag.ver=5 -classpath /home/wisenut/irag/bridge/bridge402.jar: /home/wisenut/irag/bridge/lib/jdom.jar: /home/wisenut/irag/bridge/lib/xecres.jar kr.co.wisenut.bridge3.JBridge -conf /home/wisenut/irag/config/config.xml -srcid sample_goods -mode test -filterdel true -log stdout -debug 3 -indexdir
```

위 실행 예의 명령을 실행하면, 검색 엔진 홈 디렉터리는 C:\irag로 설정되고, C:\irag\config\config.xml이 설정 파일로 인식해 bbs source를 수집하게 된다. 이때 test 모드가 실행되며 화면에 출력된 Log를 통해 실행 과정을 자세히 볼 수 있다.

[표 3.28 DB Bridge 실행 인자]

인자	값	설명
-mode	init	Table의 생성 및 수집된 SCD를 삭제.
	test	테스트 수집 모드.
	static	Log Table 또는 Queue Table을 이용하지 않고 원본 문서 Data 전부를 수집.
	dynamic	Log Table 또는 Queue Table을 이용하여 신규 추가, 수정, 삭제된 Data를 수집.
	replace	Log Table 또는 Queue Table을 이용하여 변경된 Field의 Data를 수집

-srcid	source id	Source id in config file. (case-sensitive)
-DIRAG_HOME	<Path>	VM 파라미터. irag의 home 디렉터리를 설정. IRAG_HOME 환경 변수를 overwrite함.
-Dirag.ver	5	irag의 버전을 의미.
-conf	<filepath>	config file path.
-log	stdout day	bridge log type. Standard out 과 Log File로 구분.
-Dfile.count	숫자	VM 파라미터. 수집된 json 또는 scd 파일에 기록될 Data의 최대 개수를 지정.
-filterdel	true false	첨부 문서가 Filtering 된 후의 text 파일의 삭제 여부를 선택.
-indexdir		SCD파일을 설정파일의 *<SCDdir>*에 상관없이 index 디렉터리에 생성.
-debug	1~4	1 : ERROR 2: WARNING 3: INFO 4 : DEBUG
-logpath	filepath	Bridge로그파일이 저장될 절대경로를 지정
-verbose		verbose mode 각 칼럼의 내용, filter된 파일의 내용 등을 볼 수 있음. test 단계에서 사용.
-p[:digit :]	value	파라미터로 값을 입력받아 <Select.sql> 조건절에 값을 변경. 값을 다르게 적용해야만 동시에 Bridge 실행 가능.

3.3.2. Web Bridge 실행

[배치파일 실행 예제]

```
C:\irag\bin>bridge.cmd {config path} web {source-id} test -filterdel true -debug 3 -indexdir

set global environment: irag.sh.cmd

config-file:
   config file path
```

```
avaliable bridge-types:
   db      crawl from database
   web     crawl from web
   file    crawl from file

source-id:
   source id

mode:
   test        test mode
   init        initialize data
   static      crawl to static data
   dynamic     crawl to dynamic data
   replace     crawl to replace data (db type only)

avaliable options:
   -indexdir              SCD generate location to scd/index directory
   -debug <1~4>           set debug level from 1 to 4
   -filterdel [true|false]   set filtered text file remove status
```

[바이너리 실행 예시]

```
C:\irag\bin>java -DIRAG_HOME=C:\irag -Dirag.ver=5 -classpath
C:\irag\bridge\bridge402.jar;C:\irag\bridge\lib\jdom.jar;C:\irag\bridge\lib\xecres.jar
kr.co.wisenut.wbridge3.WBridge -conf C:\irag\config\config.xml -log stdout -srcid web -mode
test -filterdel true -debug 3 -indexdir
```

위 실행 예의 명령을 실행하면, 검색 엔진 홈 디렉터리는 C:\irag로 설정되고, C:\irag\config\config.xml을 설정 파일로 인식해 web source를 수집하게 된다. 이때 test 모드가 실행되며 화면에 출력된 Log를 통해 실행 과정을 자세히 볼 수 있다.

[표 3.29 Web Bridge 실행 인자]

인자	값	설명
-mode	init	delete SCD
	test	테스트 수행
	static	전체 Data 수집

-srcid	<source id>	source id in config file.(case-sensitive)
-DIRAG_HOME	<path>	VM 파라미터. irag의 home 디렉터리를 설정. IRAG_HOME 환경 변수를 overwrite 함.
-Dirag.ver	5	irag의 버전을 의미.
-conf	<filepath>	set config file path
-log	<stdout or day>	bridge log type Standard out 과 Log File로 구분.
-filterdel	<true or false>	첨부 문서가 Filtering 된 후 text 파일을 삭제할 지 여부를 선택.
-indexdir		SCD파일을 설정 파일의 *<SCDdir>*에 상관 없이 index 디렉터리에 생성.
-debug	1~4	1 : ERROR 2: WARNING 3: INFO 4 : DEBUG
-logpath	<filepath>	Bridge로그파일이 저장될 절대경로를 지정
-verbose		각 칼럼의 내용, filter된 파일의 내용 등을 볼 수 있음. test 단계에서 사용.

3.3.3. File Bridge 실행

[배치파일 실행 예시]

```
C:\irag\bin>bridge.cmd {config path} file {source-id} test -filterdel true -debug 3 -indexdir

set global environment: irag.sh.cmd

config-file:
   config file path

avaliable bridge-types:
   db      crawl from database
   web     crawl from web
   file    crawl from file

source-id:
   source id
```

```
mode:
   test        test mode
   init        initialize data
   static      crawl to static data
   dynamic     crawl to dynamic data
   replace     crawl to replace data (db type only)

avaliable options:
   -indexdir              SCD generate location to scd/index directory
   -debug <1~4>           set debug level from 1 to 4
   -filterdel [true|false]   set filtered text file remove status
```

[바이너리 실행 예시]

```
C:\irag\bin>java -DIRAG_HOME=C:\irag -Dirag.ver=5 -classpath
C:\irag\bridge\bridge402.jar;C:\irag\bridge\lib\jdom.jar;C:\irag\bridge\lib\xecres.jar
kr.co.wisenut.fbridge3.FBridge -conf C:\irag\config\config.xml -log stdout -srcid file -mode test
-filterdel true -debug 3 -indexdir
```

위 실행 예의 명령을 실행하면, 검색 엔진 홈 디렉터리는 C:\irag로 설정되고, C:\irag\config\config.xml을 설정 파일로 인식해 File source를 수집하게 된다. 이때 test 모드가 실행되며 화면에 출력된 Log를 통해 실행 과정을 자세히 볼 수 있다.

대상 File이 단순 텍스트 형태의 파일(.txt)이 아니라 Filtering이 필요한 .doc나 .ppt 형태의 파일이라면 Bridge가 실행되는 위치에 Filter를 함께 옮겨 놓는다. 위 실행 예의 경우, C:\irag\bin에 Filter를 옮겨놓으면 .doc나 .ppt 형태의 파일도 Filtering되어 수집된다.

[표 3.30 File Bridge 실행 인자]

인자	값	설명
-mode	init	delete SCD
	test	테스트 수집.
	static	전체 Data 수집.

	dynamic	신규 추가, 수정, 삭제된 Data 수집.
-srcid	<source id>	source id in config file.(case-sensitive)
-DIRAG_HOME	<path>	VM 파라미터. irag의 home 디렉터리를 설정. IRAG_HOME 환경 변수를 overwrite함.
-Dirag.ver	5	irag의 버전을 의미.
-conf	<filepath>	set config file path
-log	<stdout or day>	bridge log type Standard out 과 Log File로 구분.
-filterdel	<true or false>	첨부 문서가 Filtering된 후의 text 파일을 삭제할지 여부를 선택.
-indexdir		SCD 파일을 설정 파일의 *<SCDdir>*에 상관 없이 index 디렉터리에 생성.
-logpath	<filepath>	Bridge로그파일이 저장될 절대경로를 지정
-debug	1~4	1 : ERROR 2: WARNING 3: INFO 4 : DEBUG
-verbose		verbose mode 각 칼럼의 내용, filter 된 파일의 내용 등을 볼 수 있음. test 단계에서 사용.

주의
접근 권한이 없는 폴더에 존재하는 파일 또는 접근 권한이 없는 파일은 수집할 수 없으므로 설정 파일에 <TargetFile>의 path를 입력할 때 주의해야 한다.

4. 사전

본 제품에서 사전은 크게 세 가지 목적으로 활용한다.

첫째, 형태소 분석 시 중요한 자원(Knowledge)으로 형태소 분석에 필요한 형태소들을 저장하는 목적으로 사용한다. 형태소 분석 성능은 형태소의 질과 양에 따라 달라지며 이는 곧 검색 품질에 영향을 끼칠 수 있다. 형태소 분석에서 사용하는 사전은 세 가지가 있다. “사용자 사전(Usernoun Dictionary)”과 동의어 확장에 사용하는 “동의어 사전(Synonym Dictionary)”, 검색 결과 대상에서 제외시키는 “금칙어 사전(Stopword Dictionary)”이 있다.

4.1. 사전 등록

사전 내용을 등록하거나 수정하기 위해서는 세 가지 정보가 필요하다. 사전 구분을 위한 “ID”값과 사전의 종류를 나타내는 “type”, 그리고 실제 사전 내용이 들어가는 “data”이다. 검색엔진 내부에서는 이 “type” 에 따라 등록되는 사전 내용인 “data” 의 정보를 분석하고 저장한다.

4.1.1. 사전 ID

사전은 사전 마다 고유한 ID를 부여하여 관리하며, 이 ID값을 이용해 컬렉션에 적용한다. {사전 ID} 로 컬렉션에 적용하는 내용은 [4.5. 컬렉션에 사전 적용] 에서 자세히 설명한다.

4.1.2. 사전 타입

사전 “type”값은 아래와 같이 3개 항목이 있으며, 사전의 등록 및 수정이 필요할 때는 해당 사전 “type” 을 명시적으로 입력해야 한다.

[표 4.1 사전 타입]

type	사전 종류

usernoun	사용자 명사 사전
synonym	동의어 사전
stopword	금칙어 사전

4.1.3. 사전 등록 (초기화)

사전 등록하기 위해서는 사전 ID, 사전 타입, 그리고 사전에 등록할 키워드 데이터를 입력한다. 검색엔진은 사전 내용이 변경되면 색인기와 검색기에 해당 내용을 업데이트 한다. 따라서 모든 사전은 API를 통한 변경 내용을 검색기에 동적으로 반영할 수 있다.

[표 4.2 사전 등록(초기화)]

HTTP Method	URL	비고
POST	/dictionaries/{dictionary-id}	초기화

[사전 등록의 예시]

```
POST : http://127.0.0.1:9001/dictionaries/{dictionary-id}
{
  "type": {dictionary-type},
  "data": {dictionary-data}
}
```

4.1.4. 사전 키워드 업데이트 및 키워드 삭제

사전 키워드 업데이트는 사전 등록과 사용 방법이 거의 유사하다. 사전 키워드 업데이트는 {dictionary-id} 를 기준으로 기존에 등록된 {dictionary-id} 의 내용과 비교한다. 키워드 단위(또는 부스팅 아이디 단위)로 데이터를 수정하며, 기존 사전에서 없는 키워드이면 추가되고 중복되는 키워드 데이터가 있을 경우 새롭게 수정 요청으로 보낸 키워드 내용으로 교체된다.

매핑 테이블의 경우는 삭제 시 field명 뒤에 true면 전부 삭제, keyword list인 경우 부분 삭제로 뒤에 예시가 있으니 참고해보자.

[표 4.3 사전 키워드 업데이트 및 삭제]

HTTP Method	URL	비고
POST	/dictionaries/{dictionary-id}/update	키워드 업데이트
POST	/dictionaries/{dictionary-id}/delete	키워드 삭제

[사전 키워드 업데이트의 예시]

```
POST : http://127.0.0.1:9001/dictionaries/{dictionary-id}/update

{
  "type": {dictionary-type},
  "data": {dictionary-data}
}
```

사전 키워드 삭제는 {dictionary-id} 를 기준으로 기존에 등록된 {dictionary-id} 의 내용과 비교한다. 키워드 단위 (또는 부스팅 아이디 단위)로 데이터를 삭제한다. 주의해야할 점은 "data"라는 key의 value값에 삭제하고 싶은 키워드를 배열로 입력해야한다는 점이다. 키워드부스팅사전의 경우, 삭제를 원하는 id값을 입력해야한다.

[사전 키워드 삭제의 예시]

```
POST : http://127.0.0.1:9001/dictionaries/{dictionary-id}/delete

{
  "type": {dictionary-type},
  "data": [
    {dictionary-data}
  ]
}
```

4.1.5. 사전 삭제

삭제하고자 하는 사전의 {dictionary-id} 를 입력하여 해당 사전을 삭제한다. 키워드 단위가 아닌 사전 단위이므로 입력한 사전과 내부 데이터가 모두 삭제된다.

[표 4.4 사전 삭제]

HTTP Method	URL	비고
POST	/dictionaries/delete	삭제

[사전 삭제의 예시]

```
POST : http://127.0.0.1:9001/dictionaries/delete

[
  "dictionary-id-1",
  "dictionary-id-2"
]
```

4.1.6. 사전 조회

[표 4.5 사전 조회]

HTTP Method	URL	비고
GET	/dictionaries	사전 리스트 조회
GET	/dictionaries/{dictionary-id}	사전 정보 조회
GET	/dictionaries/type/{dictionary-type}	사전 type 리스트 조회

사전 리스트 조회는 등록되어 있는 사전 id 목록을 불러온다. 사전 삭제 전, 사전 id 값을 확인하는 용도로 사용 가능하다.

사전 정보 조회는 등록되어 있는 사전들 사전 id를 이용해 특정 사전의 정보(id/type/data)를 불러오는 기능이다. 특정 사전이 잘 등록되었는지 확인하는 용도로 사용 가능하다.

사전 type 조회는 등록되어 있는 사전 리스트에서 해당하는 특정 type 사전 id 목록을 불러온다. analysis 등록, collection 등록, 사전 삭제 전, 사전 id 값을 확인하는 용도로 사용 가능하다.

지원하는 type 으로는 usernoun, synonym, stopword 가 있다.

4.2. 사용자 명사 사전

형태소 분석(kma)에서 "사용자 사전"은 분석기가 분석하지 못하는 복합 명사나 특수 문자가 포함된 단어를 색인어로 추출하기 위한 용도로 사용된다. 이를 이용하여 특정 분야에만 빈번하게 출현하는 어휘나 전문 분야에만 나타나는 전문 용어, 빠르게 생성되는 신조어 등에 사용자가 쉽게 대응할 수 있다.

대상 어휘는 아래와 같다.

- 신조어 및 전문 용어, 특정 분야에만 빈번히 출현하는 명사
- 기본적으로 제공하는 사전에 등록되지 않아 과분석 및 오분석이 발생할 수 있는 단일 명사
- 기본적으로 제공하는 사전에 등록되어 있으나, 복합 명사의 분해 결과가 다른 어휘를 등록할 필요가 있는 어휘
- 띄어쓰기 오류가 많은 드라마 제목, 영화 제목과 같은 고유 명사

사용자 명사 사전에 위와 같은 대상의 어휘를 등록하기 위해서는 사전 이름(ID)와 "type"을 "usernoun"으로 지정한다.

[표 4.6 사용자 명사 사전 내부 옵션]

옵션	설명
data	각각의 사전 데이터가 들어가는 영역

data.{keyword}.separation	단어의 분리 정보 분리 정보는 0~9, a~z를 사용할 수 있으며 이는 음절 길이를 뜻한다. 9 이상의 길이를 사용할 경우 10을 a에 맞추어 35인 z까지 사용할 수 있다.

[사용자 사전 등록의 예]

```
POST : http://127.0.0.1:9001/dictionaries/{usernoun-dictionary-id}
{
  "type": "usernoun",
  "data": {
    "니뮐러": {
      "separation": "0"
    },
    "와이즈넛": {
      "separation": "0"
    },
    "대학생선교회": {
      "separation": "33"
    }
  }
}
```

4.2.1. 단일 명사 분석 정보

기본 사전에 [대학, 생선, 교회] 세 개의 단일 명사가 등록되어 있다고 가정하여 "대학생선교회"를 형태소 분석하면 [대학, 생선, 교회]로 분석이 될 수 있다. 하지만 사용자가 [대학생선교회]와 같은 단일 형태소로 분석을 요구할 때, 사용자 사전에 "대학생선교회"를 단일 명사로 등록하면 단일 형태소로 인식이 가능하다.

이와 같이 전문 용어나 신조어 중 분석되지 않아야 하는 단일 명사가 사전에 등록되어 있는 다른 명사로 인해 엉뚱하게 분석되는 경우, 사용자가 사전에 단일 명사로 등록하여 과분석 및 오분석을 막는다. 단일 명사로 분석하기 위해서는 분석 정보(Separation[L26])를 0으로 등록해야 한다.

일본어와 중국어는 영어를 무조건 분리하기 때문에 사용자 명사 사전에 작성해두어도 형태소 분석기에 의해 분리된다.

```
"대학생선교회": {
    "separation": "0"
}
```

4.2.2. 복합 명사 분석 정보

복합 명사는 두 개 이상의 단일 명사가 결합된 어휘로 등록된 분해 정보를 바탕으로 복합 명사를 단일 명사로 분해한다. 복합 명사인 "대학생선교회"를 [대학생, 선교회]와 같은 형태의 단일 명사로 분해하기 위해서는 분석 정보가 "33"이면 가능하다. 여기서 분석 정보는 각 단일 명사의 음절 길이를 뜻한다.

```
"대학생선교회": {
    "separation": "33"
}
```

"사용자 사전"은 사용자 관점에서는 필요한 어휘들을 등록하는 사전이므로 분석기에서 기본으로 제공하는 사전보다 높은 우선순위를 가지고 분석에 사용된다. 그러므로 "사용자 사전"에 어휘를 등록할 때에는 신중을 기해야 한다. 특히, 2 음절 이하의 어휘는 등록하지 않는 것을 권장하고 있다. 이들 어휘들은 예기치 못한 곳에서 과분석 및 오분석을 낳을 수 있기 때문이다.

4.3. 동의어 사전

동의어란 같은 뜻이나 비슷한 뜻을 가진 다른 낱말을 이르는 말이다. 여러 개의 단어들이 하나의 같은 의미를 가지고 있을 때, 혹은 하나의 단어에 대해 같은 의미를 가진 여러 단어들을 함께 검색하고 싶을 경우에는 동의어 사전을 사용한다. 예를 들어, "은하수"와 "미리내"는 같은 의미를

가지는 단어이므로 이 두 개의 단어를 “동의어 사전”에 등록하면, “은하수”가 포함된 문서를 “미리내”로 검색할 수 있으며, “미리내”가 포함된 문서도 “은하수”로 검색할 수 있다. 즉, “은하수”나 “미리내”로 검색하면 두 단어가 포함된 모든 문서를 검색할 수 있다. 또한 v7.1 버전 이후부터는 띄어쓰기를 포함한 동의어 등록을 지원한다.

동의어 사전은 stemmer, token, kma, ema 에서만 지원되고 like, matrix, ngram의 경우는 지원하지 않는다. 동의어 사전에 [표제어 : 회사] [동의어 : 와이즈넛] (3글자 이상의 키워드) 이 등록되어 있고, bigram에 동의어 사전을 적용한다고 가정하자. 이때, [회사]가 [회사 | 와이즈넛] 으로 동의어 확장이 되어도 색인키워드에서 [와이즈넛](3글자 이상인 키워드)을 찾을 수 없게된다. 따라서 동의어 사전 적용이 무의미해지게 된다.

동의어 사전은 공통적으로 다음과 같은 형식을 따른다.

[표 4.7 동의어 사전 내부 옵션]

옵션	설명
caseSensitive	대소문자 구분 옵션 true: 사전에 입력된 모든 데이터의 대소문자가 그대로 등록된다. false (default) : 사전에 입력된 모든 데이터가 소문자로 변환되어 등록된다.
data.keyword	표제어
data.synonym	표제어의 동의어
data.subword	표제어의 하위어

[동의어 사전 등록의 예]

```
POST : http://127.0.0.1:9001/dictionaries/{synonym-dictionary-id}
{
  "type": "synonym",
  "caseSensitive": "false",
  "data": {
    "과일": {
      "synonym": [
        "후르츠"
      ],
```

```
        "subword": [
          "파인애플",
          "키위"
        ]
      },
      "라이언": {
        "synonym": [
          "사자"
        ]
      }
    }
  }
```

4.3.1. 동의어: 하위어

동의어 단어 간의 상/하 개념을 나타내야할 때 사전에 단어들을 하위어(subword)로 등록 할 수 있다. 예를 들어 쇼핑몰에서 검색결과를 하위 의미의 품목들로 확장하고자 할 때 사용할 수 있다. 가령 "과일"의 하위어로 "딸기", "포도", "사과"를 등록하는 것이다. 이렇게 했을 경우에 "과일"로 검색을 한 경우에도 하위어가 포함된 문서들도 검색결과에 포함될 수 있다.

[동의어 사전에서 하위어 등록의 예]

```
{
  "type": "synonym",
  "data": {
    "과일": {
      "synonym": [
        "후르츠"
      ],
      "subword": [
        "딸기",
        "포도",
```

```
        "사과"
      ]
    }
  }
}
```

여기서 '표제어'는 대표가 되는 단어로, 색인어와 검색어의 동의어 확장 되는 단어이다. 위 예제에서는 "과일"이 표제어가 되고 "후르츠"가 동의어가 된다.

4.3.2. 동의어 등록 시 주의사항

4.3.2.1. 동일 단어를 동의어로 등록하지 않는다.

[표 4.8 동의어 등록 주의사항 1]

구분	설명
잘못된 예	이미 등록된 동의어의 키워드가 다른 표제어의 동의어로 등록되면 안된다. "음식"이 아래와 같이 두 개의 표제어의 동의어로 등록되면 안된다. 1) 표제어: food, 동의어: 음식 2) 표제어: 푸드, 동의어: 음식
올바른 예	1) 표제어: 음식, 동의어: food, 푸드 2) 표제어: 푸드, 동의어: 음식

4.3.2.2. 하위어 개념으로 추가할 경우에는 표제어로 등록한다.

[표 4.9 동의어 등록 주의사항 2]

구분	설명
잘못된 예	1) 표제어: 과일, 동의어: 후르츠, 하위어: 딸기, 포도 2) 표제어: strawberry, 동의어: 딸기
올바른 예	1) 표제어: 과일, 동의어: 후르츠, 하위어: 딸기, 포도 2) 표제어: 딸기, 동의어: strawberry

4.3.2.3. 동의어 사전에 등록한 단어를 금칙어 사전에 등록하지 않는다.

[표 4.10 동의어 등록 주의사항 4]

구분	설명
사전의 예	1) 표제어: 과일, 동의어: 후르츠, 하위어: 딸기, 포도 2) 금칙어 : 과일
검색의 예	위와 같이 동의어사전에 등록한 단어 [과일]을 금칙어 사전에 등록한다면, 검색어는 아래와 같다. 검색어 : [후르츠] OR [딸기] OR [포도] [과일]은 금칙어 사전에 의해 걸러지지만, 동의어와 하위어로 확장되어 사용자가 원하는 검색 결과와 상이한 결과값이 나올 수 있다. 즉, 동의어사전과 금칙어사전에 중복된 단어가 있을 경우, 검색 품질 저하를 불러올 수 있으므로 권장하지 않는다.

4.3.3. 동의어 확장 룰 (Synonym Expansion Rule)

4.3.3.1. Rule

1. 표제어는 동의어, 하위어로 확장된다.
2. 동의어는 표제어, 하위어로 확장된다.
3. 동의어는 동의어끼리 확장된다.
4. 하위어는 표제어로만 확장된다.
5. 하위어는 동의어로 확장되지 않는다.
6. 하위어는 하위어끼리 확장되지 않는다.

4.3.3.2. 사전 예시

표제어 : [과일]

동의어 : [채소] [식물]

하위어 : [딸기] [포도] [grape] [strawberry]

[동의어 사전 등록 예시]

```
POST : http://127.0.0.1:9001/dictionaries/synonym-test
{
  "type": "synonym",
  "data": {
    "과일": {
      "synonym": [
        "채소",
        "식물"
      ],
      "subword": [
        "딸기",
        "포도",
        "grape",
        "strawberry"
      ]
    }
  }
}
```

[애널라이저 등록 예시]

```
POST : http://127.0.0.1:9001/analysis
{
  "analyzer": {
    "tokenId": {
      "lae": "tokenId",
      "dictionary": [
        "synonym-test"
      ]
    }
  },
  "lae": {
    "tokenId": {
      "type": "token"
```

```
    }
  }
}
```

[컬렉션 등록 예시]

```
POST : http://127.0.0.1:9001/collections/sample
{
  "numOfShards": 1,
  "numOfReplication": 0,
  "recentQuery": 5,
  "fields": [
    "DOCID",
    "DATE",
    "TITLE",
    "CONTENT",
    "DEPARTMENT"
  ],
  "indexFields": {
    "TITLE": {
      "type": "text",
      "analyzer": [
        "tokenId"
      ]
    }
  }
}
```

4.3.3.3. 검색 결과 예시

[표제어 : [과일]로 검색 예시]

```
POST : http://127.0.0.1:9001/search/sample/query-morpheme


{
```

```
  "query": "과일",
  "searchField": [
    "TITLE"
  ],
  "synonymExpansion": 1,
  "useSynonym": 1,
  "useLa": 1
}
```

```
{
  "result": {
    "TITLE": {
      "tokenId": [
        "과일",
        "식물",
        "채소",
        "strawberry",
        "grape",
        "포도",
        "딸기"
      ]
    }
  }
}
```

[동의어: [채소]로 검색 예시]

POST : http://127.0.0.1:9001/search/sample/query-morpheme

```
{
  "query": "채소",
  "searchField": [
    "TITLE"
  ],
  "synonymExpansion": 1,
```

```
  "useSynonym": 1,
  "useLa": 1
}
```

```
{
  "result": {
    "TITLE": {
      "tokenId": [
        "채소",
        "식물",
        "strawberry",
        "grape",
        "포도",
        "딸기",
        "과일"
      ]
    }
  }
}
```

[하위어: [grape]로 검색 예시]

POST : http://127.0.0.1:9001/search/sample/query-morpheme

```
{
  "query": "grape",
  "searchField": [
    "TITLE"
  ],
  "synonymExpansion": 1,
  "useSynonym": 1,
  "useLa": 1
}
```

```
{
  "result": {
```

```
    "TITLE": {
      "tokenId": [
        "grape",
        "과일"
      ]
    }
  }
}
```

4.4. 금칙어 사전

금칙어 사전은 실시간으로 질의어를 통제하기 위한 사전이다. 금칙어는 검색 시에만 적용되어 검색 결과를 노출시키지 않는다는 것이다. 즉, 금칙어로 등록된 단어로 인해 색인 데이터가 삭제되지는 않는다. 금칙어 사전은 stemmer, token, kma 에서만 지원되고 like, matrix, ngram의 경우는 지원하지 않는다

[금칙어 사전 등록의 예]

```
POST : http://127.0.0.1:9001/dictionaries/{stopword-dictionary-id}

{
  "type": "stopword",
   "caseSensitive": "false",
  "data": [
    "소인", "거인"
  ]
}
```

4.5. 애널라이저 및 컬렉션에 사전 적용

사용자 명사 사전, 동의어 사전, 금칙어 사전을 컬렉션에 적용하는 방법은 다음과 같다.

기존 컬렉션에서 사용중인 Analyzer에서 사용자 사전이 추가되거나 변경된 경우, 재색인을 수행해야 색인 분리정보가 적용된다.

4.5.1. 사용자 명사 사전 적용

[표 4.11 사용자 명사 사전 내부 옵션]

옵션	설명
dictionary	애널라이저 등록에서 “lae” 를 생성할 때, {사용자 사전 ID} 값을 넣어 생성한다. dictionary는 array 형으로 여러 개의 사용자 명사 사전 등록이 가능하다. 동일한 data를 가진 dictionary 등록 시, 순차적으로 덮어씌워 진다.

[애널라이저 등록에서 사용자 사전 적용의 예]

```
{
  "analyzer": {},
  "regulator": {},
  "lae": {
    "kma-black": {
      "type": "kma",
      "kmaOption": "A+R2",
      "licensePath": "korean/license.xml",
      "knowledgePath": "korean/wkb_nocomp.klg",
      "dictionary": [
        "usernoun-dic-1",
        "usernoun-dic-2"
      ]
    }
```

```
  }
}
```

4.5.2. 동의어 사전 적용

[표 4.12 동의어 사전 내부 옵션]

옵션	설명
dictionary	애널라이저 등록에서 “analyzer” 를 생성할 때, {동의어 사전 ID} 값을 넣어 생성한다. 동의어사전은 stemmer, token, kma 에서만 지원되고 ngram, matrix, like의 경우는 지원하지 않는다.

[애널라이저 등록에서 동의어 사전 적용의 예]

```
{
  "analyzer": {
    "title_kma": {
      "lae": "kma-black-filter",
      "regulator": "R1",
      "dictionary": [
        "synonym-dic-1",
        "synonym-dic-2"
      ]
    },
    "content_kma": {
      "lae": "kma-black-filter",
      "dictionary": [
        "synonym-dic-1"
      ]
    }
  },
  "regulator": {},
  "lae": {}
}
```

4.5.3. 금칙어 사전 적용

[표 4.13 금칙어 사전 내부 옵션]

옵션	설명
dictionary	애널라이저 등록에서 “analyzer” 를 생성할 때, {금칙어 사전 ID} 값을 넣어 생성한다. 금칙어사전은 stemmer, token, kma 에서만 지원되고 ngram, matrix, like의 경우는 지원하지 않는다.

[애널라이저 등록에서 금칙어 사전 적용의 예]

```
{
  "analyzer": {
    "title_kma": {
      "lae": "kma-black-filter",
      "regulator": "R1",
      "dictionary": [
        "stopword-dic-1"
      ]
    },
    "content_kma": {
      "lae": "kma-black-filter",
      "dictionary": [
        "stopword-dic-1",
        "stopword-dic-2"
      ]
    }
  },
  "regulator": {},
  "lae": {}
}
```

5. ModelServer

본 제품은 Language Model을 사용한 벡터 검색을 지원하며 여러 모델을 사용할 수 있도록 설계되었다. 벡터 검색 기능을 사용하기 위해서는 전용 모듈인 ModelServer를 이용해야한다.

5.1. ModelServer 등록

ModelServer 내용을 등록하거나 수정하기 위해 필요한 두 가지 데이터는 ModelServer 에 접근하기 위한 server값과 해당 서버 내에 있는 모델 이름이 작성된 name값이다. [5.1.3. ModelServer 등록 (초기화)] 에서 사용 예시를 볼 수 있다. 초기화 API를 사용하여 등록할 경우 기존에 등록된 ModelServer 정보가 삭제될 수 있으므로, 일부 업데이트는 5.1.4의 업데이트 API를 사용하여 처리해야 한다.

5.1.1. ModelServer server

server값에는 실제 서버에 접속할 수 있는 host url 값을 작성한다.

5.1.2. ModelServer name

name 값에는 서버 내에서 사용되는 실제 모델 이름을 작성해야 한다.

5.1.3. ModelServer 등록 (초기화)

- id 값: 예시에서는 "dpr-ver-1"처럼 고유한 식별자를 부여함
- server: 실제 모델이 구동되는 서버의 호스트 URL을 입력
- name: 실제 모델 서버에서 제공하는 모델 이름을 입력. 예를 들어 "wisenut/retriever/1.0.0" 같은 형식으로 작성한다.

검색엔진은 이 ModelServer 정보가 변경되면, 해당 id 를 필드 설정으로 가지고 있는 벡터 필드 정보를 Gateway 와 검색기에 반영한다.

[표 5.1 ModelServer 등록(초기화)]

HTTP Method	URL	비고
POST	/model-server	초기화

[모델과 서버 등록의 예시]

```
POST : http://127.0.0.1:9001/model-server
{
    "{model-id}": {
      "name": {model name},
      "server": {server address}
    }
}
```

5.1.4. ModelServer 업데이트 및 특정 model, server 해제

ModelServer 업데이트는 등록과 사용 방법이 거의 유사하다.

기존 ModelServer에 등록된 {model-id}인 경우 새로 업데이트 한 {model-id} 값으로 교체되고, 기존에 등록되지 않은 {model-id}인 경우 값이 새로 추가된다

[표 5.2 ModelServer 업데이트 및 삭제]

HTTP Method	URL	비고
POST	/model-server/update	모델 및 서버 업데이트
POST	/model-server/delete	모델 및 서버 삭제(해제)

[ModelServer 모델 추가 업데이트 예시]

```
POST : http://127.0.0.1:9001/model-server/update
{
    "{model-id}": {
```

```
        "name": {model name},
        "server": {server address}
    }
}
```

[ModelServer 삭제(해제) 예시]

```
POST : http://127.0.0.1:9001/model-server/delete
[
  "{model-id}"
]
```

5.1.5. ModelServer 조회

[표 5.3 ModelServer 조회]

HTTP Method	URL	비고
GET	/model-server	ModelServer 조회

Model Server 조회는 등록되어 있는 server 목록을 불러온다.

6. 언어 분석

6.1. 지원문자 집합

본 제품은 국내뿐만 아니라 각국의 언어를 처리할 수 있도록 설계되었다. 각 나라의 언어는 그것을 글로 표현하는 문자를 가지고 있고, 이 문자들을 컴퓨터에서 표현하기 위해서 문자집합(Character Set)이 존재한다. 본 제품에서는 모든 문자집합을 통일한 유니코드 형태의 문자를 인코딩(Encoding)하는 UTF-8 방식을 지원한다.

문자집합은 검색엔진의 입력과 출력으로 나오는 문자들을 표현할 때 쓰인다. 검색엔진 내부적으로도 유니코드(UTF-8) 형태로 처리된다.
따라서 검색엔진은 유니코드의 색인과 검색을 지원한다.

[표 6.1 지원 문자 집합]

언어	표현 문자집합 (Character Set)
유니코드 (다국어)	UTF-8

6.2. 색인어 및 검색어 분석

색인어 및 검색어 분석은 검색의 기본 과정이다. 색인어 분석은 색인 대상 문서의 메타 정보를 어떠한 방식을 이용하여 주요한 색인어를 추출할지 결정하는 것이다. 즉, 문서의 메타 정보가 어떠한 의미를 포함하고 있으며, 이러한 의미를 어떻게 추출하는 가에 따라 검색의 품질 및 정확성이 보장된다.

검색어 분석은 색인어 분석과 동일한 알고리즘으로 동작하지만 그 대상은 사용자가 입력한 검색어가 된다. 색인어 및 검색어 분석은 레큘레이션이라는 과정을 거쳐서 색인어 및 검색어를 추출한다. 레큘레이션은 입력 단어를 특정 문자로 구분하는 작업을 말하며, 색인어 및 검색어

추출은 검색을 위한 주요 단어를 추출하는 작업을 말한다. 아래는 색인어 및 검색어를 추출하는 과정이다.

[그림 6.1 색인어 및 검색어 분석 과정]

문서 > 레귤레이션 > 색인어 추출 > 색인DB < 검색어 추출 < 레귤레이션 < 검색어

6.2.1. 레귤레이터 – 구분자 및 병합자 지정

특정 문자를 기준으로 문자열을 나누거나 병합하는 과정을 레귤레이터(Regulator)이라고 한다. 이러한 기준이 되는 문자를 '구분자' 혹은 '병합자'라고 칭한다. 색인기와 검색기에서는 분석 대상 문자열을 토큰(token)으로 분리하여, 색인어 및 검색어 추출기의 입력으로 사용하게 된다. 이때 토큰으로 분리할 기준 문자를 설정하는 것이 레귤레이터 설정이다.

먼저, 아래 표에 나열된 유니코드 문자는 '기본 구분자'로 사용되며, 별도의 레귤레이션 설정이 없을 경우 해당 문자를 기준으로 문자열을 나눈다.

[표 6.2 기본구분자]

0X0000	NUL	0X000B	VT	0X0016	SYN	0X0021	!	0X002C	,	0X005B	[
0X0001	SOH	0X000C	FF	0X0017	ETB	0X0022	“	0X002D	-	0X005C	\
0X0002	STX	0X000D	CR	0X0018	CAN	0X0023	#	0X002E	.	0X005D	]
0X0003	ETX	0X000E	SO	0X0019	EM	0X0024	$	0X002F	/	0X005E	^
0X0004	EOT	0X000F	SI	0X001A	SUB	0X0025	%	0X003A	:	0X005F	_
0X0005	ENQ	0X0010	DLE	0X001B	ESC	0X0026	&	0X003B	;	0X0060	`
0X0006	ACK	0X0011	DC1	0X001C	FS	0X0027	‘	0X003C	<	0X007B	{

0X0007	BEL	0X0012	DC2	0X001D	GS	0X0028	(	0X003D	=	0X007C	|
0X0008	BS	0X0013	DC3	0X001E	RS	0X0029	)	0X003E	>	0X007D	}
0X0009	HT	0X0014	DC4	0X001F	US	0X002A	*	0X003F	?	0X007E	~
0X000A	LF	0X0015	NAK	0X0020	SP	0X002B	+	0X0040	@	0X007F	DEL

예를 들어, 레귤레이션을 설정하지 않은 경우에 “안녕하세요? 와이즈넛! 홍길동입니다.”라는 문자열에서는 “안녕하세요”, “와이즈넛”, “홍길동입니다”가 토큰으로 추출되고, 이 토큰에서 다시 색인어를 추출하게 된다.

또한 본 제품은 특정 문자를 무시하고 앞뒤의 문자열을 붙이는 ‘병합자’ 개념을 지원한다. 예를 들어 [C#J#B]를 처리할 때 # 을 구분자로 정하는지, 병합자로 정하는지에 따라 분석이 달라진다

[C#J#B] 레귤레이션 처리 시

- #이 ‘구분자’일 경우 : C, J, B 로 분리하여 추출
- #이 ‘병합자’일 경우 : CJB 로 합쳐서 추출

이러한 특수문자를 처리방식을 레귤레이터(Regulator)라고 부르고, 이 레귤레이터는 분석기(Analyzer) 에서 호출되어 사용할 수 있다. 레귤레이터에는 일반 특수 문자나 한국어, 중국어, 일본어 등에 포함된 그림 문자를 지정하거나 16진수의 code값으로 지정할 수 있다.

[표 6.3 레귤레이터 종류]

레귤레이터 종류	설명
allow	allow는 정의된 특수 문자를 구분자에서 제외함으로서 해당 특수 문자가 색인어로 추출된다. allow 레귤레이터는 [표 6.2 기본 구분자]에서 정의된 특수 문자를 구분자로 설정하고, value와 code에 정의된 특수 문자를 구분자에서 제외한다.

divide	divide 값으로 추가한 특수 문자만 '구분자'로 설정한다. divide 레귤레이터는 [표 6.2 기본 구분자]에서 정의된 특수 문자를 사용하지 않고, 설정한 value와 code에 정의된 특수문자만 구분자로 설정한다.
unite	미리 정의된 특수 문자를 사용하지 않고, unite 에 정의된 특수 문자만 '병합자'로 설정한다. unite 레귤레이터는 [표 6.2 기본 구분자]에서 정의된 특수 문자를 사용하지 않고, 설정한 value와 code에 정의된 특수문자만 병합자로 설정한다.
allAllow	*** 주의 : 레귤레이터의 key값 (allAllow, allow, divide, unite)은 대소문자 구분해주어야한다. * allAllow : 기존 V5에서 allow나 divide, unite 값으로 빈값("")을 넣어주게 되면, 기본구분자를 모두 허용하는 효과가 난다. V7에서는 allAllow 옵션을 이용해 해당 기능을 구현하였으며, allow나 divide, unite 값으로 빈값("")을 넣을 수 없다. True로 등록 시, 매뉴얼의 기본구분자표와 더불어 8진수로 표현 가능한 모든 특수문자가 허용된다. *allAllow는 allow, unite와의 조합이 가능하나, divide와 조합은 허용되지 않는다.*

[표 6.4 레귤레이터 내부 옵션]

옵션	설명
code	16진수의 code 값으로 레귤레이터 값을 설정할 수 있다. {startCode}~{endCode} 의 패턴으로 범위값을 입력할 수도 있다.
value	한 글자로 된 특수문자를 직접 입력하여 설정할 수 있다.

[애널라이저 등록에서 레귤레이션 설정의 예]

```
{
  "analyzer": {
    "kma": {
      "lae": "kma-black",
      "regulator": "R2",
      "dictionary": []
    },
    "token-with-regulator": {
      "lae": "token",
      "regulator": "R1"
```

```
      }
    },
    "regulator": {
      "R1": {
        "allow": {
          "code": [
            "0x003A~0x003E",
            "0x0040"
          ],
          "value": [
            "#",
            "="
          ]
        }
      },
      "R2": {
        "divide": {
          "value": [
            ",",
            "."
          ]
        }
      },
      "R3": {
        "unite": {
          "value": [
            "#"
          ]
        }
      },
      "R4": { //allAllow 옵션
        "allAllow": "true"
      }
    },
    "lae": {
```

```
      "kma-black": {
        "type": "kma",
        "kmaOption": "A+R2",
        "licensePath": "korean/license.xml",
        "knowledgePath": "korean/wkb_nocomp.klg",
        "dictionary": [
          "usernoun1"
        ]
      },
      "token": {
        "type": "token"
      }
    }
  }
```

6.2.2. 고급 레귤레이션

irag v5, v6의 경우 레귤레이터 정의 시, 하나의 method(allow, divide, unite) 만 지정이 가능하였다. iRAG Search Engine에서는 하나의 레귤레이터에 다수의 method 설정이 가능하다. 다수의 method를 적용하는 방법은 '동시 적용', '각각 적용'으로 총 2가지 방법이다. 하단의 예시를 통해 다중 method 적용법과 결과값이 어떻게 상이한지 확인할 수 있다.

6.2.2.1. 다중 method : 동시 적용

다중 method를 동시 적용은 초기 애널라이저 설정 시 진행한다. 이때, 한 레귤레이터에 다중 method를 정의한다.

[표 6.5 동시 적용 내부 옵션]

옵션	설명
{regulator_id}	레귤레이터의 아이디 값을 정의한다. 하단의 method를 여러 개 등록 가능하다.

	*** 주의 : 레귤레이터의 key값 (allAllow, allow, divide, unite)은 대소문자 구분해주어야한다. * allAllow : 기존 V5에서 allow나 divide, unite 값으로 빈값(“”)을 넣어주게 되면, 기본구분자를 모두 허용하는 효과가 난다. V7에서는 allAllow 옵션을 이용해 해당 기능을 구현하였으며, allow나 divide, unite 값으로 빈값(“”)을 넣을 수 없다. True로 등록 시, 매뉴얼의 기본구분자표의 모든 특수문자가 허용된다. * allow : allow는 정의된 특수 문자를 구분자에서 제외함으로서 해당 특수 문자가 색인어로 나오게 한다. * divide : divide 값으로 추가한 특수 문자만 ‘구분자’로 설정한다. * unite : 미리 정의된 특수 문자를 사용하지 않고, unite 에 정의된 특수 문자만 ‘병합자’로 설정한다.

- 주의1 : allow method(allAllow 포함)는 divide와 함께 사용할 수 없다. divide method 정의를 살펴보면, divide에 추가한 특수문자만 ‘구분자’로 설정하고 나머지 특수문자는 모두 구분자에서 제외된다. allow method는 allow에 추가한 특수문자를 허용하는 것을 의미한다. 하지만 divide를 사용하게 되면, divide에 입력한 특수문자를 제외한 모든 특수문자값이 허용되므로 allow의 의미와 상충된다. allow와 divide를 동시에 사용했을 경우, 사용자의 의도를 파악할 수 없으므로 allow, divide 동시적용은 지원하지 않는다.
- 주의2 : allAllow, unite, allow를 함께 사용할 때에는 우선순위가 적용되므로 주의하여야한다. allow + unite 조합에서는 unite의 우선순위가 높다. 예를 들어 allow에 ‘@’, ‘|’ 문자를 적용하고 unite에 ‘@’ 문자를 적용하는 경우, ‘@’문자는 unite 옵션이 적용되고 ‘|’ 문자는 allow 옵션이 적용된다. allAllow + unite 조합에서는 allAllow의 우선순위가 높다. 따라서, 기본구분자를 제외한 특수문자를 unite 옵션으로 사용하는 경우에만 의미를 가진다.

[애널라이저 설정의 예]

```
POST : http://127.0.0.1:9001/analysis
 {
   "analyzer": {
     "token_R1": {
```

```
      "lae": "token",
      "regulator": "R1"
    },
    "token_R2": {
      "lae": "token",
      "regulator": "R2"
    },
    "token_R3": {
      "lae": "token",
      "regulator": "R3"
    }
  },
  "regulator": {
    "R1": { //allow 레귤레이터
      "allow": {
        "value": [
          "_",
          "."
        ]
      }
    },
    "R2": { //unite 레귤레이터
      "unite": {
        "value": [
          "/"
        ]
      }
    },
    "R3": { //allow와 unite '동시적용' 하고 싶을 경우
      "allow": {
        "value": [
          "_",
          "."
        ]
      },
```

```
        "unite": {
          "value": [
            "/"
          ]
        }
      },
      "R4": { //allAllow 옵션
        "allAllow": "true"
      }
    },
    "lae": {
      "token": {
        "type": "token"
      }
    }
}
```

이후, 컬렉션 세팅 시, '동시 적용' 되어있는 레귤레이터를 등록한다.

```
POST : http://127.0.0.1:9001/collections/sample
{
  "numOfShards": 1,
  "numOfReplication": 0,
  "recentQuery": 5,
  "fields": [
    "DOCID",
    "DATE",
    "TITLE",
    "CONTENT",
    "DEPARTMENT"
  ],
  "indexFields": {
    "DATE": {
      "type": "date"
    },
```

```
    "TITLE": {
      "type": "text",
      "analyzer": [
        "token_R1",
        "token_R2"
      ]
    },
    "CONTENT": {
      "type": "text",
      "analyzer": [
        "token_R3"
      ]
    },
    "DEPARTMENT": {
      "type": "long"
    }
  }
}
```

위의 예시에서는 CONTENT 필드에 allow와, unite 레귤레이터를 '동시 적용'하였다. [형태소 분석 조회] 기능을 통해 해당 필드에 레귤레이터가 어떻게 적용되는지 확인할 수 있다.

[형태소 분석 조회 예시]

```
POST : http://127.0.0.1:9001/search/sample/query-morpheme
{
  "query": "홍길동_시력_0.2(왼쪽)_키/164",
  "searchField": [
    "CONTENT"
  ],
  "synonymExpansion": 1,
  "useSynonym": 1,
  "useLa": 1
```

```
}
```

[결과값 예시]

```
{
  "result": {
    "CONTENT": {
      "token_R3": [
        "홍길동_시력_0.2",
        "왼쪽",
        "_키164"
      ]
    }
  }
}
```

CONTENT 필드에는 "_", "." 를 allow 하고 "/" 를 기준으로 unite 하도록 설정되어 있는 상태이다. 해당 필드는 allow 레귤레이터와 unite 레귤레이터가 동시적용되어 [홍길동_시력_0.2], [왼쪽], [_키164] 의 키워드로 추출되게 된다.

6.2.2.2. 다중 method: 각각 적용

다중 method를 '각각 적용'하기 위해서는 여러 개의 레귤레이터를 정의해야한다. 하단의 예시와 같이 allow method를 가진 레귤레이터, unite method를 가진 레귤레이터를 각각 정의하고, 해당 레귤레이터를 두 개의 다른 애널라이저에 각각 등록한다.

[표 6.6 각각 적용 내부 옵션]

옵션	설명
{regulator_id}	레귤레이터의 아이디 값을 정의한다. 하단의 method를 여러 개 등록 가능하다. *** 주의 : 레귤레이터의 key값 (allAllow, allow, divide, unite)은 대소문자 구분해주어야한다.

	* allAllow : 기존 V5에서 allow나 divide, unite 값으로 빈값(“”)을 넣어주게 되면, 기본구분자를 모두 허용하는 효과가 난다. V7에서는 allAllow 옵션을 이용해 해당 기능을 구현하였으며, allow나 divide, unite 값으로 빈값(“”)을 넣을 수 없다. True로 등록 시, 매뉴얼의 기본구분자표의 모든 특수문자가 허용된다. * allow : allow는 정의된 특수 문자를 구분자에서 제외함으로서 해당 특수 문자가 색인어로 나오게 한다. * divide : divide 값으로 추가한 특수 문자만 ‘구분자’로 설정한다. * unite : 미리 정의된 특수 문자를 사용하지 않고, unite 에 정의된 특수 문자만 ‘병합자’로 설정한다.

[애널라이저 설정의 예]

POST : http://127.0.0.1:9001/analysis

```
{
  "analyzer": {
    "token_R1": {
      "lae": "token",
      "regulator": "R1"
    },
    "token_R2": {
      "lae": "token",
      "regulator": "R2"
    },
    "token_R3": {
      "lae": "token",
      "regulator": "R3"
    }
  },
  "regulator": {
    "R1": { //allow 레귤레이터
      "allow": {
        "value": [
          "_",
          "."
        ]
```

```
      }
    },
    "R2": { //unite 레귤레이터
      "unite": {
        "value": [
          "/"
        ]
      }
    },
    "R3": { //allow와 unite '동시적용' 하고 싶을 경우
      "allow": {
        "value": [
          "_",
          "."
        ]
      },
      "unite": {
        "value": [
          "/"
        ]
      }
    },
    "R4": { //allAllow 옵션
      "allAllow": "true"
    }
  },
  "lae": {
    "token": {
      "type": "token"
    }
  }
}
```

또한 컬렉션 세팅 시, '각각 적용' 되어있는 레귤레이터를 등록해야 한다.

POST : http://127.0.0.1:9001/collections/sample

```
{
  "numOfShards": 1,
  "numOfReplication": 0,
  "recentQuery": 5,
  "fields": [
    "DOCID",
    "DATE",
    "TITLE",
    "CONTENT",
    "DEPARTMENT"
  ],
  "indexFields": {
    "DATE": {
      "type": "date"
    },
    "TITLE": {
      "type": "text",
      "analyzer": [
        "token_R1",
        "token_R2"
      ]
    },
    "CONTENT": {
      "type": "text",
      "analyzer": [
        "token_R3"
      ]
    },
    "DEPARTMENT": {
      "type": "long"
    }
  }
}
```

위의 예시에서는 TITLE 필드에 allow와, unite 레귤레이터를 가진 애널라이저를 각각 등록하였다.

[형태소 분석 조회] 기능을 통해 해당 필드에 레귤레이터가 어떻게 적용되는지 확인할 수 있다.

[형태소 분석 조회 예시]

```
POST : http://127.0.0.1:9001/search/sample/query-morpheme

{
  "query": "홍길동_시력_0.2(왼쪽)_키/164",
  "searchField": [
    "TITLE"
  ],
  "synonymExpansion": 1,
  "useSynonym": 1,
  "useLa": 1
}
```

[결과값 예시]

```
{
 "result": {
  "TITLE": {
   "token_R1": [
    "홍길동_시력_0.2",
    "164",
    "왼쪽",
    "_키"
   ],
   "token_R2": [
    "홍길동_시력_0.2",
    "왼쪽",
    "_키164"
   ]
  }
 }
}
```

TITLE 필드에는 "_", "." 를 allow 하는 애널라이저 하나, "/" 를 기준으로 unite 하도록 설정되어 있는 애널라이저 하나가 각각 등록되어 있는 상태이다. 해당 필드는 allow 레귤레이터가 적용되어

추출된 키워드인 [홍길동_시력_0.2], [164], [왼쪽], [_키] 그리고 unite 레귤레이터가 적용되어 추출된 키워드 [홍길동_시력_0.2], [왼쪽], [_키164] 를 각각 애널라이저 별로 확인이 가능하다.

위의 두 예시처럼 애널라이저와 컬렉션의 필드에 등록된 레귤레이터의 설정값에 따라 같은 쿼리, 문서임에도 불구하고 결과값은 매우 상이하다. 따라서 원하는 결과값이 무엇인지 정확히 파악한 뒤, 해당 결과값을 얻기 위한 레귤레이터 등록 방법을 충분히 숙지해야 한다.

6.2.3. 색인어 및 검색어 추출

색인어 및 검색어 추출은 언어 독립 분석과 언어 종속 분석 방법으로 나뉜다. 언어 독립 분석은 문서의 언어와는 상관없이 특정 규칙(예를 들어, 임의의 2음절씩 분리하는 것)에 따라 색인어 및 검색어를 추출하며, 언어 종속 분석은 언어별 사전과 형태소 분석 알고리즘에 따라 주요 단어를 추출하는 것을 말한다. 즉, 언어 종속 분석은 해당 언어의 형태소 분석기가 포함되어 있음을 의미하며, 문서의 언어와 다른 분석 방식을 사용할 경우, 정확한 주요 단어 추출이 이루어지지 않는다. (예: 영어 문서를 CMA(중국어 형태소 분석기)로 분석할 경우, 정확한 결과를 얻을 수 없다.)

6.2.3.1. 언어 독립 분석

irag에서는 언어 독립 분석을 지원하기 위해 token, ngram, like, matrix 기능을 제공하고 있으며, 해당 기능들은 대소문자를 구별하여 색인 및 검색이 가능하다. 아래 세부 챕터에서 각 분석 방법에 대해서 알아본다.

6.2.3.1.1. token

token 분석은 입력 단어를 그대로 추출하는 방법이다. 즉, 레귤레이터에서 분리된 단어를 추출하게 된다.

[표 6.7 token 방식 색인어 및 검색어 추출의 예]

입력	추출 결과	설명
검색엔진	검색엔진	입력을 색인어로 추출한다.
개발	개발	입력을 색인어로 추출한다.
및	및	입력을 색인어로 추출한다.
구축	구축	입력을 색인어로 추출한다.

6.2.3.1.2. ngram

ngram 분석은 입력 단어를 N 음절씩 분리하여 추출하는 방법이다. 형태소 분석기가 없는 언어에 대해서 적용할 수 있는 방법이다. 기본 분석은 2음절씩 분리하는 bi-gram이며, level 옵션으로 tri-gram(3음절씩 분리)까지 추출할 수 있다.

[표 6.8 ngram 방식 색인어 및 검색어 추출의 예]

입력	추출 결과	설명
검색엔진	검색 색엔 엔진	2 음절씩 색인어로 추출한다.
개발	개발	2 음절씩 색인어로 추출한다.
및	및	2 음절씩 색인어로 추출한다.
구축	구축	2 음절씩 색인어로 추출한다.

6.2.3.1.3. like

like 분석은 입력 단어를 최소 길이부터 최대 길이까지 분리하여 추출하는 방법이다. 일반적으로 DBMS의 like와 비슷한 효과를 내기 위해서 사용한다. 아래는 최소 길이를 2로 설정하고, 최대 길이를 6으로 설정한 경우에 추출되는 예이다.

[표 6.9 like 방식 색인어 및 검색어 추출의 예]

입력	추출 결과	설명
검색엔진	검색 색엔 엔진 검색엔 색엔진 검색엔진	2 음절부터 4 음절까지 색인어로 추출한다.
개발	개발	2 음절 색인어로 추출한다.
및	및	최소 길이가 2 이여도 1음절 색인어로 추출한다.
구축	구축	2 음절 색인어로 추출한다.

6.2.3.1.4. matrix

matrix 분석은 입력 단어를 앞에서 혹은 뒤에서부터 분리하여 추출하는 방법이다. 아래는 “begin”=true 설정일 경우 앞에서부터 단어를 분리하는 예이다.

[표 6.10 matrix 방식 색인어 및 검색어 추출의 예 (“begin”=true)]

입력	추출 결과	설명
검색엔진	검 검색 검색엔 검색엔진	앞에서부터 색인어로 추출한다.
개발	개 개발	앞에서부터 색인어로 추출한다.
및	및	앞에서부터 색인어로 추출한다.
구축	구 구축	앞에서부터 색인어로 추출한다.

또한, “end”=true 설정을 하였을 경우에는 아래와 같이 분리하여 추출한다.

[표 6.11 matrix 방식 색인어 및 검색어 추출의 예 (“end”=true)]

입력	추출 결과	설명
검색엔진	검색엔진 색엔진 엔진 진	뒤에서부터 색인어로 추출한다.
개발	개발 발	뒤에서부터 색인어로 추출한다.
및	및	뒤에서부터 색인어로 추출한다.
구축	구축 축	뒤에서부터 색인어로 추출한다.

[표 6.12 matrix 방식 색인어 및 검색어 추출의 예 (“begin”=true) (“end”=true)]

입력	추출 결과	설명
검색엔진	검 검색 검색엔 검색엔진 색엔진 엔진 진	앞에서부터 색인어로 추출하고, 뒤에서부터도 색인어를 추출한다.
개발	개 개발 발	앞에서부터 색인어로 추출하고, 뒤에서부터도 색인어를 추출한다.
및	및	앞에서부터 색인어로 추출하고, 뒤에서부터도 색인어를 추출한다.
구축	구 구축 축	앞에서부터 색인어로 추출하고, 뒤에서부터도 색인어를 추출한다.

6.2.3.2. 언어 종속 분석

언어 종속 분석은 각 언어별 형태소 분석 방식에 따라 형태소 분석을 수행한다.

6.2.3.2.1. 형태소 분석

형태소 분석은 언어별 사전과 형태소 분석 알고리즘을 이용하여, 형태소를 분리하고 색인어 혹은 검색어에 필요한 단어를 추출한다. 예를 들어, 기본적으로 한국어 형태소 분석은 입력된 단어와 명사를 추출하고, 옵션을 활용하면 동사의 원형까지 추출할 수 있는 기능을 제공한다.

[표 6.13 형태소 분석 방식 색인어 및 검색어 추출의 예]

입력	추출 결과	설명
검색엔진	검색/엔진/검색엔진	입력 단어 및 단위 명사 추출
개발	개발	입력 단어 및 단위 명사 추출
및	및	입력 단어 추출
구축	구축	입력 단어 및 단위 명사 추출

6.2.3.2.2. stemmer

stemmer는 입력된 영어 및 서유럽어 단어의 동사 원형을 추출하는 방식이다.

[표 6.14 stemmer 분석 방식 색인어 및 검색어 추출의 예]

입력	추출 결과	설명
likes	like	동사 원형 추출
swimming	swim	동사 원형 추출

6.3. 형태소 분석

형태소 (形態素)란? 일정한 의미가 있는 가장 작은 말의 단위를 뜻하며, 우리가 일반적으로 사용하고 있는 단어는 여러 개의 형태소로 구성되어 있다. “중학국어문법이다”라는 문장은 “중학”, “국어”, “문법”, “이”, “다”라는 여러 개의 명사, 어미로 구성되어 있는 것을 볼 수 있다.

만약 검색 엔진에서 형태소 분석과 같은 언어 분석 도구를 사용하지 않고 토큰 단위로 색인어를 추출한다면 높은 검색 품질을 기대하기 힘들다. 예를 들어 “대학생들은”, “대학생이”, “대학생도”, “대학생과”, “대학생을”과 같이 다른 형태소들과 결합된 검색 대상 문서를 “대학생”으로 검색할 수 없다.

검색 엔진에서의 형태소 분석은 검색 대상의 문서로부터 좀 더 효율적인 검색이 가능하도록 토큰에서 의미를 가진 최소한의 단위인 형태소만 뽑아 색인어로 추출이 가능하게 한다.

6.3.1. 한국어 형태소 분석 (KMA)

본 제품에서 제공하는 한국어 분석기는 기본적으로 한자를 한글로 변환하여 색인하므로 한글로 검색할 경우 해당되는 한자가 포함된 문서 출력이 가능하다. 예를 들어 “發展” 문자열은 “발전”으로 변환되어 색인 되므로 “발전” 키워드로 해당 한자가 위치한 문서를 검색할 수 있다. 또한 한국어 분석기에는 아래와 같이 다양한 옵션을 지정할 수 있다. 아래의 옵션들은 색인 및 검색 시 형태소 분석에 적용된다.

6.3.1.1. 토큰 색인 옵션(A)

기본적인 한국어 분석 방식은 토큰에서 명사만 색인어로 추출하게 된다. 하지만, “구 검색”, “인접 검색” 등과 같은 고급 검색을 연산을 사용하기 위해서는 토큰 자체를 색인해야 할 필요가 있다. 고급 검색을 사용하지 않는 경우에는 토큰을 색인하지 않도록 설정하여 색인 데이터의 크기를 줄이는 것이 좋다. 예를 들어, “검색엔진은”을 “토큰 색인 옵션”을 설정하여 색인할 경우에는

“검색”, “엔진”, “검색엔진은”이 색인어로 추출되며 “토큰 색인 옵션”을 설정하지 않고 색인할 경우에는 “검색”, “엔진”만 색인어로 추출된다. 또한 자연어 쿼리를 분석하기 위해서 가장 중요한 명사 형태소만 포함된 토큰 자체만 추출하는 옵션도 포함되어 있다.

[표 6.15 한국어 형태소 분석 - 토큰 색인 옵션 (A)]

옵션	기본값	옵션 설명
A+	O	명사가 포함된 토큰을 색인어로 추출 예) [검색엔진은] >> [검색, 엔진, 검색엔진은]
A-		토큰에서 명사만 색인어로 추출 예) [검색엔진은] >> [검색, 엔진]
A*		모든 토큰을 색인어로 추출 예) [검색엔진은 빠르다] >> [검색, 엔진, 검색엔진은, 빠르다]

6.3.1.2. 접사 관련 옵션 (F)

접사(접두사, 접미사)를 명사에 결합하거나 분해할 수 있는 옵션을 제공한다.

[표 6.16 한국어 형태소 분석 - 접사 관련 옵션 (F)]

옵션	기본값	옵션 설명
F+		접사를 명사와 분해하지 않음 예) [기업용] >> [기업용] 주의 : F-옵션의 경우 A-옵션을 함께 적용해야 정확한 분석 결과를 확인할 수 있다. 예) Input : 기업용 F- 옵션을 적용할 경우 [기업]만 추출되어야하는데, default 값인 A+ 옵션에 의해 [기업용]이 추출되어 [기업] [기업용]이 모두 F-에 의해 추출된 것 처럼 보이게 된다.
F-		접사를 명사와 분해 예) [기업용] >> [기업]
F*	O	접사를 명사와 분해된 형태, 분해되지 않은 형태로 추출 예) [기업용] >> [기업, 기업용]

6.3.1.3. 분석 결과 개수 옵션 (R)

형태소 분석 결과를 모두 추출할 것인지, 분석 결과 중 우선 순위를 두어 순위가 높은 상위 몇 개를 택하여 추출할 것인지를 결정한다. 예를 들어, “대학생선교회”는 “대학생”, “선교회”와 “대학”, “생선”, “교회”의 두 개의 분석 결과가 나올 수 있으며 둘 중에 어느 하나를 색인할 것인지를 결정해야 한다. 첫 번째 분석 결과와 같이 색인된다면 원하는 결과에 근접한 문서를 검색할 수 있지만, 두 번째 분석 결과와 같이 색인된다면 원하는 결과와는 조금 거리가 있는 문서가 검색될 수 있다. 그래서 형태소 분석에서는 각 분석 결과에 경험적인(heuristic) 통계치를 이용하여 각 분석 결과에 점수를 부여, 이 점수를 이용하여 우선 순위를 정하게 된다.

[표 6.17 한국어 형태소 분석 - 분석 결과 개수 옵션 (R)]

옵션	기본값	옵션 설명
R+	O	분석 결과 중에서 가장 적합한 2개의 결과를 선택하여 색인어로 추출 예) [대학생선교회] >> [대학생, 선교회] 주의 : A- 옵션의 경우 token 또한 그대로 색인하므로, A- 옵션을 추가하여야 정확한 결과값을 얻을 수 있다.
R-		모든 분석 결과를 색인어로 추출 예) [대학생선교회] >> [대학생, 선교, 선교회] 주의 : A- 옵션의 경우 token 또한 그대로 색인하므로, A- 옵션을 추가하여야 정확한 결과값을 얻을 수 있다.
R1~R5		분석 결과 중에서 숫자에 명시된 순위만큼, 적합한 결과를 선택하여 색인어로 추출. R1은 가장 적합한 상위 1개의 분석 결과만 색인어로 추출하며, R2는 상위 2개, R5는 상위5개의 분석 결과를 색인어로 추출. 예) [대학생선교회] ■ R1 : [대학생, 선교회] ■ R2 : [대학생, 선교회] ■ R3 : [대학생, 선교회] ■ R4 : [대학생, 선교, 선교회] ■ R5 : [대학생, 선교, 선교회]

		주의 : ■ 분석된 상위의 확률이 같은 경우, 동일한 분석 결과를 출력할 수 있다. ■ A- 옵션의 경우 token 또한 그대로 색인하므로, A- 옵션을 추가하여야 정확한 결과값을 얻을 수 있다.

6.3.1.4. 숫자 형식 결합 옵션 (N)

숫자와 단위 구분 기호(,) 및 소수점 기호(.)와 결합 여부를 설정한다.

[표 6.18 한국어 형태소 분석 - 숫자 형식 결합 옵션 (N)]

옵션	기본값	옵션 설명
N+		숫자와 단위 구분 기호(,) 및 소수점 기호(.)와 결합 예) [12,000] >> [12,000] 예) [3.14] >> [3.14] 주의 : 형태소 분석기의 경우 tokenizer(regulator)가 처리된 결과값이 들어오게 되므로, 구분 기호와 소수점 기호를 allow regulator에 등록해주어야한다. 또한, A- 옵션의 경우 token을 그대로 색인하므로, A- 옵션을 추가하여야 정확한 결과값을 얻을 수 있다.
N-	O	숫자와 단위 구분 기호(,) 및 소수점 기호(.)와 결합하지 않음 예) [12,000] >> [12, 000] 예) [3.14] >> [3, 14] 주의 : 형태소 분석기의 경우 tokenizer(regulator)가 처리된 결과값이 들어오게 되므로, 구분 기호와 소수점 기호를 allow regulator에 등록해주어야한다. 또한, A- 옵션의 경우 token을 그대로 색인하므로, A-옵션을 추가하여야 정확한 결과값을 얻을 수 있다.

- 주의 : N+ 옵션을 사용할 경우, 해당 필드에 단위 구분 기호와 소수점 기호(,.)가 allow로 등록 된 regulator를 설정해야 숫자와 기호가 결합된 형태로 색인어가 추출된다.

6.3.1.5. 숫자 추출 옵션 (N)

숫자를 색인어로 추출할 것인지 아닌지를 결정한다. ‘N’ 뒤의 숫자는 추출할 숫자의 길이를 의미한다. (예: N1 – 길이가 1 이상인 숫자를 색인어로 추출)

[표 6.19 한국어 형태소 분석 – 숫자 추출 옵션 (N)]

옵션	기본값	옵션 설명
N0		숫자를 색인어로 추출하지 않음 예) [디자인패턴12] >> [디자인, 패턴] 주의 : A- 옵션의 경우 token 또한 그대로 색인하므로, A- 옵션을 추가하여야 정확한 결과값을 얻을 수 있다.
N1	O	길이가 1 이상인 숫자를 색인어로 추출 예) [디자인패턴12] >> [디자인, 패턴, 12] 주의 : A- 옵션의 경우 token 또한 그대로 색인하므로, A-옵션을 추가하여야 정확한 결과값을 얻을 수 있다.
N2~N9		N2는 길이가 2 이상인 숫자를 색인어로 추출하며, N9는 길이가 9 이상인 숫자를 색인어로 추출 주의 : A- 옵션의 경우 token 또한 그대로 색인하므로, A-옵션을 추가하여야 정확한 결과값을 얻을 수 있다.

Note.
일부 단어에서 결과값이 다르게 형태소 분석이 될 수 있으나 이는 코퍼스(말뭉치, 말모둠) 언어 표본자료에 따른 정상 동작입니다. ex)글자와 숫자 구분 10, 20, 80 등

6.3.1.6. 용언 색인 옵션 (V)

어간과 어미로 구성되어 있는 용언(동사, 형용사)의 색인어 추출 여부를 결정하는 옵션이다.

용언은 다양한 활용형으로부터 원형을 복원하여 어간만 색인어로 추출한다. 예를 들어 “때려”, “때리고”, “때리지도”, “때린”의 다양한 활용형으로부터 “때리”라는 어간만 색인어로 추출한다.

[표 6.20 한국어 형태소 분석 – 용언 색인 옵션 (V)]

옵션	기본값	옵션 설명
V+		용언의 어간만 색인어로 추출 예) [때리지도] >> [때리] 예) [먹고] >> [먹] 주의 : A- 옵션의 경우 token 또한 그대로 색인하므로, A- 옵션을 추가하여야 정확한 결과값을 얻을 수 있다.
V-	O	용언의 어간을 색인어로 추출하지 않음 예) [때리지도] >> [] 주의 : A- 옵션의 경우 token 또한 그대로 색인하므로, A- 옵션을 추가하여야 정확한 결과값을 얻을 수 있다.

6.3.1.7. 단위성 의존명사 결합 옵션 (B)

숫자와 단위성 의존명사인 “개”, “번째”, “위”, “켤레”와 같이 단위 명사로 알려진 명사와 함께 출현할 경우, 이 둘을 결합하여 색인어로 추출한다.

[표 6.21 한국어 형태소 분석 – 단위성 의존명사 결합 옵션 (B)]

옵션	기본값	옵션 설명
B+	O	숫자와 단위성 의존명사를 결합하여 색인어로 추출 예) [12개를] >> [12개를, 12개] 주의 : B+옵션의 경우 N1 옵션을 함께 적용해야 정확한 분석 결과를 확인할 수 있다. 예) Input : 12개를

		B+ 옵션을 적용하면 [12개] [12개를]만 추출되어야하는데 default값인 N1 옵션에 의해 [12]가 추출되어 [12] [12개] [12개를]이 모두 B+에 의해 추출된 것 처럼 보이게 된다.
B-		숫자와 단위성 의존명사를 결합하지 않음 예) [12개를] >> [12개를, 12, 개] 주의 : 추출된 색인어 '12개를'는 A+ 옵션을 추가한 경우에 추출되며, A- 옵션을 추가한 경우에는 추출되지 않는다.

Note.
일부 단어에서 결과값이 다르게 형태소 분석이 될 수 있으나 이는 코퍼스(말뭉치, 말모둠) 언어 표본자료에 따른 정상 동작입니다. ex)말, 쌍, 쾌 등

6.3.1.8. 기호/알파벳/숫자 자동 조합 옵션 (X)

기호를 알파벳/숫자와 조합하거나 알파벳/숫자/기호 까지 조합할 수 있는 옵션이다. 일반적으로 특수 용어(3D 등)나 제품 코드 (IMT-123M) 등 알파벳/숫자/기호가 결합된 형태의 검색 품질을 높이기 위해서 사용된다. X* 옵션은 색인어가 다량으로 생성되어 주의 할 필요가 있다.

[표 6.22 한국어 형태소 분석 - 기호/알파벳/숫자 자동 조합 옵션 (X)]

옵션	기본값	옵션 설명
X+		기호를 제외한 알파벳/숫자만 결합하여 추출 예) [3D-KR/EN] >> [3, 3d, d, kr, en] 주의 : 형태소 분석기의 경우 tokenizer(regulator)가 처리된 결과값이 들어오게 되므로, 허용하고 싶은 기호를 allow regulator에 등록해주어야 한다. 또한, A- 옵션의 경우 token을 그대로 색인하므로, A- 옵션을 추가하여야 정확한 결과값을 얻을 수 있다.

X-	O	알파벳/숫자/기호를 분리 예) [3D-KR/EN] >> [3, d, kr, en] 주의 : 형태소 분석기의 경우 tokenizer(regulator)가 처리된 결과값이 들어오게 되므로, 허용하고 싶은 기호를 allow regulator에 등록해주어야 한다. 또한, A- 옵션의 경우 token을 그대로 색인하므로, A-옵션을 추가하여야 정확한 결과값을 얻을 수 있다.
X*		기호를 알파벳/숫자/기호에 결합 예) [3D-KR/EN] >> [3, 3d, 3d-, 3d-kr, 3d-kr/, 3d-kr/en, d, d-, …] 주의 : 형태소 분석기의 경우 tokenizer(regulator)가 처리된 결과값이 들어오게 되므로, 허용하고 싶은 기호를 allow regulator에 등록해주어야 한다. 또한, A- 옵션의 경우 token을 그대로 색인하므로, A-옵션을 추가하여야 정확한 결과값을 얻을 수 있다.

6.3.1.9. 복합명사 분해 옵션 (C)

복합명사에 대해서 분해 여부를 설정할 수 있다. 또한, 분해된 단일명사를 재조합하는 옵션도 제공된다.

[표 6.23 한국어 형태소 분석 - 복합명사 분해 옵션 (C)]

옵션	기본값	옵션 설명
C+	O	복합명사를 단일명사로 분해 예) [기업검색엔진] >> [기업, 검색, 엔진] C+ 옵션 사용 시, A- 옵션을 추가하지 않으면 default인 A+ 옵션(token을 그대로 색인어에 추가)으로 처리되어 [기업검색엔진] 이라는 색인어가 추가된다. 따라서 위와 같이 C+ 옵션의 결과값을 제대로 받기 위해서는 A-옵션을 추가해주어야한다.
C-		복합명사를 단일명사로 분해하지 않음 예) [기업검색엔진] >> [기업검색엔진]
C*		복합명사를 단일명사로 분해하고, 분해된 단일명사를 조합

		예) [기업검색엔진] >> [기업, 검색, 엔진, 기업검색, 검색엔진, 기업검색엔진]

6.3.1.10. 영어 스테밍 옵션 (S)

토큰이 영어일 경우 스테밍 적용 여부를 결정한다. 스테밍(stemming)이란, 영어 형태소 분석의 한 방법으로 입력된 단어의 어간(stem)만 추출하는 알고리즘이다. 예를 들어 likes은 like로 liked도 like와 같이 어간을 색인어로 추출하는 옵션이다.

[표 6.24 한국어 형태소 분석 – 영어 스테밍 옵션 (S)]

옵션	기본값	옵션 설명
S+	O	영어로 구성된 토큰에 스태밍을 적용하여 어간을 색인어로 추출 예) [likes] >> [likes like]
S-		영어로 구성된 토큰에 스태밍을 적용하지 않음 예) [likes] >> [likes]

6.3.2. 영어 형태소 분석

6.3.2.1. 명사 원형 색인 옵션 (M)

영어에서 명사는 단수/복수형으로 사용될 수 있는 복수형 명사를 단수형으로 바꾸어 색인어로 추출하는 옵션이다.

[표 6.25 영어 형태소 분석 – 명사 원형 색인 옵션 (M)]

옵션	기본값	옵션 설명
M+		복수형 명사에서 단수형 명사를 색인어로 추출 예) [likes] >> [likes like]
M-	**O**	복수형 명사에서 단수형 명사를 색인어로 추출하지 않음 예) [likes] >> [likes]

6.3.2.2. 동사 원형 색인 옵션 (V)

영어에서 동사는 다양한 활용형을 가지고 있다. “동사 원형 색인 옵션은” 은 동사로부터 원형을 색인어로 추출하는 옵션이다.

[표 6.26 영어 형태소 분석 – 동사 원형 색인 옵션 (V)]

옵션	기본값	옵션 설명
V+		동사의 원형을 색인어로 추출 예) [addicted] >> [addicted addict]
V-	**O**	동사의 원형을 색인어로 추출하지 않음 예) [addicted] >> [addicted]

6.3.3. 일본어 형태소 분석 (JMA)

6.3.3.1. 기본 옵션(Basic option)

6.3.3.1.1. 단위 명사 결합 옵션 (B)

숫자와 단위성 의존명사인 “口”, “食”, “杯”, “匹”와 같이 단위 명사로 알려진 명사와 함께 출현할 경우, 이 둘을 결합하여 색인어로 추출한다.

[표 6.27 일본어 형태소 분석 – 단위성 의존명사 결합 옵션 (B)]

옵션	기본값	옵션 설명
B+	O	숫자와 단위성 의존명사를 결합하여 색인어로 추출 예) [一食, 1食] >> [一, 一食, 1, 1食]
B-		숫자와 단위성 의존명사를 결합하지 않음 예) [一食, 1食] >> [一, 1] 주의 : B+ 옵션을 사용하지 않으면 단위 명사는 색인어로 추출하지 않는다.

Note.
일부 단어에서 결과값이 다르게 형태소 분석이 될 수 있으나 이는 코퍼스(말뭉치, 말모둠) 언어 표본자료에 따른 정상 동작입니다. ex)コマ(만화 한 컷에서의 '컷')

6.3.3.1.2. 후리가나 색인 옵션 (D)

후리가나는 일본어에서 한자를 읽는 법을 한자 근처에 표시해 주는 문자이며 읽는 문자도 색인어로 추출한다.

[표 6.28 일본어 형태소 분석 - 후리가나 색인 옵션 (D)]

옵션	기본값	옵션 설명
D-	O	후리가나를 색인어로 추출하지 않음 예) [振り仮名] >> [振り, 仮名]
D+		후리가나를 색인어로 추출 예) [振り仮名] >> [フリ, 振り, 仮名] 주의 : 숫자나 가타카나는 후리가나가 추출되지 않는다.

6.3.3.1.3. 접사 결합 옵션 (F)

토큰에서 명사와 접사 순서로 출현한 형태소를 결합하여 색인어로 추출할지 여부를 결정하는 옵션이다. "故障中"와 같이 고장을 나타내는 명사 "故障"(고장)과 "中"(중) 접사가 같이 출현한 토큰은 두 형태 소를 "故障中"(고장중)으로 결합하여 하나의 색인어로 추출한다.

[표 6.29 일본어 형태소 분석 - 접사 결합 옵션 (F)]

옵션	기본값	옵션 설명
F-	O	명사와 접사를 결합하지 않음 예) [故障中] >> [故障]

F+		명사와 접사를 하나의 형태소로 결합하여 색인어로 추출 예) [故障中] >> [故障, 故障中]

6.3.3.1.4. 영어 스테밍 옵션(S)

토큰이 영어일 경우 스테밍 적용 여부를 결정한다. 스테밍(stemming)이란, 영어 형태소 분석의 한 방법으로 입력된 단어의 어간(stem)만 추출하는 알고리즘이다. 예를 들어 likes은 like로 liked도 like와 같이 어간을 색인어로 추출하는 옵션이다.

[표 6.30 일본어 형태소 분석 – 영어 스테밍 옵션 (S)]

옵션	기본값	옵션 설명
S-		영어로 구성된 토큰에 스테밍을 적용하지 않음 예) [likes] >> [likes]
S+	O	영어로 구성된 토큰에 스테밍을 적용하여 색인어로 추출 예) [likes] >> [like, likes]

6.3.3.1.5. 용언 색인 옵션(V)

일본어에서 형용사 및 동사가 과거형으로 활용된 토큰에서 색인어를 추출할 때, 원형도 색인어로 추 출할지 여부를 결정하는 옵션이다.

[표 6.31 일본어 형태소 분석 – 용언 색인 옵션 (V)]

옵션	기본값	옵션 설명
V-	O	동사의 원형을 색인어로 추출하지 않음 예)[言っ] >> [言っ]
V+		입력된 동사와 동사의 원형을 색인어로 추출 예) [言っ] >> [言っ, 言う]

6.3.3.2. 확장 옵션(Extension option)

6.3.3.2.1. 히라가나 변환 옵션(H)

일본어는 히라가나와 가타카나 두 가지 형태의 문자를 사용한다. 히라가나 변환 옵션은

히라가나로 구성된 단어를 가타카나로 변환하여 색인어로 추출하는 옵션이다.

[표 6.32 일본어 형태소 분석 – 히라가나 변환 옵션 (H)]

옵션	기본값	옵션 설명
H-	O	히라가나를 가타카나로 변환하지 않음 예)[のりば] >> [のり]
H+		히라가나를 가타카나로 변환하여 색인어로 추출 예) [のりば] >> [のり, ノリ, バ]

6.3.3.2.2. 가타카나 변환 옵션(K)

일본어는 히라가나와 가타카나 두 가지 형태의 문자를 사용한다. 가타카나 변환 옵션은

가타카나로 구성된 단어를 히라가나로 변환하여 색인어로 추출하는 옵션이다.

[표 6.33 일본어 형태소 분석 – 가타카나 변환 옵션 (K)]

옵션	기본값	옵션 설명
K-	O	가타카나를 히라가나로 변환하지 않음 예)[マンション] >> [マンション]
K+		가타카나를 히라가나로 변환하여 색인어로 추출 예) [マンション] >> [マンション, まんしょん]

6.3.3.2.3. 일본어 흔들림 표기 변환 옵션(L)

일본어에서 종종 하나의 영어 단어에 대해서 두가지 형태의 가타카나로 표현되는 단어가 존재한다. 예를 들어 바이올린(violin)은 バイオリン과 ヴァイオリン으로 표기할 수 있다. 원칙상으로는 バイオリ ン을 쓰는 것이 올바르지만 ヴァイオリン도 용인한다. 여기서 문제가 되는 [ヴ,ヸ,ヴ,ヴ,ヲ]을 [バ,ビ,ブ, ベ,ボ]로 변환하는 기능을 제공한다.

[표 6.34 일본어 형태소 분석 – 일본어 흔들림 표기 변환 옵션 (L)]

옵션	기본값	옵션 설명
L-	O	흔들림 표기를 변환하지 않음 예)[ヴァイオリン] >> [ヴァイオリン]
L+		흔들림 표기를 변환하여 색인어로 추출 예) [ヴァイオリン] >> [ヴァイオリン, バイオリン]

6.3.3.2.4. 작은 가나 변환 옵션(S)

“작은 가나”는 일본어에 없는 음가가 있는 외래어 등을 표기하기 위해 사용된다. 대표적인 예로 ヴ ァ행, ツァ행, ファ행을 들 수 있다. 해당 옵션은 앞에 예를 든 행에서 축소된 작은 가나인 [ァ,ィ,ゥ, ェ,ォ]을 원래 형태로 복원하는 옵션이다.

[표 6.35 일본어 형태소 분석 – 작은 가나 변환 옵션 (S)]

옵션	기본값	옵션 설명
S-	O	작은 가나를 변환하지 않음 예)[ファ] >> [ファ]
S+		작은 가나를 변환하여 색인어로 추출 예) [ファ] >> [ファ, フア]

Note.

기본 옵션에서의 영어 스테밍과는 다른 옵션이지만 옵션에 사용하는 알파벳은 동일하기 때문에 주의가 필요합니다.

6.3.4. 중국어 형태소 분석 (CMA)

6.3.4.1. 기본 옵션(Basic option)

6.3.4.1.1. 단위 명사 결합 옵션 (B)

숫자와 단위성 의존명사인 "元", "人"과 같이 단위 명사로 알려진 명사와 함께 출현할 경우, 이 둘을 결합하여 색인어로 추출한다.

[표 6.36 중국어 형태소 분석 - 단위성 의존명사 결합 옵션 (B)]

옵션	기본값	옵션 설명
B+	O	숫자와 단위성 의존명사를 결합하여 색인어로 추출 예) [500元] >> [500元, 500, 元]
B-		숫자와 단위성 의존명사를 결합하지 않음 예) [500元] >> [500, 元]

6.3.4.1.2. 병음 변환 옵션 (P)

중국어에서 "人工智能"은 "rengongzhineng", "韩国"은 "hanguo"와 같이 발음을 영어로 작성한 병음을 한자와 함께 색인어로 추출한다.

[표 6.37 중국어 형태소 분석 - 병음 변환 옵션 (P)]

옵션	기본값	옵션 설명
P+		한자와 병음을 색인어로 추출 예) [人工智能] >> [人工智能, rengongzhineng]
P-	O	병음을 색인어로 추출하지 않음

		예) [人工智能] >> [人工智能] 주의 : 병음이 존재하지 않는 한자의 경우 색인어로 추출되지 않을 수 있음

6.3.4.1.3. 번체 변환 옵션 (T)

중국어는 주로 번체, 간체 두 가지 유형의 한자를 사용하며 번체는 이전부터 써 오던 전통적인 형태의 한자를 의미한다. 번체 변환 옵션은 간체로 작성된 단어를 번체로 변환하여 색인어로 추출하는 옵션이다.

[표 6.38 중국어 형태소 분석 – 번체 변환 옵션 (T)]

옵션	기본값	옵션 설명
T+		간체로 작성된 단어를 색인 할 때 해당 단어를 번체로 변환한 단어도 색인어로 추출 예) [电视] >> [电视, 電視]
T-	O	간체를 번체로 변환한 단어를 색인어로 추출하지 않음 예) [电视] >> [电视] 주의 : 변환할 번체가 존재하지 않는 한자의 경우 색인어로 추출되지 않을 수 있음

6.3.4.1.4. 간체 변환 옵션 (S)

중국어는 주로 번체, 간체 두 가지 유형의 한자를 사용하며 간체는 1960년대에 만들어진 기존 한자보다 간략화된 한자를 의미한다. 간체 변환 옵션은 번체로 작성된 단어를 간체로 변환하여 색인어로 추출하는 옵션이다.

[표 6.39 중국어 형태소 분석 – 간체 변환 옵션 (S)]

옵션	기본값	옵션 설명

S+		번체로 작성된 단어를 색인 할 때 해당 단어를 간체로 변환한 단어도 색인어로 추출 예) [學生] >> [学生, 學生]
S-	O	번체를 간체로 변환한 단어를 색인어로 추출하지 않음 예) [學生] >> [學生] 주의 : 변환할 간체가 존재하지 않는 한자의 경우 색인어로 추출되지 않을 수 있음

6.3.4.2. 확장 옵션(Extension option)

6.3.4.2.1. 영어 스테밍 옵션(E)

영어는 단수와 복수의 형태가 다른 명사가 있고, 시제에 따라 차이가 존재하는 동사가 있다. 영어 스테밍 옵션은 복수 형태의 명사와 기본 단어와 다른 형태의 동사를 모두 원형으로 변환하여 색인어로 추출하는 옵션이다.

[표 6.40 중국어 형태소 분석 - 영어 스테밍 옵션 (E)]

옵션	기본값	옵션 설명
E+		단수형 명사와 동사의 원형을 기존 단어와 함께 색인어로 추출 예) [bananas, going] >> [banana, bananas, go, going]
E-	O	작성된 영어를 그대로 색인어로 추출 예) [bananas, going] >> [bananas, going]

6.3.4.2.2. 복합 명사 분해 옵션(R)

한자로 구성된 중국어의 경우 띄어쓰기를 사용하지 않아 3글자 이상의 복합 명사가 색인어로 추출되는 경우가 있다. 복합 명사 분해 옵션은 3글자 이상의 명사를 구성하는 단어 중 의미 있는 단어가 존재한다면 이를 추출하는 옵션이다.

[표 6.41 중국어 형태소 분석 - 복합 명사 분해 옵션 (R)]

옵션	기본값	옵션 설명
R+		복합명사를 구성하는 단어 중 의미있는 단어가 존재할 경우 입력된 단어와 함께 색인어로 추출 예) [搜索引擎] >> [搜索引擎, 搜索, 引擎] 주의 : 기존에 설정한 기본 옵션 중 병음 추출, 간체 변환, 번체 변환의 기능이 R옵션으로 추출된 색인어(예시에서의 搜索, 引擎)에도 적용되며 설정에 따라 색인어가 많이 추출되어 속도 저하가 될 수 있음
R-	O	입력으로 들어온 단어의 기본 분석을 색인어로 추출 예) [搜索引擎] >> [搜索引擎]

6.3.5. 동의어 분석

동의어란 같은 뜻이나 비슷한 뜻을 가진 다른 낱말을 이르는 말이다. 본 제품에서는 동의어 사전을 통해서 색인과 검색어를 추출하는 과정에서 동의어 확장을 할 수 있다.

동의어 확장은 단독으로 사용하는 기능이 아니며, 본 제품의 모든 형태소 분석 언어에 포함되는 기능이다. 형태소 분석으로 나온 단어들은 기본적으로 동의어 확장이 된다. 그렇기 때문에 동의어 사전에 등록하는 단어들은 형태소 분석으로 나올 수 있는 단어여야 한다. 이를 위해서 “사용자 사전”에 동의어/표제어를 등록하는 것을 권장한다. 동의어/표제어 등록에 대한 자세한 내용은 “4.3 동의어 사전” 내용을 참고한다.

6.4. 애널라이저 관리

애널라이저는 앞서 설명한 사전, 형태소 분석 옵션, 레귤레이터 등을 조합하여 색인과 검색 시 색인어를 추출하는 분석기이다. 이후 애널라이저는 컬렉션 설정 단계에서 사용되며, 색인 필드 중

(Text)형 필드를 설정하기 위해서는 한 개 이상의 애널라이저 등록이 필요하다. 본 문단에서는 애널라이저 관리 내용과 방법에 대해 설명한다.

6.4.1. 애널라이저 관리 내용

[표 6.42 애널라이저 내부 옵션]

옵션	설명
analyzer	lae, regulator에서 정의되는 각각의 ID를 기반으로 정의한다.
regulator	구분자와 병합자를 설정하는 레귤레이터를 정의한다.
lae	형태소 분석 옵션과 사용자 사전 옵션을 조합하여 형태소 분석기를 정의한다. type으로는 (token, ngram, like, matrix, stemmer, kma, ema, jma, cma)를 사용할 수 있다.

[표 6.43 애널라이저 관리 API]

비고	HTTP Method	URL
초기화	POST	/analysis
업데이트	POST	/analysis/update
조회	GET	/analysis
삭제	POST	/analysis/delete

위의 표의 조회 API는 Engine 노드, Indexer 노드에서 수행 가능하며, 그 외의 애널라이저 관리 API의 경우 Engine 노드를 통해 수행 할 수 있다.

[애널라이저 등록의 예]

```
POST : http://127.0.0.1:9000/analysis
{
  "analyzer": {
    "kma": {
      "lae": "kma-black",
```

```
        "regulator": "R2",
        "dictionary": [
          "stopwordDictionaryId", "synonymDictionaryId"
        ]
      },
      "standard": {
        "lae": "token"
      },
      "bigram-without-regulator": {
        "lae": "bigram"
      },
      "trigram-without-regulator": {
        "lae": "trigram"
      }
    },
    "regulator": {
      "R1": {
        "allow": {
          "value": [
            "_"
          ]
        }
      },
      "R2": {
        "divide": {
          "value": [
            ",",
            "."
          ]
        }
      },
      "R3": {
        "allow": {
          "value": [
            "_",
```

```
            "."
          ]
        },
        "unite": {
          "value": [
            "/"
          ]
        }
      }
    },
    "lae": {
      "kma-black": {
        "type": "kma",
        "kmaOption": "A+R2",
        "licensePath": "korean/license.xml",
        "knowledgePath": "korean/wkb_nocomp.klg",
        "dictionary": [
          "usernounDictionaryId"
        ]
      },
      "bigram": {
        "type": "ngram",
        "length": 2
      },
      "trigram": {
        "type": "ngram",
        "length": 3
      },
      "token": {
        "type": "token"
      },
      "like": {
        "type": "like",
        "minlength": 2,
        "maxlength": 6,
```

```
        "maxno": 64
      },
      "matrix": {
        "type": "matrix",
        "begin": true,
        "end": true
      }
    }
  }
```

초기화(post)를 할 경우, 입력된 값을 기준으로 초기화를 하며 이전의 값은 무시한다. 업데이트의 경우 부분 기존 값의 데이터 추가가 가능하다. 특정 필드의 삭제가 필요한 경우, 업데이트 API요청을 보낼때 특정 필드를 빈값 (“”)으로 주게 될 경우, 해당 필드의 삭제가 가능하다. 표 5.24의 항목이 모두 포함되어야 컬렉션 설정 같은 향후 동작 시 이상없이 작동한다.

[애널라이저 업데이트의 예]

```
POST : http://127.0.0.1:9000/analysis/update
 {
   "analyzer": {
     "kma": {
       "lae": "kma-black",
       "regulator": "R2",
       "dictionary": [
         "stopwordDictionaryId", “synonymDictionaryId”
       ]
     },
     "standard": {
       "lae": "token"
     },
     "bigram-without-regulator": {
       "lae": "bigram"
     },
```

```
    "trigram-without-regulator": {
      "lae": "trigram"
    }
  },
  "regulator": {
    "R1": {
      "allow": {
        "value": [
          "_"
        ]
      }
    },
    "R2": {
      "divide": {
        "value": [
          ",",
          "."
        ]
      }
    },
    "R3": {
      "allow": {
        "value": [
          "_",
          "."
        ]
      },
      "unite": {
        "value": [
          "/"
        ]
      }
    }
  },
  "lae": {
```

```
      "kma-black": { //업데이트 한 부분
        "type": "kma",
        "kmaOption": "A+",
        "dictionary": [
          "usernoun1"
        ]
      },
      "bigram": {
        "type": "ngram",
        "length": 2
      },
      "trigram": {
        "type": "ngram",
        "length": 3
      },
      "token": {
        "type": "token"
      },
      "like": {
        "type": "like",
        "minlength": 2,
        "maxlength": 6,
        "maxno": 64
      },
      "matrix": {
        "type": "matrix",
        "begin": true,
        "end": true
      }
    }
  }
```

[애널라이저에서 특정 옵션 삭제 예]

특정 옵션에 대해 삭제를 수행할 경우, 각 옵션 중 한 가지 또는 여러 개의 옵션을 동시에 삭제 할 수 있다. 이때 해당 옵션이 사용중일 경우 삭제가 진행되지 않는다.

```
  POST : http://127.0.0.1:9000/analysis/delete
{
    "lae": ["kma"],
    "regulator": ["R3"],
    "analyzer": []
}
```

6.4.2. 애널라이저 관리 방법 및 예시

애널라이저 등록에서 만든 애널라이저는 공유 데이터(Shared Data) 영역에서 관리된다. 컬렉션 설정 시에는 공유 데이터를 참조하여 등록된 {애널라이저 ID}를 통해 색인 필드를 설정한다.

애널라이저가 업데이트될 경우, 애널라이저를 적용하고자 하는 컬렉션에 대해 Collection Refresh API를 호출해야 변경 내용이 적용된다.

[컬렉션 설정에서 애널라이저 사용의 예]

```
  POST : http://127.0.0.1:9001/collections/{collection-id}
  {
    "numOfShards": 2,
    "numOfReplication": 0,
    "fields": [ … ],
    "indexFields": {
      "DATE": {
        "type": "Double"
      },
      "SDATE": {
        "type": "date",
        "format": "yyyyMMddHHmmss"
      },
      "TITLE": {
        "type": "text",
       "analyzer": [
```

```
            "standard",
            "kma"
          ]
        },
        "CONTENT": {
          "type": "text",
          "analyzer": [
            "standard",
            "kma"
          ]
        }
      }
    }
```

6.4.3. 일본어, 중국어 형태소 분석기 애널라이저 등록 방법 및 예시

일본어와 중국어 형태소 분석기는 한국어 형태소 분석기와는 다른 형식으로 애널라이저를 선언한다. 한국어 형태소 분석기는 옵션(option)을 선언하는 필드가 하나이지만, 일본어와 중국어 형태소 분석기는 옵션(option)과 확장 옵션(extension)이 존재한다.

[일본어 형태소 분석기에서 애널라이저 사전 적용의 예]

```
{
  "analyzer": {
    "jma": {
      "lae": "jma"
    }
  },
  "regulator": {
    "R1": {
      "allow": {
        "value": [
          "?"
        ]
      }
    }
```

```
    },
    "lae": {
      "jma": {
        "type": "jma",
        "option": "",
        "extension": ""
      }
    }
  }
```

7. 컬렉션

7.1. 컬렉션 정의

7.1.1. Collection(컬렉션)

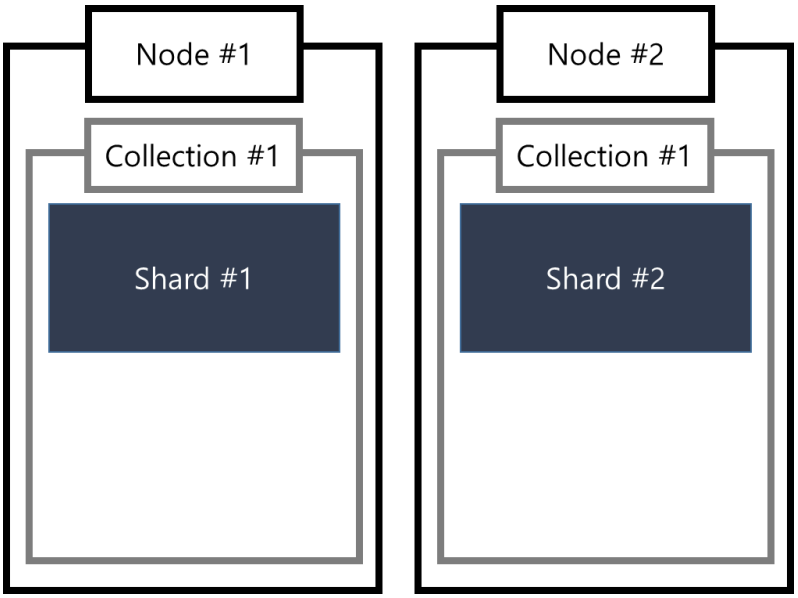
컬렉션이란 검색 대상이 되는 하나의 데이터 세트를 의미한다. 한 컬렉션에 들어가 있는 모든 데이터는 동일한 필드 이름과 속성을 가진다.

7.1.2. Shard(샤드)

Shard는 컬렉션을 수평 분할한 파티션 단위를 의미한다. 컬렉션을 두꺼운 책 한권으로 비유하면, Shard는 두꺼운 책을 챕터별로 나눈 단위라고 생각할 수 있다.

각 Shard는 엔진 인스턴스에서 부하 분산을 위해 전체 컬렉션을 작은 조각으로 관리하고 생성하기 위해 생성된다. Shard는 데이터에 대한 복사본이 아닌, 실제 색인 데이터를 갖고 있는 부분이다. 각 Shard는 이 데이터 부분을 위해 "하나"의 소스로서 동작한다.

[그림 7.1 Shard 개념도]



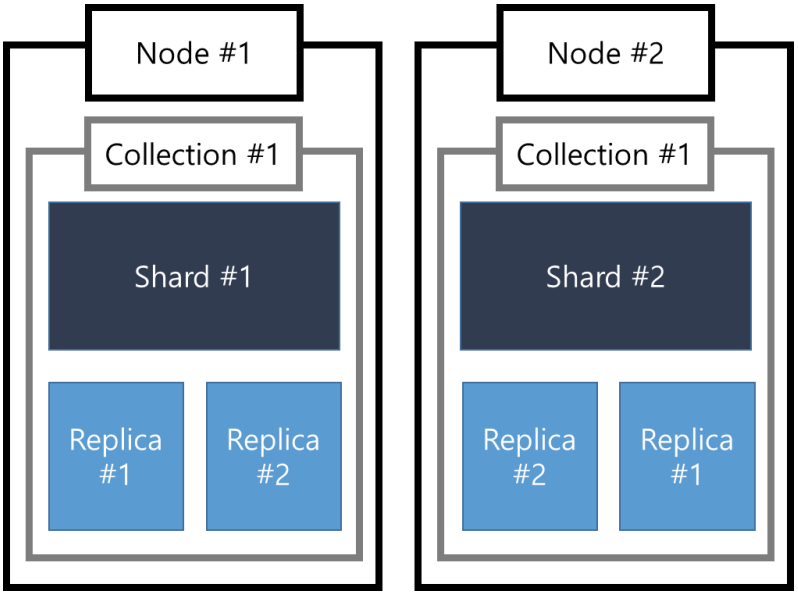
7.1.3. Replica(레플리카)

Replica는 Shard의 복사본으로 검색 요청이 들어왔을 때 읽기 역할을 수행한다.

1. 같은 노드에 같은 샤드 넘버를 가진 레플리카는 같은 샤드 넘버의 샤드의 데이터를 폴더를 하나로 관리한다.
2. 샤드 하나 당 무조건 레플리카 한 개는 생성된다. (CQRS)

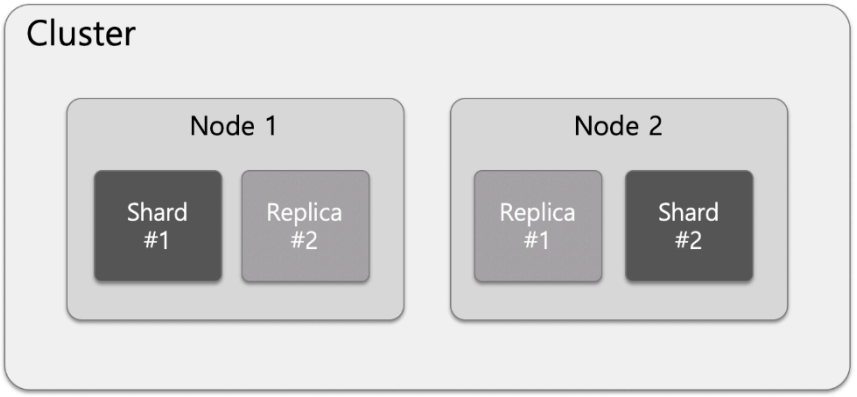
장애 발생 시 임시로 사용될 수 있는 데이터 구성의 하나를 지칭한다.

[그림 7.2 Replica 개념도]



아래 그림과 같이 클러스터를 구성할 경우, 일반적인 상황에서는 Node 1의 Shard #1과 Node 2의 Shard #2를 사용해 Collection #1의 검색을 수행한다. 만약 Node 1에 장애가 발생하더라도, Node 2에 Collection #1의 전체 데이터(Replica #1 + Shard #2)가 존재하기 때문에 검색 서비스를 문제 없이 가동할 수 있다.

[그림 7.3 클러스터 구성의 예]



7.2. 컬렉션 관리

컬렉션은 검색을 위한 기본 단위이며, 검색 대상에 따라서 문서의 구성이 달라질 수 있다. 컬렉션 설정에서는 문서의 구성 정의 및 색인 필드를 정의하며, 부가적으로 Shard와 Replica의 개수, 캐시, 실시간 검색어 설정을 추가할 수 있다.

[표 7.1 컬렉션 관리 API]

비고	HTTP Method	URL

초기화	POST	/collections/{collection_id}
삭제	POST	/collections/{collection_id}/delete
조회	GET	/collections/{collection_id}

7.2.1. 초기화

[표 7.2 컬렉션 초기화 API]

HTTP Method	URL	비고
POST	/collections/{collection_id}	초기화

아래 항목의 정의가 해당 API를 통해 이뤄진다.

[표 7.3 Collection의 Index Setting]

key	설명
numOfShards	Collection의 Shard 개수 (cluster에 있는 shard의 수) Default : 1 설정 권장 값 : {Node 개수} 설정 최적화 값 : 1. 사용할 컬렉션과 동일한 구성의 Test collection 을 생성한다. 2. 샘플 문서 1건을 색인한 후 색인된 폴더의 용량을 확인 3. (예상되는 전체 건수) X (색인된 폴더의 용량) / (40GB) 으로 나오는 값을 numOfShards 수 만큼 설정한다. - 예시: 1000만건 X (40KB) / (40GB) = 9개
numOfReplication	추가적으로 설정할 Collection의 Replica 개수 (node에 있는 Replica의 수) Default : 0 설정 가능한 최대값 : {Node 개수} -1
recentQuery	최근 검색어 리스트의 개수를 설정한다. 최근 검색어 조회 기능 사용 시, 해당 개수 만큼 반환된다. (실시간 검색어 조회 API 사용법 참고) Default : 5

maxQueryCacheCount, maxQueryCacheRamBytesUsed	쿼리 캐시 설정 (메모리 상에서 가장 최근에 사용된 적이 없는 캐시의 메모리부터 대체하며 새로운 데이터로 갱신) 1) maxQueryCacheCount(0보다 큰 개수) ● 쿼리 적재량 ● Default : 1000 1) maxQueryCacheRamBytesUsed (0보다 큰 수) ● 캐시로 사용되는 최대 메모리 사용량 (bytes) ● Default : 50*1024*1024
fields	문서 필드 정의
indexFields	색인 필드 정의 1) 필드별 색인 타입 정의 ● type: date, double, long, text, category, authority, user 2) 필드별 analyzer 정의 ● Analysis API에서 정의한 analyzer id ● 배열을 통해 여러 개의 analyzer를 지원한다. ● 동의어와 금칙어는 색인 시 반영되지 않고, 검색 시에만 반영되므로, analyzer를 생성할 때 동의어 사전과 금칙어 사전은 자동으로 제외된다. 3) date 타입의 경우 format이 지정되지 않으면 기본값으로 적용된다. - format 지정 시, 대소문자가 구분되므로 주의하여야한다. 예를 들어, 20210101 형식의 date type을 위한 format은 yymmdd 이 아니라 yyMMdd 이다. - Default format : yyyyMMddHHmmss 4) indexFields에 docid를 넣어도 지정된 타입이 아닌 docid field로 색인이 된다. 5) user 타입 - 사용자 피드백을 검색에 로깅하기 위한 필드로 Integer 범위의 숫자값을 색인한다. 6) model 옵션 : - model 옵션은 벡터 색인 및 검색을 사용하는 경우, text type 필드에만 추가적으로 적용이 가능하다. - indexFields.{fieldName}.model 옵션의 값으로 model id 를 지정해야 한다.

	- model 값은 ModelServer API 로 등록된 것이어야 하며, 선언되지 않은 model id를 사용 할 수 없다.
indexFields.{fieldname}.deduplicate	중복된 문서 검색을 하기위한 설정이다.
numOfIndexThreads	- 기본값은 core 개수/2 값으로 세팅되며, 병렬 색인을 위한 indexer 의 thread 개수를 의미 - 색인을 위해 해당 옵션 값만큼 인덱싱 쓰레드를 생성하여 사용하며, 만약 gateway 에서 요청된 값이 0 이하이면 기본값으로 설정한 thread 수 (core/2) 를 사용
searchRoutingStrategy	- 기본값 : fail-over - fail-over : 노드가 2개 이상인 경우에서 노드 1개가 꺼진 장애상황에서, 검색시 replica 로 검색을 할 수 있게 해주는 옵션
advancedSettings	totalHitsThreshold(default : 10000) : 1. totalHits을 기준으로 해당 값을 넘기면 엔진 cache에 저장한다. 2. 너무 작은 값을 넣으면 값에 입출력에 따른 I/O 관련 부하가 생겨 검색 속도 저하가 생길 수 있다.

[컬렉션 초기화 예제]

```
POST /collections/sample_bbs

{
  "numOfShards": 1,
  "numOfReplication": 0,
  "recentQuery": 10,
  "maxQueryCacheCount": 1000,
  "maxQueryCacheRamBytesUsed": 500000,
  "mappingTableId": "mappingTable1",
  "spellerId": "spellerA",
  "fields": [
    "DOCID",
    "Date",
    "TITLE",
    "WRITER",
    "CONTENT",
    "ATTACHNAME",
```

```
    "ATTACHEXT",
    "ATTACHCON",
    "LINK",
    "FILELINK",
    "ALIAS"
  ],
  "indexFields": {
    " Date": {
      "type": "date",
      "format": "yyMMdd"
    },
    "TITLE": {
      "type": "text",
      "analyzer": [
        "korean"
      ],
      "model": "dpr"
    },
    "WRITER": {
      "type": "text",
      "analyzer": [
        "standard"
      ]
    },
    "CONTENT": {
      "type": "text",
      "deduplicate": true,
      "analyzer": [
        "korean"
      ]
    },
    "LINK": {
      "type": "text",
      "analyzer": [
        "bigram",
        "trigram"
      ]
    }
  },
  "advancedSettings": {
    "totalHitsThreshold": 10000
```

```
    }
  }
```

7.2.1.1. 삭제

컬렉션을 엔진에서 삭제할 때 사용된다. 기존 POST 요청과는 다르게, 삭제 요청에는 추가 파라미터 없이 요청을 진행 할 수 있다.

[표 7.4 컬렉션 삭제 API]

HTTP Method	URL	비고
POST	/collections/{collection_id}/delete	삭제

7.2.1.2. 조회

엔진에 설정되어 있는 컬렉션의 정보를 조회할 때 사용 된다. 삭제 요청과 동일하게 추가 파라미터가 없다.

[표 7.5 컬렉션 조회 API]

HTTP Method	URL	비고
GET	/collections/{collection_id}	조회

7.2.1.3. 컬렉션 리스트 조회

엔진에서 서비스중인 컬렉션 리스트를 반환한다. 삭제, 조회와 동일하게 파라미터 없이 API를 통해 조회 가능하다.

[표 7.6 컬렉션 리스트 조회 API]

HTTP Method	URL	비고

GET	/collections	조회

7.2.2. 컬렉션 업데이트

기존의 컬렉션에서 일부 항목에 대해 수정하거나 신규 필드를 추가 할 수 있다.

[표 7.7 컬렉션 업데이트 API]

HTTP Method	URL	비고
POST	/collections/{collection_id}/update	업데이트

수정 가능한 항목은 다음 표를 참고한다. fields 및 indexFields의 경우 신규로 추가할 필드만 기입한다. 기존에 존재하는 필드에 대해서는 수정이 불가능하다.

[표 7.8 컬렉션 업데이트 가능 항목]

key	설명
recentQuery	최근 검색어 리스트의 개수를 설정한다. 최근 검색어 조회 기능 사용 시, 해당 개수 만큼 반환된다. (실시간 검색어 조회 API 사용법 참고)
maxQueryCacheCount, maxQueryCacheRamBytesUsed	쿼리 캐시 설정 (메모리 상에서 가장 최근에 사용된 적이 없는 캐시의 메모리부터 대체하며 새로운 데이터로 갱신) 1) maxQueryCacheCount ▪ 쿼리 적재량 (개수) 1) maxQueryCacheRamBytesUsed ▪ 캐시로 사용되는 최대 메모리 사용량 (bytes)
keywordWeightDictionary	키워드 가중치 사전의 id 값을 등록한다. 여러개의 사전 id 값을 등록하여 weight-id 가 겹친 경우 마지막에 등록한 사전의 값으로 머징된다.
spellerId	오타교정 사전을 설정하며 한 컬렉션에 한개만 등록할 수 있다. 컬렉션 등록시, speller 옵션에 등록해놓은 오타교정사전 id값을 입력해준다.

mappingTableId	그룹핑(Grouping) 대상 값에 대해서 별도의 그룹 이름을 붙이기 위하여 사용되는 사전
fields	추가할 문서 필드 정의
indexFields	A) 추가할 색인 필드 정의 1) 필드별 색인 타입 정의 ▪ type: date, double, long, text, category, authority, user 2) 필드별 analyzer 정의 ▪ Analysis API에서 정의한 analyzer id ▪ 배열을 통해 여러 개의 analyzer를 지원한다. ▪ 동의어와 금칙어는 색인 시 반영되지 않고, 검색 시에만 반영되므로, analyzer를 생성할 때 동의어 사전과 금칙어 사전은 자동으로 제외된다. 3) date 타입의 경우 format이 지정되지 않으면 기본값으로 적용된다. - format 지정 시, 대소문자가 구분되므로 주의하여야한다. 예를 들어, 20210101 형식의 date type을 위한 format은 yymmdd 이 아니라 yyMMdd 이다. - Default format : yyMMddHHmmss 4) model 옵션 : 벡터 색인 및 검색을 사용하는 경우 - model 옵션은 벡터 색인 및 검색을 사용하는 경우, text type 필드에만 추가적으로 적용이 가능하다. - indexFields.{fieldName}.model 옵션의 값으로 model id 를 지정해야 한다. - model 값은 ModelServer API 로 등록된 것이어야 하며, 선언되지 않은 model id를 사용 할 수 없다. B) 수정할 색인 필드 정의 1) 기존 색인 필드 정의하듯이 사용하면 되지만 이 기능을 사용할 경우 재색인을 해야 정상적으로 동작한다. 2) 카테고리 관련 색인 필드 업데이트는 지원하지 않는다.
advancedSettings	totalHitsThreshold(default : 10000) : 1. totalHits을 기준으로 해당 값을 넘기면 저장한다. 2. 너무 작은 값을 넣으면 값에 입출력에 따른 I/O 관련 부하가 생겨 검색 속도 저하가 생길 수 있다.

[컬렉션 업데이트 예제]

```
POST /collections/sample_bbs/update

{
  "recentQuery": 10,
  "maxQueryCacheCount": 1000,
  "maxQueryCacheRamBytesUsed": 500000,
  "spellerId": {spellerDictionaryId},
  "mappingTableId": {mappingTableId},
  "fields": [
    "NEWFIELD"
  ],
  "indexFields": {
    "NEWFIELD": {
      "type": "text",
      "analyzer": [
        "korean"
      ],
      "model": "dpr"
    }
}
```

7.3. 컬렉션 상태

7.3.1. 컬렉션 Shard 상태

지정된 컬렉션의 Shard 상태 정보를 리턴한다. 각각의 상태에 대한 정보는 아래 표를 참고한다.

[표 7.9 컬렉션 Shard 상태 API]

HTTP Method	URL	비고
GET	/collections/{collection_id}/shard	조회

[표 7.10 컬렉션 Shard 상태 설명]

상태	설명	서비스	복구
GOOD	PRIMARY와 REPLICA 모두 존재	O	가능
WARNING	일부 REPLICA 존재하지 않음	O	가능
BAD	일부 PRIMARY 존재하지 않으며, 누락된 REPLICA 존재	X	가능
CANNOT_RECOVER	일부 PRIMARY 존재하지 않으며, 누락된 REPLICA 존재하지 않음	X	불가능
INDEXER_REMOVAL	사용 중이던 인덱서가 제거된 상태	O	배포가 정상적으로 처리 되었으면 가능

[결과 예시 - 정상]


```
{
 "collectionId": "test",
 "status": "GOOD",
 "description": "해당 Collection은 서비스가 가능합니다.",
 "numOfShards": 2,
 "numOfReplica": 1,
 "activePrimaryShards": [
  {
   "uuid": "213fd918-fc78-4230-ad13-111865415a9c",
   "id": 0,
   "type": "PRIMARY",
   "address": "akka://irag@127.0.0.1:2552"
  },
  {
   "uuid": "69db4542-0b68-470b-b400-08e862f2a121",
   "id": 1,
   "type": "PRIMARY",
   "address": "akka://irag@127.0.0.1:2551"
  }
 ],
 "activeReplica": [
  {
```

```
    "uuid": "68a8b2e1-c08f-418e-ad69-c7c07354f3c2",
    "id": 0,
    "type": "REPLICA",
    "address": "akka://irag@127.0.0.1:2551"
   },
   {
    "uuid": "b010d21e-4be3-435d-8499-6735289cfd18",
    "id": 1,
    "type": "REPLICA",
    "address": "akka://irag@127.0.0.1:2552"
   }
  ]
}
```


[결과 예시 - 비정상]


```
{
 "collectionId": "test",
 "status": "BAD",
 "description": "The number of active primary shards is not enough. Must be rebalanced.",
 "numOfShards": 2,
 "numOfReplica": 1,
 "activePrimaryShards": [
  {
   "uuid": "69db4542-0b68-470b-b400-08e862f2a121",
   "id": 1,
   "type": "PRIMARY",
   "address": "akka://irag@127.0.0.1:2551"
  }
 ],
 "activeReplica": [
  {
   "uuid": "68a8b2e1-c08f-418e-ad69-c7c07354f3c2",
   "id": 0,
   "type": "REPLICA",
   "address": "akka://irag@127.0.0.1:2551"
  }
 ]
}
```

7.4. 컬렉션 새로고침

7.4.1. 컬렉션 새로고침

Analysis, 사전 또는 권한 컬렉션 등의 업데이트가 발생 시, 이를 적용하기 위해 컬렉션의 데이터를 다시 로딩하는 기능이다. 또한 반복적으로 색인 시 색인 데이터의 용량이 지속적으로 증가할 경우에도 해당 API를 호출 시 사용하지 않는 데이터를 정리 할 수 있다.

해당 API는 Engine 노드에서 호출 시 모든 Engine 노드의 색인 데이터를 다시 로딩 한다.

[표 7.11 컬렉션 새로고침 API]

HTTP Method	URL	비고
POST	/collections/{collection_id}/refresh	새로고침

8. 색인 및 배포

8.1. 색인 운영 방식

본 제품은 JSON 색인 방식과 Gateway를 통한 SCD 파일 색인 방식을 지원한다. 색인 완료 이후에는 색인어보기(Index Morpheme)를 이용하여 색인된 문서를 확인할 수 있고, 색인 업데이트(Index Update), 색인 삭제(Index Delete)를 통해 색인 문서를 관리할 수 있다.

색인 작업을 수행하기 위해서는 애널라이저, 사전, 컬렉션 정보가 등록되어 있어야 하며, 컬렉션 설정에서 useIndexer 옵션을 활성화했을 경우에는 색인 노드를 통해 색인 및 배포 이후 Engine 노드에 색인이 반영된다.

예제 실행을 위한 애널라이저와 컬렉션 정보는 하단의 JSON을 참고하여 등록한다. 사전 등록 예제는 생략한다.

[애널라이저 설정의 예]

```
POST : http://127.0.0.1:9001/analysis
 {
  "analyzer": {
    "kma": {
      "lae": "kma-black-filter",
      "regulator": "R1"
    },
    "token": {
      "lae": "token",
      "regulator": "R1"
    },
    "bigram-without-regulator": {
      "lae": "bigram"
    }
  },
```

```
  "regulator": {
    "R1": {
      "allow": {
        "value": [
          "_",
          "."
        ]
      }
    }
  },
  "lae": {
    "kma-black-filter": {
      "type": "kma",
      "kmaOption": "A+R2",
      "licensePath": "korean/license.xml",
      "knowledgePath": "korean/wkb_nocomp.klg",
      "dictionary": [
        "usernoun"
      ]
    },
    "bigram": {
      "type": "ngram",
      "length": 2
    },
    "token": {
      "type": "token"
    }
  }
}
```

[sample 컬렉션 설정의 예]

```
POST : http://127.0.0.1:9001/collections/sample

{
  "numOfShards": 1,
```

```
  "numOfReplication": 0,
  "recentQuery": 5,
  "fields": [
    "DOCID",
    "DATE",
    "TITLE",
    "CONTENT",
    "DEPARTMENT"
  ],
  "indexFields": {
    "DATE": {
      "type": "long"
    },
    "TITLE": {
      "type": "text",
      "analyzer": [
        "kma"
      ]
    },
    "CONTENT": {
      "type": "text",
      "analyzer": [
        "kma"
      ]
    },
    "DEPARTMENT": {
      "type": "long"
    }
  }
}
```

8.1.1. 색인 방식 – Gateway

Gateway는 검색 서버 또는 원격지의 SCD 또는 JSON 형식의 파일을 설정한 문서 개수 단위로 검색 엔진의 멀티플 색인 API를 호출하는 유틸리티 모듈이다. 본 모듈의 실행을 위해서는 JDK(Java development kit) 1.8 이상의 버전을 필요로 한다.

8.1.1.1. Config 설정

다음은 설정 파일의 예시 및 각 설정에 대한 내용이다. engine 및 collection 설정은 필수적으로 기재해야 하며, option 설정은 선택적으로 사용 가능하다.

[Gateway 설정파일 예제]

```
engine:
  addressList: ["x.x.x.x:xxxx", "x.x.x.x:xxxx"]
  mode: "SCD"
  timeout: 1000000

collection:
  collectionList: [
    "sample_bbs"
  ]
  scdDirectory: "/home/wisenut/irag-package/collection/"
  docUnitLength: 10000
  maxBodiySize: 13421778
distribute
dist:
  distribute: true

tea:
  useTea: false
  teaIp: "x.x.x.x"
  teaPort: 11000

options:
  static: false
  lastCommit: false
  secret: "secret"
  encoding: "UTF-8"
  fileThreadNum: 4
  postThreadNum: 2
```

[표 8.1 Gateway 설정]

속성 (Attribute)	설명

engine.addressList	색인 요청을 받을 클러스터 내 노드 주소. 리스트의 첫번째 노드 주소부터 색인 가능 여부를 확인하여, 색인 가능 시 해당 노드를 통해 색인을 진행한다. (실행 시 최초 1회 설정하며, 색인 진행 중간에 연결이 끊길 경우에는 자동으로 다른 노드로 연결하지 않음.)
engine.mode	색인할 파일의 포맷 또는 색인 옵션 - json : json 단일 문서 또는 Json array 구조의 문서를 색인한다 - scd : SCD 구조의 문서를 색인한다. - convert-json : SCD 파일을 JSON array 파일로 변환하는 기능이다. - convert-vector : 컬렉션에 설정된 vector 타입 필드가 존재할 때 JSON 파일에서 동일한 이름의 필드 값을 벡터 값으로 변환하고 저장하는 기능이다. (사용 예시는 [13.5.4. 색인 및 검색] 을 참조)
engine.timeout	단위 색인 요청 시 timeout 설정 (ms)
collection.collectionList	색인을 수행할 컬렉션 리스트
collection.scdDirectory	색인할 SCD 파일이 들어있는 모든 컬렉션 폴더의 상위 디렉토리. scdDirectory 하위에 collectionList의 컬렉션 명과 동일한 디렉토리 내 SCD파일을 색인한다.
collection.docUnitLength	한번의 단위 색인 시 마다 색인할 문서의 개수
collection.maxBodySize	한번 색인 할 때 최대로 색인 가능한 body size. (기본값 : 128MB) 문서 1개의 크기가 maxBodySize가 넘을 경우 해당 문서는 색인을 수행하지 않는다.
dist.distribute	색인 완료 후 배포 작업을 수행할 지 여부 (default : false)
options.static	기존 엔진에 색인된 데이터를 지우고 새로 색인할 지 선택하는 옵션
options.lastCommit	Commit 동작을 마지막에 한 번만 수행할 지 여부
options.secret	엔진 API 권한 제한 사용시, 사용되는 키값
options.encoding	색인할 문서 파일의 인코딩을 지정하는 옵션 (default : UTF-8)
options.fileThreadNum	동시에 색인 작업을 수행할 문서 파일 쓰레드 수 (default : 4)
options.postThreadNum	동시에 호출할 멀티플 색인 api 쓰레드 수 (default : 1)
options.docUnitQueueSize	색인 대기중인 문서 단위의 개수를 지정한다. (default : 100) (색인 대기중인 문서 개수 = docUnitLength * docUnitQueueSize)

options.engineIndexThreads	색인 작업 시 사용할 스레드 수. 많을수록 CPU 사용량이 증가하며, 색인 속도가 증가한다. (default : 컬렉션 설정에 따름)

예제와 같은 설정 파일 구성의 경우, 다음과 같은 경로에 각 컬렉션 별 SCD 파일을 준비한다. 각 SCD 또는 JSON 파일의 이름은 3.4장 SCD 파일 이름 규칙을 따른다.

[Gateway 컬렉션 별 색인 경로 생성 예제 – SCD 파일 색인]

```
/home/wisenut/irag-package/collection/sample_bbs/scd/index
/home/wisenut/irag-package/collection/sample_edu/scd/index
```

[Gateway 컬렉션 별 색인 경로 생성 예제 – JSON 파일 색인]

```
/home/wisenut/irag-package/collection/sample_bbs/json/index
/home/wisenut/irag-package/collection/sample_edu/json/index[L60]
```

주의
– Gateway 실행 전 엔진에 해당 컬렉션을 미리 생성해야 하며, 생성한 컬렉션의 필드 구성과 SCD 파일의 필드 구성이 동일해야 한다. – R(Replace) 타입 색인의 경우 현재 Json 포맷으로만 지원한다.

8.1.1.2. Gateway 실행

Gateway는 패키지에서 제공하는 배치파일을 통해 다음과 같이 실행한다.

[Gateway 배치파일 실행 예제 – Windows]

```
C:\irag-package\batch\gateway.cmd ..\config\gateway.yaml
```

[Gateway 배치파일 실행 예제 – Linux]

```
wisenut@wisenut:/home/wisenut/irag-package/batch/gateway.sh ../config/gateway.yaml
```

[표 8.2 Gateway 실행 인자]

인자	값	설명

-conf	<path>	gateway_config.yaml의 경로
-home	<path>	IRAG_HOME 의 경로

로그 파일은 ${home}\log\gateway 하위에 생성되며, 해당 내용은 콘솔에서도 확인 할 수 있다.

Gateway가 정상적으로 작동되면 collectionList의 컬렉션 순서대로 색인을 수행하며, 다음과 같은 INFO 로그를 확인 할 수 있다.

```
  [ 00001 ] [ 16:06:48.030 ] [ INFO  ] [ kr.co.wisenut.Main   ] [ main     ] [ 122 ] [
[SYS]<GWL_0001> irag-v2 Gateway 모듈이 실행되었습니다.                                  ]
  [ 00001 ] [ 16:06:48.094 ] [ INFO  ] [ kr.co.wisenut.Main   ] [ main     ] [ 132 ] [
[SYS]<GWL_0024> 127.0.0.1:9001 노드 연결에 성공                                        ]
  [ 00001 ] [ 16:06:26.267 ] [ INFO  ] [ t.gateway.SCDIndexer ] [ index    ] [ 224 ] [
[IDX]<GWL_0102> Collection ( scity2 ) 3개 파일 색인이 시작되었습니다.                    ]
  [ 00001 ] [ 16:06:26.267 ] [ INFO  ] [ t.gateway.SCDIndexer ] [ index    ] [ 224 ] [
[IDX]<GWL_0102> Collection ( scity2 ) 3개 파일 색인이 시작되었습니다.
```

다음과 같이 단위 색인이 완료되면 다음과 같은 완료 정보를 확인 할 수 있다.

```
  [ 00002 ] [ 15:49:00.668 ] [ INFO  ] [ nut.api.APIRequester ] [ printLog   ] [ 247 ] [
[IDX]<GWL_0110> Collection ( scity ) 성공 : 5,000, 실패 : 0, 소요 시간 : 24.953 (sec), 전체 성공 :
10,000       ]
```

하나의 SCD 내 모든 문서 색인이 완료되면 다음과 같은 로그를 출력하며, 정상 색인 완료 된 SCD 파일은 backup 디렉토리에 옮겨진다.

```
[ 00004 ] [ 16:06:46.025 ] [ INFO  ] [ t.gateway.SCDIndexer ] [ runIndex   ] [ 394 ] [
[IDX]<GWL_0104> Collection ( scity1 ) 파일 색인 진행 : 2 / 3                            ]
```

주의
Gateway 모듈을 실행한 계정으로 directory 경로 및 SCD 파일의 소유자, 권한이 설정 되어 있지 않을 시 색인 완료된 SCD 파일이 정상적으로 이동 되지 않을 수 있으므로, Gateway 실행 전 반드시 확인해야 한다.

8.1.1.3. TEA 연동 가이드

IRAG-v7 에서는 Gateway를 통해 색인 시 TEA 서버의 실시간 분석 기능을 사용하여 색인 대상 문서가 속한 주제(Topic) 와 주요 주제어(Keyword) 분석 결과를 색인 데이터로 저장한다.

본 장에서는 패키지를 통해 제공되는 sample_terms에 대한 TEA 연동 색인 예제를 설명한다. TEA 패키지는 본 제품에서 포함되어 있지 않기 때문에 별도의 패키지 다운로드를 필요로 한다. 먼저 다음과 같이 Collection 설정에서 TOPIC, TERMS를 Document Field로 필요로 한다.

[sample_terms 컬렉션 예제]

```
{
 "numOfShards": 1,
 "numOfReplication": 0,
 "recentQuery": 0,
 "fields": [
   "DOCID",
   "Date",
   "TITLE",
   "WRITER",
   "USERID",
   "EMAIL",
   "DEPT",
   "PHONE",
   "CONTENT",
   "ATTACHNAME",
   "ATTACHCON",
   "ALIAS",
   "TERMS",
   "TOPIC"
 ],
 "indexFields": {
   "TITLE": {
     "type": "text",
     "analyzer": [
       "kma"
     ]
   },
```

```
    "CONTENT": {
      "type": "text",
      "analyzer": [
        "kma"
      ]
    },
    "ATTACHCON": {
      "type": "text",
      "analyzer": [
        "kma"
      ]
    },
    "ATTACHNAME": {
      "type": "text",
      "analyzer": [
        "kma"
      ]
    },
    "WRITER": {
      "type": "text",
      "analyzer": [
        "token"
      ]
    },
    "ALIAS": {
      "type": "text",
      "analyzer": [
        "token"
      ]
    }
  }
}
```

연동을 위해 TEA listener, TEA server의 설정 파일을 각각 수정한다.

[TEA listener 설정 파일]

```
<TeaConfig version="2.0">
```

```
        <Listener port="11000" useHttpd="y" httpdPort="11200" systemThreads="10" indexerThreads="8">
    <Data path="d:\tea2\data" />
    <PID path="d:\tea2\pid\tea2_listener.pid" />
    <Log path="d:\tea2\log" prefix="listener">
  </Listener>
  <Servers>
    <Server ip="x.x.x.x" port="11100" />
  </Servers>
</TeaConfig>
```

[TEA 서버 설정 – 기본 설정]

```
 <Server port="11100" systemThreads="10" indexerThreads="8">
  <Knowledge>
   <BasePath dir="D:\tea2\knowledge"/>
    <Dictionary type="blackword" id="system_black" fileName="system_black.txt"/>
    <Dictionary type="whiteword" id="system_white" fileName="system_white.txt"/>
  </Knowledge>
  <PID path="D:\tea2\pid\tea2_server.pid"/>
  <Log path="D:\tea2\log\server" prefix="server"/>
 </Server>
```

[TEA 서버 설정 – 주제 분석 모델]

```
 <RoasterModels>
    <Model id="sample_terms">
      <RefRoasterColl id="sample_terms"/>
      <ModelType>LDA</ModelType>
            <Params topicSize="50" alpha="0.027" beta="0.09" sampling="30" realtime="y" modelDocumentSize="50000"/>
      <Delimiter name="@" topic="^" pair=":"/>
      <TopicFields>
            <TopicField name="TOPIC" tagName="TERMS" outputMode="1" topicCount="-1" compareCount="20"/>
      </TopicFields>
    </Model>
  </RoasterModels>
```

실시간 분석 연동을 위해 realtime="y"로 설정해야 하며, Delimiter는 기본 값으로 설정해야 한다.

[TEA 서버 설정 - 주제 분석 컬렉션]

```
<RoasterCollections>
    <!-- sample_terms Sample Roaster Collection -->
    <Collection id="sample_terms" charset="UTF-8">
      <RefExtractorColl id="sample_terms"/>
      <SourceDir dir="D:\IdeaProjects\24-tea-connection\sample\sample_terms\scd\teindex"/>
      <DeniedInfo field="y" tags="y"/>
      <Delimiter term="^" score=":"/>
      <FieldInfo>
        <Field name="DOCID" type="DOCID"/>
        <Field name="Date" type="DATE"/>
        <Field name="TITLE" type="TITLE"/>
        <Field name="CONTENT" type="TEXT"/>
        <Field name="WRITER" type="CUSTOM"/>
        <Field name="USERID" type="CUSTOM"/>
        <Field name="EMAIL" type="CUSTOM"/>
        <Field name="DEPT" type="CUSTOM"/>
        <Field name="PHONE" type="CUSTOM"/>
        <Field name="TERMS" type="KEYWORD"/>
      </FieldInfo>
...
    </Collection>
</RoasterCollections>
```

주제 분석 설정 시 FieldInfo의 필드 이름 순서와 주제어 분석 설정의 OutputInfo의 Field 노드의 필드 이름 순서가 일치해야 한다.

실시간 분석 수행 전 tea_util 모듈을 통해 다음과 같이 초기 분석을 수행한다.

```
D:\tea2\batch>tea2_listener.cmd start
D:\tea2\batch>tea2_server.cmd start
```

SCD 파일을 주제어 추출을 위한 SourceDir에 위치시키고 분석 유틸리티를 실행한다.

```
D:\tea2\batch>tea2_util.cmd 127.0.0.1 11000 sample_terms -te 2
```

분석 유틸리티를 통해 주제 분석(주제 모델링) 과정을 수행한다.

```
D:\tea2\batch>tea2_util.cmd 127.0.0.1 11000 sample_terms -topic y
```

gateway에 다음과 같이 TEA 설정을 활성화하고, mode를 SCD로 설정한다.

```
engine:
  ip: "x.x.x.x"
  port: "9001"
  mode: "SCD"
collection:
  collectionList: ["sample_terms"]
  scdDirectory: "D:\scd\sample_terms"
  docUnitLength: 1000
tea:
  useTea: true
  teaIp: "x.x.x.x"
  teaPort: 11000
```

gateway를 실행하여 tea 연동이 정상적으로 되었는지 확인한다.

```
D:\irag-package\batch\gateway.cmd ..\config\gateway_config.yaml
```

8.1.2. JSON 색인 방식

8.1.2.1. 색인 (Index)

색인 (Index)의 경우 여러 문서에 대한 색인을 지원한다. 주의해야할 점은 항상 배열 형태로 입력해야 한다는 점이다. 색인 시 문서를 하나 이상 입력하도록 한다.

[표 8.3 멀티플 색인 API]

HTTP Method	URL	비고
POST	/index/{collection_id}	요청 & 리턴 있음

[멀티플 색인 설정의 예]

POST : http://127.0.0.1:9001/index/sample


```
[
  {
    "DOCID": "0",
    "DATE": "20200702030101",
    "TITLE": "검색 엔진",
    "CONTENT": "본 매뉴얼은 와이즈넛 검색 엔진에 대한 개발 정보들을 제공하기 위해 작성 되었다.",
    "DEPARTMENT": 100
  },
  {
    "DOCID": "1",
    "DATE": "20200703113301",
    "TITLE": "검색",
      "CONTENT": "와이즈넛이 제공하는 다른 문서들이 본 매뉴얼의 내용보다 더 최신의 내용을 담고 있을 수 있습니다.",
    "DEPARTMENT": 100
  },
  {
    "DOCID": "2",
    "DATE": "20200702113433",
    "TITLE": "엔진",
    "CONTENT": "본 매뉴얼의 수정 없이 솔루션의 기능 및 성능을 향상시킬 수 있으므로 문의사항이 있으면 아래 연락처로 연락 주시기 바랍니다.",
    "DEPARTMENT": 200
  },
  {
    "DOCID": "3",
    "DATE": "20200702030101",
    "TITLE": "웹 검색 엔진",
```

```
      "CONTENT": "아울러 솔루션의 실질적인 구성은 사례에 따라 다르며, 본 매뉴얼에 기술된 내용은 기본 사양에 대한 것임을 밝힙니다.",
      "DEPARTMENT": 100
    }
]
```

[결과값 예시]

```
{
  "result": true,
  "elapsedTime": 112.51769999999999,
  "total": 4,
  "success": 4
}
```

8.1.3. 색인어 보기 (Index Morpheme)

'색인어 보기'는 필드에 설정된 애널라이저를 통해 분석된 결과값을 불러오는 기능이다.

[표 8.4 색인어 보기 API]

HTTP Method	URL	비고
POST	/search/{collection_id}/index-morpheme	조회

[색인어 보기 설정 예시]

```
POST : http://127.0.0.1:9001/search/sample/index-morpheme
 [
   "0",
   "1"
]
```

[결과값 예시]

```
{
  "0": {
```

```
  "TITLE@kma": [
   "검색",
   "엔진"
  ],
  "CONTENT@kma": [
   "개발",
   "검색",
   "대한",
   "되었다",
   "매뉴얼",
   "매뉴얼은",
   "본",
   "엔진",
   "엔진에",
   "와이즈넛",
   "위해",
   "작성",
   "정보",
   "정보들",
   "정보들을",
   "제공",
   "제공하기"
  ]
 },
 "1": {
  "TITLE@kma": [
   "검색"
  ],
  "CONTENT@kma": [
   "내용",
   "내용보다",
   "내용을",
   "다른",
   "담고",
   "더",
   "매뉴얼",
   "매뉴얼의",
   "문서",
   "문서들",
   "문서들이",
   "본",
```

```
    "수",
    "와이즈넛",
    "와이즈넛이",
    "있습니다",
    "있을",
    "제공",
    "제공하는",
    "최신",
    "최신의"
   ]
  }
}
```

Sample 컬렉션의 경우, DATE Field와 DEPARTMENT Field를 indexFields로 설정하였으나 Long Type이므로 애널라이저를 통한 분석결과값이 없기 때문에 '색인어 보기'를 통해서 확인할 수는 없다. docId는 배열 형태로 이루어져있어 여러 개 문서를 동시에 확인 가능하다.

결과값에서 "TITLE@kma"는 필드이름과 필드에 설정되어 있는 애널라이저를 의미한다. TITLE Field에 kma, token 과 같이 여러 개의 애널라이저가 등록되어 있을 경우, 다음과 같이 두 개의 결과값("TITLE@kma", "TITLE@token")을 얻을 수 있다.

8.1.4. 색인 문서 업데이트

색인 문서 업데이트는 색인 완료된 문서에서 필드 중 부분적으로 새로운 내용으로 변경 후 재색인하는 과정이다.

8.1.4.1. 색인 문서 업데이트 (Index Update)

수정하고자 하는 문서들을 한 번에 여러 개 수정이 가능하다.

[표 8.5 멀티플 색인 문서 업데이트 API]

HTTP Method	URL	비고

POST	/index/{collection_id}/update	요청 & 리턴 있음

[DOCID가 동일한 문서를 멀티플 색인으로 색인할 경우]

POST : http://127.0.0.1:9001/index/sample/update

```
[
  {
    "DOCID": "0",
    "TITLE": "엔진"
    "CONTENT": "수정된 본 매뉴얼은 와이즈넛 검색 엔진에 대한 개발 정보들을 제공하기 위해 작성되었다."
  },
  {
    "DOCID": "1",
     "CONTENT": "수정된 와이즈넛이 제공하는 다른 문서들이 본 매뉴얼의 내용보다 더 최신의 내용을 담고 있을 수 있습니다."
  },
  {
    "DOCID": "2",
     "CONTENT": "본 매뉴얼의 수정 없이 솔루션의 기능 및 성능을 향상시킬 수 있으므로 문의사항이 있으면 아래 연락처로 연락 주시기 바랍니다.",
    "DEPARTMENT": 200
  },
  {
    "DOCID": "3",
    "DATE": "20200702030101",
    "TITLE": "웹 검색 엔진"
  }
]
```

상단의 예시에서는 모든 문서가 동일한 DOCID를 가지고 있다. 멀티플 색인 시 중복되는 DOCID를 잘못(또는 고의)로 입력했을 경우, 순차적으로 색인이 진행되므로 가장 마지막에 있는 문서의 값이 들어가게 된다. 즉, 상단의 예시에서 DOCID 0번 문서의 TITLE 값은 “웹 검색 엔진”이 된다.

8.1.5. 색인 삭제

색인 삭제 방법 DOCID를 명시하여 삭제하는 방법이 있다.

8.1.5.1. DOCID 명시 삭제 방법

[표 8.6 색인 삭제 API]

HTTP Method	URL	비고
POST	/index/{collection_id}/delete	삭제 & 리턴 있음

[DOCID 명시 삭제 예시]

```
POST : http://127.0.0.1:9001/index/sample/delete

[
  "2",
  "3",
  "4"
]
```

[결과값 예시]

```
{
  "result": true,
  "elapsedTime": 0.3758,
  "success": ["2", "3"],
  "fail": ["4"],
 "total": 3
}
```

삭제를 요청한 DOCID가 없거나 삭제가 불가능한 경우, 결과값에 fail로 요청한 DOCID 값을 반환한다. 삭제하고자 하는 문서가 하나뿐이더라도 배열 형태로 입력해야만 한다.

7.1.5.2 TimeStamp 삭제 방법

HTTP Method	URL	비고
POST	/index/{collection_id}/delete	삭제 & 리턴 있음

[TimeStamp 삭제 예시]

```
POST : http://127.0.0.1:9001/index/sample/delete

{
  "query":"<$TIMESTAMP:lt:1707963261629>"
}
```

[결과값 예시]

```
{
  "result": true,
  "elapsedTime": 69.93780000000001,
  "success": [
    "2",
    "3",
    "4",
    "5"
  ],
  "fail": [],
  "total": 4
}
```

삭제를 요청한 Timestamp 이전의 날짜에 색인한 데이터들을 삭제한다.

사용시 해당 날짜 이전 모든 색인 데이터가 삭제되니 주의해야 한다.

8.1.6. 색인 데이터 정보

컬렉션 색인 관련 정보를 가져온다. 해당 기능을 통해 가져오는 항목은 다음 표를 참고한다.

[표 8.7 색인 데이터 정보 항목]

항목	설명

totalDocs	전체 문서 수
totalSize	전체 디렉토리 사이즈
enableShards	현재 활성화된 샤드에 대한 정보
disableShards	현재 비활성화된 샤드

[표 8.8 색인 정보 API]

HTTP Method	URL	비고
GET	/index/{collection_id}	요청 & 리턴 있음

[결과값 예시]


```
{
  "totalDocs": 0,
  "totalSize": 69,
  "enableShards": [
    {
      "shardName": "P_5f0b3eee-d0f0-4587-a46a-768620a85d07",
      "documentNum": 0,
      "directorySize": 69
    },
    {
      "shardName": "R_b49e4057-5702-46ef-8026-507b08ecd2d5",
      "documentNum": 0,
      "directorySize": 69
    }
  ],
  "disableShards": [
    {
      "shardName": "P_b49e4057-5702-46ef-8026-507b08ecd2d5",
      "documentNum": -1,
      "directorySize": -1
    },
```

```
    {
      "shardName": "R_5f0b3eee-d0f0-4587-a46a-768620a85d07",
      "documentNum": -1,
      "directorySize": -1
    }
  ]
}
```

8.2. 배포

8.2.1. 배포 운영 방식

배포는 색인 전용 노드로 부터 색인된 데이터를 검색 노드로 전송하는 기능이다.

8.2.1.1. 색인 노드 재할당

색인 노드 재할당 기능은 엔진 노드에서 컬렉션을 생성한 이후, 인덱서 증감에 따른 구동중인 색인 노드들에 색인 샤드를 할당하는 기능이다. 색인 노드를 사용할 경우 해당 작업이 이루어져야 색인 및 배포를 수행할 수 있다.

[표 8.9 색인 노드 재할당 API]

HTTP Method	URL	비고
POST	/collections/{collection_id}/relocate	할당 & 리턴 있음

[색인 노드 재할당 예시]

```
POST : http://127.0.0.1:9900/collections/{collection_id}/relocate
```

8.2.1.2. 정적 색인 시작

해당 API 를 보내면 정적 색인 시작 준비를 한다. 향후 똑같이 동적 색인을 사용하면 된다.

[표 8.10 정적 배포 API]

HTTP Method	URL	비고
POST	/index/{collection_id}/static	배포 & 리턴 있음

[정적 배포예시]

POST : http://127.0.0.1:9900/index/{collection_id}/static

9. 검색

검색은 “기본 검색”, “필터 쿼리” 등 목적에 따라 쿼리를 선택하여 다양한 검색을 수행할 수 있다.

[표 9.1 검색 쿼리 종류]

쿼리명	설명
기본 검색	입력한 키워드가 정확히 일치하는 결과값을 제공 벡터 검색 시 기본 검색에 사용한 쿼리를 벡터로 변환하여 검색 진행
필터 쿼리	검색 결과를 정해진 조건으로 필터링(Filtering)

검색을 수행하기 위해서는 애널라이저, 사전, 컬렉션 정보 등록 또는 Model Server 등록 및 색인 과정이 먼저 진행되어야 한다. 테스트를 위한 애널라이저, 컬렉션, 색인 설정은 각 쿼리의 예제에 제시된 JSON을 참고하여 등록한다. 단, 각 예제에서 사전 등록과 Model Server 등록은 생략한다.

9.1. [예제 실행을 위한 설정]

검색 예제를 위한 사전, 애널라이저, 컬렉션 설정으로 9장 검색에서 공용으로 사용하는 설정이다.

[사용자사전 usernounA 등록]

```
POST : http://127.0.0.1:9001/dictionaries/usernounA

{
  "type": "usernoun",
  "data": {
    "반팔티": {
      "separation": "21"
    }
  }
```

```
}
```

[사용자사전 usernounB 등록]

```
POST : http://127.0.0.1:9001/dictionaries/usernounB

{
  "type": "usernoun",
  "data": {
    "반팔티": {
      "separation": "12"
    }
  }
}
```

[키워드 가중치 사전 field_keyword_weight 등록]

```
POST : http://127.0.0.1:9001/dictionaries/field_keyword_weight

{
  "type": "keyword-weight",
  "data": {
    "id-2": {
      "fieldScores": [
        "TITLE/500",
        "CONTENT/50"
      ],
      "keywords": [
        "엔진"
      ]
    }
  }
}
```

[동의어 사전 synonym 등록]

```
POST : http://127.0.0.1:9001/dictionaries/synonym
{
  "type": "synonym",
  "data": {
```

```
    "검색": {
      "synonym": [
        "wisenut"
      ],
      "subword": [
        "웹"
      ]
    },
    "엔진": {
      "synonym": [
        "engine"
      ],
      "subword": [
        "자바"
      ]
    }
  }
}
```

[애널라이저 설정]

```
POST : http://127.0.0.1:9001/analysis
{
  "analyzer": {
    "kma": {
      "lae": "kma-black",
      "regulator": "R1",
      "dictionary": [
        "synonym"
      ]
    },
    "kma-a": {
      "lae": "kma-a"
    },
    "kma-b": {
      "lae": "kma-b"
```

```
    },
    "token": {
      "lae": "token",
      "regulator": "R2"
    },
    "bigram-without-regulator": {
      "lae": "bigram"
    }
  },
  "regulator": {
    "R1": {
      "allow": {
        "value": [
          "?",
          "*"
        ]
      }
    },
    "R2": {
      "divide": {
        "value": [
          "#"
        ]
      }
    }
  },
  "lae": {
    "kma-black": {
      "type": "kma",
      "kmaOption": "A+S+",
      "licensePath": "korean/license.xml",
      "knowledgePath": "korean/wkb_nocomp.klg"
    },
    "kma-a": {
      "type": "kma",
```

```
        "kmaOption": "A+R2",
        "licensePath": "korean/license.xml",
        "knowledgePath": "korean/wkb_nocomp.klg",
        "dictionary": [
          "usernounA"
        ]
      },
      "kma-b": {
        "type": "kma",
        "kmaOption": "A+R2",
        "licensePath": "korean/license.xml",
        "knowledgePath": "korean/wkb_nocomp.klg",
        "dictionary": [
          "usernounB"
        ]
      },
      "bigram": {
        "type": "ngram",
        "length": 2
      },
      "token": {
        "type": "token"
      }
    }
}
```

[sample 컬렉션 설정]

```
POST : http://127.0.0.1:9001/collections/sample
{
  "numOfShards": 1,
  "numOfReplication": 0,
  "recentQuery": 10,
  "fields": [
```

```
    "DOCID",
    "DATE",
    "TITLE",
    "CONTENT",
    "GOODS_NAME",
    "CATEGORY",
    "DEPARTMENT",
    "AUTHORITY_ID",
    "LATITUDE",
    "LONGITUDE"

  ],
  "indexFields": {
    "DATE": {
      "type": "date",
      "format": "yyyyMMddHHmmss"
    },
    "TITLE": {
      "type": "text",
      "analyzer": [
        "kma",
        "token"
      ],
      "model": "kodpr"
    },
    "CONTENT": {
      "type": "text",
      "analyzer": [
        "kma"
      ],
      "deduplicate": true
    },
    "GOODS_NAME": {
      "type": "text",
      "analyzer": [
```

```
          "kma"
        ]
      },
      "CATEGORY": {
        "type": "category",
        "depthIndicator": ">",
        "separator": ",",
      },
      "DEPARTMENT": {
        "type": "long"
      },
      "AUTHORITY_ID": {
        "type": "authority"
      },
      "LATITUDE": {
        "type": "double"
      },
      "LONGITUDE": {
        "type": "double"
      }
    }
}
```

[sample 컬렉션에 대한 authority 컬렉션 설정]

```
POST : http://127.0.0.1:9001/authorities/sample

[
  {
    "key": "1",
    "accounts": [
      "user",
      "admin"
    ]
  },
  {
    "key": "2",
    "accounts": [
```

```
        "admin"
      ]
    },
    {
      "key": "3",
      "accounts": [
        "user"
      ]
    }
]
```

[sampleA 컬렉션 설정 예시]

```
POST : http://127.0.0.1:9001/collections/sampleA

{
  "numOfShards": 2,
  "numOfReplication": 0,
  "recentQuery": 5,
  "fields": [
    "DOCID",
    "sampleA_GOODS_NAME",
    "sampleA_PRICE"
  ],
  "indexFields": {
    "sampleA_GOODS_NAME": {
      "type": "text",
      "analyzer": [
        "kma-a"
      ]
    },
    "sampleA_PRICE": {
      "type": "long"
    }
  }
}
```

[sampleB 컬렉션 설정 예시]

```
POST : http://127.0.0.1:9001/collections/sampleB
```

```
{
  "numOfShards": 1,
  "numOfReplication": 0,
  "recentQuery": 5,
  "fields": [
    "DOCID",
    "sampleB_GOODS_NAME",
    "sampleB_PRICE"
  ],
  "indexFields": {
    "sampleB_GOODS_NAME": {
      "type": "text",
      "analyzer": [
        "kma-b"
      ]
    },
    "sampleB_PRICE": {
      "type": "long"
    }
  }
}
```

9.2. 기본 검색 쿼리 (Basic Query)

9.2.1. 개요

이번 장에서는 검색 쿼리를 수행하기 위해 필요한 기본 조건에 대해 예제를 기준으로 알아본다. 예제 실행을 위해 먼저 애널라이저, 컬렉션, 색인 설정을 진행한다. 애널라이저, 컬렉션, 색인 설정 방법은 해당 각 문단을 참고한다.

9.2.2. 기본 검색

실행하기 위해 기본검색 API 호출과 query 옵션 사용 방법에 대하여 알아본다.

9.2.2.1. API 호출

[표 9.2 검색 API]

HTTP Method	URL	비고
POST	/search	요청
POST	/search/{collection_id}	요청

9.2.2.2. 예제

9.2.2.2.1. 검색 기본 구조

9.2.2.2.1.1. [기본 검색 예시]

```
POST : http://127.0.0.1:9001/search/sample
{
  "commonQuery": "검색",
  "traceInfo": {
    "trace": true,
    "timeout": 0.0
  },
```

```
  "sessionInfo": [
    "user1",
    "user2"
  ],
  "hideQueryLog": false,
  "searchMode": 0,
  "synonymExpansion": 0,
  "useSynonym": 0,
  "useLa": 0,
  "searchField": [
    "TITLE"
  ],
  "paging": {
    "from": 0,
    "size": 10
  },
  "sorting": [
    {
      "field": "score",
      "reverse": false
    }
  ]
}
```

9.2.2.2.1.2. [통합 검색 예시]

```
POST : http://127.0.0.1:9001/search
{
  "collection": {
    "sample": {
      "id": "sample",
      "collectionId": "sample",
      "searchMode": 0,
      "synonymExpansion": 0,
      "useSynonym": 0,
      "useLa": 0,
```

```
        "collectionQuery" : "<TITLE:contains:검색>",
        "searchField": [
          "TITLE"
        ],
        "documentField": [
          "DOCID",
          "TITLE",
          "CONTENT"
        ],
        "paging": {
          "from": 0,
          "size": 10
        },
        "sorting": [
          {
            "field": "score",
            "reverse": false
          }
        ]
      }
    }
}
```

9.2.2.2.2. 검색 옵션

[표 9.3 검색 옵션]

옵션	설명	필수
traceInfo	질의를 처리하는데 수행된 작업들을 보여준다. trace : trace 정보를 보여줄 것인지 설정 timeout : 쿼리의 검색시간 타임아웃 설정. 해당 시간이 넘어가는 로그만 찍힌다. 실제 서비스시 사용하면 성능에 영향이 있을 수 있으므로 개발시 또는 디버깅 시에만 사용하는 것이 바람직하다.	X
sessionInfo	사용자 세션 관리를 위해 사용자의 정보를 입력한다. 입력 값은 질의 로그에 기록된다.	X

hideQueryLog	해당 질의를 로그파일에 기록할 것인지 여부를 결정할 때 사용한다. 사용하지 않을 경우 기본적으로 질의 로그에 기록한다. querylog를 남김 : false (default) querylog를 남기지 않음 : true	X

9.2.2.2.3. 쿼리 내부 옵션

[표 9.4 기본 검색 쿼리 내부 옵션]

옵션	설명	필수
searchMode	어떤 방식으로 검색할 지 선택하는 옵션이다. 키워드 검색 : 0, “keyword” 자연어 검색 : 1, “natural” 벡터 검색 : 2, “vector” 자연어 검색은 쿼리 확장 검색이라고 설명할 수 있으며 자연어 쿼리가 들어올 경우 해당 쿼리에서 특정 키워드를 추출하여 검색하는 방식이다. 이 경우 기존 키워드 검색에서 사용하던 예약어에 해당되는 특수문자를 모두 검색 쿼리로 만들어 사용한다. 고급 검색을 사용하기 위한 특수문자도 문자열로서 일반 검색으로 사용되기에 고급 검색을 사용할 수 없다. 벡터 검색의 경우 자세한 예시는 [9.2.3] 벡터 검색 예제에서 확인할 수 있다. 옵션을 작성하지 않을 경우 키워드 검색이 기본으로 진행된다.	X
paging	검색 결과를 페이징 처리하기 위한 옵션이다. 해당 옵션을 생략하였을 때 기본값은 10으로 문서 결과가 최대 10건 출력된다.	X
synonymExpansion	동의어확장법을 설정하는 옵션이다. allTerm과 originalTerm 두가지 방식이 있다. allTerm : 0 (default) originalTerm : 1 originalTerm으로 설정할 경우 형태소분석된 term에 대해서는 동의어확장이 수행되지 않는다. 예시) 검색어 : [정부산하에] 일 경우, 동의어에 [정부산하] [정부] 등이 등록되어 있더라도 동의어확장이 되지 않는다. [정부산하에] 라는 originalTerm에 대한 동의어 확장이 수행된다.	X
useSynonym	동의어 사전 적용여부를 설정하는 옵션이다.	X

	사용하지 않음 : 0 (default) 동의어사전 적용 : 1 useSynonym의 여부와 관계없이 like, matrix, ngram은 동의어사전이 적용 불가능하다. like, matrix, ngram은 애널라이저 등록 시, 동의어사전을 적용할 수 없으며, 검색 시에도 쿼리식에 useSynonym 옵션을 1으로 설정하여도 동의어확장이 수행되지 않는다. 자세한 예시는 [8.2.2.3] 쿼리 확장 옵션 예제에서 확인할 수 있다.	
useLa	형태소분석기 사용여부를 설정하는 옵션이다. 사용하지 않음 : 0 (default) 형태소분석기 적용 : 1 useLa 사용하지 않을 경우, useSynonym의 여부와 관계없이 동의어사전 적용이 불가하다. 쿼리는 token과 동일하게 확장되고, useSynonym 옵션을 1으로 설정하여도 동의어확장이 수행되지 않는다. 자세한 예시는 [8.2.2.3] 쿼리 확장 옵션 예제에서 확인할 수 있다.	X
useOrigin	원본 토큰 사용여부를 사용하는 옵션이다. 사용하지 않음 : 0 원본토큰 사용 : 1 형태소 분석이 실패한 원본 토큰을 쿼리에 사용하기 위한 옵션	X
documentField	검색 결과 문서내용을 출력할 때, 출력한 필드명을 적는다. 생략하였을 때 기본값은 컬렉션 세팅 기준 전체 필드를 출력한다.	X
commonQuery	기본 검색을 수행하기 위해 필요한 필수 옵션이다.	O
vectorQuery	벡터 검색을 수행하기 위한 옵션이다. 자세한 예시는 [9.2.3.2.1] 벡터 검색 구조에서 확인할 수 있다. 자세한 옵션은 [9.2.3.2.2] 벡터 쿼리 내부 옵션에서 확인할 수 있다.	X
sorting	검색 결과를 정렬하기 위한 옵션이다. 생략하였을 때 기본값은 문서가 색인된 순서이다.	X
similarity	랭킹 설정 (설정을 하지 않을 경우, classic를 기본으로 사용함) - bm25 - boolean - classic - dfi - dfr - lm	X

9.2.2.2.4. 결과

상단 예시의 JSON 형태로 요청을 보내게 되면, "TITLE" 필드에 "검색"이라는 키워드가 포함된 모든 문서가 추출된다. 이와 같이 쿼리문만 입력하여도 검색이 수행되게 된다. 이는 필수값들이 모두 default로 설정되어 검색 요청을 수행하기 때문이다. 결과값 필드에 대한 설명은 하단의 표를 참고한다.

[표 9.5 결과값 필드 별 설명]

key	설명
totalHits	총 검색 결과 건수를 의미한다.
pagingInfo	paging 정보를 의미한다.
	from : 페이지의 첫번째 위치
	size : 한 페이지에 출력 될 문서 건수를 의미한다.
document	검색 결과 문서들을 의미한다.
typoKeyword	오타교정 사용 시, 오타교정 된 키워드들을 의미한다.
elapsedTime	총 검색 수행 시간을 의미한다.
aggregations	Aggregation Query의 결과를 출력하는 영역이다.

9.2.2.3. 쿼리 확장 옵션 예제

쿼리 확장 옵션에 따라 검색 쿼리가 달라지게 되고, 그에 따라 결과값도 달라지게 된다. 하단의 예시를 통해 쿼리 옵션별로 확장되는 쿼리에 대해 알아볼 수 있다.

[기본 검색 예시]

```
POST : http://127.0.0.1:9001/search/sample
{
```

```
  "commonQuery": "검색하는 엔진"
}
```

[통합 검색 예시]

```
POST : http://127.0.0.1:9001/search
{
  "collection": {
    "sample": {
      "id": "sample",
      "collectionId": "sample",
      "collectionQuery" : "<TITLE:contains:검색엔진>"
    }
  }
}
```

9.2.2.3.1. 쿼리 확장 옵션 기본 구조

```
{
  "useLa": {value},
  "useSynonym": {value},
  "synonymExpansion": {value},
}
```

9.2.2.3.2. 쿼리 옵션

옵션	설명	필수
synonymExpansion	동의어확장법을 설정하는 옵션이다. allTerm과 originalTerm 두가지 방식이 있다. allTerm : 0 (default) originalTerm : 1 originalTerm으로 설정할 경우 형태소분석된 term에 대해서는 동의어확장이 수행되지 않는다. 예시) 검색어 : [정부산하에] 일 경우, 동의어에 [정부산하] [정부] 등이 등록되어 있더라도 동의어확장이 되지 않는다. [정부산하에] 라는 originalTerm에 대한 동의어 확장이 수행된다.	X
useSynonym	동의어 사전 적용여부를 설정하는 옵션이다.	X

	사용하지 않음 : 0 동의어사전 적용 : 1 (default) useSynonym의 여부와 관계없이 like, matrix, ngram은 동의어사전이 적용 불가능하다. like, matrix, ngram은 애널라이저 등록 시, 동의어사전을 적용할 수 없으며, 검색 시에도 쿼리식에 useSynonym 옵션을 1으로 설정하여도 동의어확장이 수행되지 않는다.	
useLa	형태소분석기 사용여부를 설정하는 옵션이다. 사용하지 않음 : 0 형태소분석기 적용 : 1 (default) useLa 사용하지 않을 경우, useSynonym의 여부와 관계없이 동의어사전 적용이 불가하다. 쿼리는 token과 동일하게 확장되고, useSynonym 옵션을 1으로 설정하여도 동의어확장이 수행되지 않는다. 일본어와 중국어의 경우 띄어쓰기를 거의 사용하지 않는 문장 구조를 가지고 있기에 정확한 검색을 위해서 형태소분석기 적용이 필요하다.	X

9.2.2.3.3. 검색 옵션 별 쿼리 확장

9.2.2.3.3.1. useLa, useSynonym 을 사용하지 않을 경우

useSynonym 값이 0일경우, 동의어 사전이 적용되지 않으므로 synonymExpansion의 값은 무의미하다.

[일반 검색 쿼리 예시]

```
POST : http://127.0.0.1:9001/search/sample
{
  "useLa": 0,
  "useSynonym": 0,
  "synonymExpansion": 0,
}
```

[통합 검색 쿼리 예시]

```
POST : http://127.0.0.1:9001/search
{
  "collection": {
    "sample": {
```

```
        "collectionId": "sample",
        "useLa": 0,
        "useSynonym": 0,
        "synonymExpansion": 0,
    }
}
```

CONTENT 필드는 kma 애널라이저를 사용하지만, 적용되지 않는 것을 확인할 수 있다. 또한, useLa 값이 0일경우, 동의어 사전이 적용되지 않기 때문에 useSynonym과 synonymExpansion의 값은 무의미하다.

[검색 쿼리 예시 1 쿼리 확장]

```
{ AND
  { OR
    { AND
      { OR
        TITLE@token:검색하는
        TITLE@kma:검색하는
      }
      { OR
        TITLE@token:엔진
        TITLE@kma:엔진
      }
    }
  }
}
```

9.2.2.3.3.2. 형태소 분석기만 적용하는 경우

다음은 형태소분석기는 사용하되(useLa = 1), 동의어사전은 적용하지 않는 경우이다.(useSynonym = 0)

[일반 검색 쿼리 예시]

```
POST : http://127.0.0.1:9001/search/sample
{
  "useLa": 1,
```

```
  "useSynonym": 0,
  "synonymExpansion": 0,
}
```

[통합 검색 쿼리 예시]

```
POST : http://127.0.0.1:9001/search
{
  "collection": {
    "sample": {
      "collectionId": "sample",
      "useLa": 1,
      "useSynonym": 0,
      "synonymExpansion": 0,
    }
}
```

쿼리가 동의어 확장 없이 형태소분석만 된 것을 확인할 수 있다. 앞서 말했듯, useSynonym 값이 0일경우, 동의어 사전이 적용되지 않으므로 synonymExpansion의 값은 무의미하다.

[검색 쿼리 예시2 쿼리 확장]

```
    { AND
     { OR
       { AND
         { OR
           TITLE@token:검색하는
           TITLE@kma:검색
         }
         { OR
           TITLE@token:엔진
           TITLE@kma:엔진
         }
       }
     }
  }
```

9.2.2.3.3.3. 형태소 분석기, 동의어사전을 모두 사용하는 경우

동의어 사전이 적용되므로 (useSynonym=1) synonymExpansion의 값에 따라 동의어확장 범위가 조절된다. 하단의 예시는 synonymExpasion 값을 0으로 두어 모든 term에 대한 확장이 수행된다.

[일반 검색 쿼리 예시]

```
POST : http://127.0.0.1:9001/search/sample
{
  "useLa": 1,
  "useSynonym": 1,
  "synonymExpansion": 0,
}
```

[통합 검색 쿼리 예시]

```
POST : http://127.0.0.1:9001/search
{
  "collection": {
    "sample": {
      "collectionId": "sample",
      "useLa": 1,
      "useSynonym": 1,
      "synonymExpansion": 0,
    }
}
```

아래와 같이 [와이즈넛에서]라는 originalTerm에서 형태소 분석된 term인 [와이즈넛] 에 대해서도 동의어확장이 수행되는 것을 확인할 수 있다.

[검색 쿼리 예시3 쿼리 확장]

```
 { AND
  { OR
    { OR
```

```
        TITLE@kma:와이즈넛에서
        TITLE@kma:wisenut에서
        { AND
          { OR
            TITLE@kma:와이즈넛
            TITLE@kma:표준검색엔진
            TITLE@kma:wisenut
            TITLE@kma:검색엔진
          }
        }
      }
      { OR
        { AND
          { OR
            TITLE@token:와이즈넛에서
          }
        }
      }
    }
}
```

9.2.2.3.3.4. originalTerm 에 대해서만 확장하는 경우

마지막으로, synonymExpansion 값을 1로 두어 originalTerm에 대해서만 동의어확장을 수행하는 경우이다.

[일반 검색 쿼리 예시]

```
POST : http://127.0.0.1:9001/search/sample
{
  "useLa": 1,
  "useSynonym": 1,
  "synonymExpansion": 1,
}
```

[통합 검색 쿼리 예시]

```
POST : http://127.0.0.1:9001/search
{
  "collection": {
    "{collection-alias-id}": {
      "collectionId": "sample",
      "useLa": 1,
      "useSynonym": 1,
      "synonymExpansion": 1,
    }
}
```

형태소 분석된 term인 [와이즈넛] 에 대해서는 동의어확장이 수행되지 않고,
[와이즈넛에서]라는 originalTerm에 대해서만 동의어확장이 수행되기때문에 일치하는 동의어가 없어 확장되지 않는다.

[검색 쿼리 예시4 쿼리 확장]

```
{ AND
  { OR
    { OR
      TITLE@kma:와이즈넛에서
      TITLE@kma:wisenut에서
      { AND
        { OR
          TITLE@kma:와이즈넛
          TITLE@kma:wisenut에서
        }
      }
    }
    { OR
      { AND
        { OR
```

```
            TITLE@token:와이즈넛에서
          }
        }
      }
    }
}
```

9.2.2.3.4. 부가 기능

v7.1 버전 이후부터 띄어쓰기를 포함한 동의어 및 표제어에 대한 확장을 지원한다. 띄어쓰기가 포함된 검색어에 대해 동의어 확장을 하고자하는 경우 반드시 **useLa 옵션과 useSynonym 옵션이 1**이어야하며 **동의어 확장을 원하는 문장을 큰따옴표로 감싸서 검색 요청**을 보내야한다.

[띄어쓰기를 포함한 검색어에 대한 동의어 사전 확장 예시]

```
POST : http://127.0.0.1:9001/search
{
  "commonQuery":  "\"와이즈넛 검색엔진\" 매뉴얼"
  "collection": {
    "{collection-alias-id}": {
      "collectionId": "sample",
      "useLa": 1,
      "useSynonym": 1,
      "synonymExpansion": 0,
      ...
    }
}
```

9.2.3. 벡터 검색

실행하기 위해 벡터검색 API 호출과 query 옵션 사용 방법에 대하여 알아본다.

9.2.3.1. API 호출

벡터 검색을 위한 별도의 API는 없으며 기본 검색과 동일한 API를 사용한다. 벡터 검색을 하기 위해서는 “searchMode” 옵션의 값을 2로 설정한 후 검색을 진행해야 한다. “searchField”에는 collection 설정 시 “model”값을 작성한 필드 값을 작성하여 진행한다.

아래 예시는 키워드가 색인된 필드와 벡터가 색인된 필드를 동시에 사용하는 예시이다.

9.2.3.2. 예제

9.2.3.2.1. 벡터 검색 구조

[commonQuery를 사용한 검색어 기반 벡터 검색 예시]

```
POST : http://127.0.0.1:9001/search/sample_bbs
{
  "commonQuery": "와이즈넛은 무슨 회사야",
  "searchField": [
    "TITLE"
  ],
  "searchMode": 2
}
```

[7.2.1] 컬렉션 초기화의 컬렉션 예시를 통해 살펴볼 수 있다. 선언 된 “indexField”에서 “TITLE” 필드만 “model”값을 가지고 있으므로 벡터 검색이 가능한 필드는 “TITLE” 하나이다. 모델이 작성된 필드가 더 있다면 “searchField”에 추가하여 벡터 검색이 가능하다. commonQuery를 사용한 검색어 기반 벡터 검색 시 k는 기본값으로 설정된다.

[vectorQuery를 사용한 검색어 기반 벡터 기본 검색 예시]

```
POST : http://127.0.0.1:9001/search/sample_bbs
{
  "vectorQuery": [
```

```
    {
      "field": "TITLE",
      "queryStr": "와이즈넛은 무슨 회사야",
      "k": 5
    }
  ]
}
```

[vectorQuery를 사용한 검색어 기반 벡터 통합 검색 예시]

```
POST : http://127.0.0.1:9001/search
{
  "collection": {
    "sample_bbs": {
      "id": "sample_bbs",
      "collectionId": "sample_bbs",
      "vectorQuery": [
        {
          "field": "TITLE",
          "queryStr": "와이즈넛은 무슨 회사야",
          "k": 5
        }
      ]
    }
  }
}
```

vectorQuery를 사용하면 여러 개의 검색 필드나 검색어를 구성하여 벡터 검색을 수행할 수 있다. 벡터 검색 시 queryStr을 사용할 경우, 해당 검색 필드에 설정된 모델 서버가 정상 작동할 때 검색을 수행할 수 있다.

[vectorQuery를 사용한 사용자 입력 기반 벡터 검색 예시]

```
POST : http://127.0.0.1:9001/search/sample_bbs
{
  "vectorQuery": [
    {
```

```
        "field": "TITLE",
        "vector": [0.13, 0.2, ···, -0.1],
        "k": 5
      }
    ],
    "searchMode": 2
}
```

검색어가 아닌 사용자가 직접 입력한 임베딩된 벡터를 통해 벡터 검색을 수행할 수 있다. 검색할 "indexField"의 "model"에서 사용하는 벡터의 길이와 입력한 벡터의 길이가 일치하지 않으면 검색이 수행되지 않는다.

9.2.3.2.2. 벡터 쿼리 옵션

[표 9.6 벡터 쿼리 옵션]

옵션	설명	필수
field	벡터 검색을 수행할 검색 필드 명을 입력한다.	O
queryStr	벡터 검색을 수행할 검색어를 입력한다. 해당 검색 필드에 설정된 모델 서버가 정상 작동할 때 검색을 수행할 수 있다. 벡터 쿼리 구성 시 queryStr과 vector 옵션 중 반드시 하나만 사용해야 한다.	X
vector	벡터 검색을 수행할 임베딩된 벡터를 입력한다. 벡터 쿼리 구성 시 queryStr과 vector 옵션 중 반드시 하나만 사용해야 한다.	X
k	벡터 검색을 통해 검색할 문서의 개수를 입력한다. 해당 값이 커질수록 벡터 검색의 탐색 범위가 넓어져 검색 속도가 늘어나므로, 아래 권장값을 참고하거나 서버 사양에 맞는 적정 값을 선택한다. - 권장값 : 1.5 * {검색하고자 하는 문서 건수} - 기본값 : 10,000	X

9.2.3.2.3. 벡터 검색 결과

상단 예시의 searchField 형태로 요청을 보내게 되면, "TITLE" 필드에 "와이즈넛은 무슨 회사야"라는 쿼리가 컬렉션 설정 시 "TITLE" 필드에 작성된 모델을 사용하여 벡터 변환이 진행된 후 벡터 검색을 수행하고 쿼리와 유사한 내용을 가진 문서를 추출한다. 아래 예시에서 검색 결과를 확인할 수 있다.

[벡터 통합 검색 쿼리 예시]

```
POST : http://127.0.0.1:9001/search
{
  "commonQuery": "와이즈넛은 무슨 회사야",
  "collection": {
    "sample_bbs": {
      "id": "sample_bbs",
      "collectionId": "sample_bbs",
      "searchMode": 2,
      "useSynonym": 1,
      "useLa": 1,
      "documentField": [
        "DOCID",
        "TITLE",
        "CONTENT"
      ],
      "searchField": [
        "TITLE"
      ],
```

```
        "paging": {
          "from": 0,
          "size": 2
        },
        "snippet": {
          "field": {
            "CONTENT": {
              "length": 200,
              "option": "frequency"
            }
          }
        }
      }
    }
}
```

[벡터 통합 검색 결과 예시]

```
POST : http://127.0.0.1:9001/search
{
  "sample_bbs": {
    "totalHits": 2,
    "pagingInfo": {
      "from": 0,
      "size": 10
    },
    "document": [
      {
```

"distance": -1.0,
"score": 191.87103,
"weight": 0.0,
"fields": {
"DOCID": "20",
"TITLE": "와이즈넛, 구글과 함께 공공기관 대상 공동 세미나 개최",
"CONTENT": "통합 검색솔루션 대표기업 ㈜와이즈넛(대표 박재호, http://www.wisenut.com)과 세계적인 검색포털 구글코리아(대표 이원진, www.google.co.kr)이 공공 분야 온라인 광고 부문 핵심 파트너 관계를 맺고 공공 기관을 대상으로 오는 26일(화) 소공동 롯데호텔에서 "대민사이트, 니즈 충족에서 관심 획득으로"라는 주제의 공동 세미나를 개최..."
}
},
{
"distance": -1.0,
"score": 177.49304,
"weight": 0.0,
"fields": {
"DOCID": "5",
"TITLE": "시맨틱 기술 적용 검색솔루션 출시",
"CONTENT": "와이즈넛(대표 박재호)은 정교한 언어분석기술과 시맨틱 기술을 적용한 '와이즈 티(WISE TEA)'를 출시했다고 11일 밝혔다. 와이즈 티는 형태소 분석기술과 시맨틱 기술이 적용돼 문서 내의 내용을 기반으로 의미를 분석해 핵심 주제어를 자동 추출하고, 핵심 주제어간 연관도 분석을 다양한 사용자 인터페이스(UI)로 재구성해 사용자 의도에 적합한 검색결과를 제..."
}
}
],
"typoKeyword": [],

```
    "elapsedTime": 20.0,
    "aggregations": {}
  }
}
```


[7.2.1] 컬렉션 설정 예시를 바탕으로 입력된 쿼리가 “dpr” 모델을 사용해 벡터 변환하여 벡터 검색이 진행된 결과를 볼 수 있다.

9.3. 필터 쿼리 (Filter Query)

9.3.1. 개요

이번 장에서는 검색 결과를 정해진 조건으로 필터링(Filtering)하는 기능을 살펴보도록 하자. 필터링이란 문서의 메타 정보와 일치여부, 크다, 작다 등의 조건에 적합한 검색 결과만 산출하는 기능이다. 텍스트 타입 필드에 필터 operator를 사용하기 위해서는 token 애널라이저로만 필드를 색인해야한다. 필터 하려는 문자를 분석하게 되버리면 쿼리가 분석된 term만큼 or 확장되고 의도하지 않은 결과가 나올 수 있기 때문이다.

9.3.2. Type별 연산자 정리

필터 쿼리는 데이터 타입별로 지원하는 연산자가 다르다. 필터 쿼리를 통해 필터링(Filtering)되는 조건들은 score 값에 영향을 미치지 않는다. text type과 long type 두 가지를 지원하며, 타입 별 지원 연산자는 하단의 표를 따른다.

9.3.2.1. Text Type operator

[표 9.7 Text Type operator 설명]

Text type operator	**설명**
match	문서의 필드 값과 기준 값이 일치하는 문서를 검색한다.
not_match	문서의 필드 값과 기준 값이 일치하지 않는 문서를 검색한다.
in	문서의 필드 값이 기준 값들 중 일치하는 문서를 검색한다.
not_in	문서의 필드 값이 기준 값들에 포함되지 않는 문서를 검색한다.
lt	문서의 필드 값이 기준 값보다 작은 문서를 검색한다. (아스키코드값 기준)
lte	문서의 필드 값이 기준 값보다 작거나 같은 문서를 검색한다.

	(아스키코드값 기준)
gt	문서의 필드 값이 기준 값보다 큰 문서를 검색한다. (아스키코드값 기준)
gte	문서의 필드 값이 기준 값보다 크거나 같은 문서를 검색한다. (아스키코드값 기준)
start	문서의 필드 값이 기준 값으로 시작하는 문서를 검색한다.
end	문서의 필드 값이 기준 값으로 끝나는 문서를 검색한다.
substring	문서의 필드 값이 기준 값을 포함하는 문서를 검색한다.
contains	문서의 필드 값이 기준 값을 가지고 있는 문서를 검색한다.

9.3.2.2. Long Type operator

[표 9.8 Long Type operator 설명]

Long type operator	**설명**
eq	문서의 필드 값과 기준 값이 일치하는 문서를 검색한다.
not_eq	문서의 필드 값과 기준 값이 일치하지 않는 문서를 검색한다.
in	문서의 필드 값이 기준 값들 중 일치하는 문서를 검색한다.
not_in	문서의 필드 값이 기준 값들에 포함되지 않는 문서를 검색한다.
lt	문서의 필드 값이 기준 값보다 작은 문서를 검색한다.
lte	문서의 필드 값이 기준 값보다 작거나 같은 문서를 검색한다.
gt	문서의 필드 값이 기준 값보다 큰 문서를 검색한다.
gte	문서의 필드 값이 기준 값보다 크거나 같은 문서를 검색한다.
range	문서의 필드 값이 주어진 범위의 값 사이에 있는 문서를 검색한다. 내부에 lt, lte, gt, gte 연산자 사용하여 range 지정

9.3.2.3. filterQuery 기본구조

[filterQuery 연산 기본구조]

```
"filterQuery": "<{fieldName}:{operator}:{query}>"
```

9.3.2.4. 옵션설명

옵션	설명	필수
fieldName	필터링을 수행할 필드명을 입력한다. indexField로 잡힌 필드명만 기입 가능하다.	O
operator	사용할 연산자를 기입한다. 필드별 타입에 따라 연산 가능한 연산자를 입력해야한다. 8.3.2 Type별 연산자 정리에서 사용가능한 연산자를 확인할 수 있다.	O
query	검색어를 입력한다.	O

9.3.3. MATCH 연산자

Match 연산자는특정 필드에 입력한 키워드와 일치하는 문서를 검색한다. filter는 score에 영향을 미치지 않는다.

[match 연산자 사용 예시]

```
POST : http://127.0.0.1:9001/search
{
  "commonQuery": "입찰",
  "collection": {
    "alias": {
      "id": "sample",
      "collectionId": "sample",
      "searchField": [ "TITLE" ],
      "filterQuery": "<CONTENT:match:경영 지원팀>"
```

```
    }
  }
}
```

위 예시는 CONTENT 필드의 값이 “경영 지원팀”인 결과만 리턴한다.

9.3.4. NOT_MATCH 연산자

NOT_MATCH 연산자는 특정 필드에 입력한 키워드가 포함되지 않는 문서를 검색한다.

[not_match 연산자 사용 예시]

```
POST : http://127.0.0.1:9001/search
{
  "commonQuery": "입찰",
  "collection": {
    "alias": {
      "id": "sample",
      "collectionId": "sample",
      "searchField": [ "TITLE" ],
      "filterQuery": "<CONTENT:not_match:경영 지원팀>"
    }
  }
}
```

위 예시는 CONTENT 필드의 값이 “경영 지원팀”이 아닌 결과만 리턴한다.

9.3.5. IN 연산자

IN 연산자는 특정 필드에 입력한 하나 이상의 키워드가 일치하는 문서를 검색한다. IN 연산자 사용 시 띄어쓰기를 기준으로 나뉜 키워드들 중 하나라도 완전 일치하는 문서는 모두 추출된다. 예를 들어, <CONTENT:in:경영관리팀>의 경우 CONTENT 필드의 내용이 "경영관리팀"인 경우에만 문서가 추출 된다. <CONTENT:in:경영관리팀 경영지원팀>의 경우 CONTENT 필드의

내용이 "경영관리팀" 혹은 "경영지원팀"인 경우에만 문서가 추출 된다.즉 MATCH 쿼리 여러 개를 should(OR)연산 하여 만들어진 쿼리와 같다.

[in 연산자 사용 예시]

```
POST : http://127.0.0.1:9001/search
{
  "commonQuery": "입찰",
  "collection": {
    "alias": {
      "id": "sample",
      "collectionId": "sample",
      "searchField": [ "TITLE" ],
      "filterQuery": "<CONTENT:in:경영관리팀 경영지원팀>"
    }
  }
}
```

9.3.6. NOT_IN 연산자

NOT_IN 연산자는 특정 필드에 입력한 하나 이상의 키워드가 포함되지 않는 문서를 검색한다. NOT_IN 연산자 사용 시 띄어쓰기를 기준으로 나뉜 키워드가 모두 포함되지 않는 문서만 추출된다. 예를 들어 <CONTENT:not_in:경영관리팀 경영지원팀>의 경우 CONTENT 필드의 내용이 "경영관리팀" 혹은 "경영지원팀"과 완전히 일치하는 문서들은 추출하지 않는다. 즉 NOT_MATCH 쿼리 여러 개를 MUST(AND)연산 하여 만들어진 쿼리와 같다.

[not_in 연산자 사용 예시]

```
POST : http://127.0.0.1:9001/search
{
  "commonQuery": "입찰",
  "collection": {
    "alias": {
      "id": "sample",
      "collectionId": "sample",
      "searchField:" [ "TITLE" ],
```

```
        "filterQuery": "<CONTENT:not_in:경영관리팀 경영지원팀>"
      }
    }
}
```

9.3.7. START 연산자

START 연산자는 특정 필드에서 입력한 값으로 시작하는 값을 포함한 문서를 검색한다.

[start 연산자 사용 예시]

```
POST : http://127.0.0.1:9001/search
{
  "commonQuery": "입찰",
  "collection": {
    "alias": {
      "id": "sample",
      "collectionId": "sample",
      "searchField:" [ "TITLE" ],
      "filterQuery": "<CONTENT:start:경영>"
    }
  }
}
```

9.3.8. END 연산자

END 연산자는 특정 필드에서 입력한 값으로 끝나는 값을 포함한 문서를 검색한다.

[end 연산자 사용 예시]

```
POST : http://127.0.0.1:9001/search
{
  "commonQuery": "입찰",
  "collection": {
    "alias": {
      "id": "sample",
```

```
        "collectionId": "sample",
        "searchField:" [ "TITLE" ],
        "filterQuery": "<CONTENT:end:지원팀>"
      }
    }
}
```

9.3.9. SUBSTRING 연산자

SUBSTRING 연산자는 특정 필드에서 입력한 값을 포함한 문서를 검색한다.

[substring 연산자 사용 예시]

```
POST : http://127.0.0.1:9001/search
{
  "commonQuery": "입찰",
  "collection": {
    "alias": {
      "id": "sample",
      "collectionId": "sample",
      "searchField:" [ "TITLE" ],
      "filterQuery": "<CONTENT:substring:지원팀>"
    }
  }
}
```

9.3.10. GT, GTE, LT, LTE 연산자

GT 연산자는 특정 필드의 값이 입력한 값보다 큰 문서를 검색한다. (아스키코드값 기준)

GTE 연산자는 특정 필드의 값이 입력한 값보다 크거나 같은 문서를 검색한다. (아스키코드값 기준)

LT 연산자는 특정 필드의 값이 입력한 값보다 작은 문서를 검색한다. (아스키코드값 기준)

LTE 연산자는 특정 필드의 값이 입력한 값보다 작거나 같은 문서를 검색한다. (아스키코드값 기준)

하단의 예시는 GTE 연산 사용 예시이다. GT, LT, LTE도 아래 예시와 같이 사용할 수 있다.

[GTE 연산자 사용 예시]

```
POST : http://127.0.0.1:9001/search
{
  "commonQuery": "입찰",
  "collection": {
    "alias": {
      "id": "sample",
      "collectionId": "sample",
      "searchField:" [ "TITLE" ],
      "filterQuery": "<DEPARTMENT:gte:42>"
    }
  }
}
```

9.3.11. RANGE 연산

RANGE 연산은 특정 필드 값이 주어진 범위의 값 사이에 있는 문서를 검색한다. (내부에 GT, GTE, LT, LTE 연산자 사용하여 range를 지정)

9.3.11.1. API 호출

[표 9.9 filterQuery range 검색 API]

HTTP Method	URL	비고
POST	/search/	요청

9.3.11.2. range 연산 기본구조

[range 연산 기본구조]

```
"filterQuery": "<{fieldName}:{operator}:{value}> <{fieldName}:{operator}:{value}>"
```

9.3.11.3. 옵션설명

옵션	설명	필수

fieldName	필터링을 수행할 필드명을 입력한다. indexField로 잡힌 필드명만 기입 가능하다.	O
operator	range 연산자을 위해 gt, gte, lt, lte 연산자를 조합하여 범위 안에 있는 문서를 검색한다. 사용할 수 있는 조합은 다음과 같다. * gt, lt * gt, lte * gte, lt * gte, lte	O
value	필터링할 값을 입력한다.	O

9.3.11.4. 예시

[range 연산 사용 예시]

```
POST : http://127.0.0.1:9001/search
{
  "commonQuery": "입찰",
  "collection": {
    "alias": {
      "id": "sample",
      "collectionId": "sample",
      "searchField:" [ "TITLE" ],
      "filterQuery": "<DEPARTMENT:gt:25> <DEPARTMENT:lte:30>"
    }
  }
}
```

9.3.12. 날짜 제한 검색 쿼리 (Date Range Query)

날짜 제한 검색은 날짜/시간 범위를 지정하여 검색 결과를 제어하는 기능이다. 날짜 제한 검색쿼리는 date type의 필드에서만 사용이 가능하다. 그렇기에 컬렉션 세팅 시, index field에 date type으로 필드를 설정해주어야 한다.

9.3.12.1. API 호출

[표 9.10 date range 검색 API]

HTTP Method	URL	비고
POST	/search/	요청

9.3.12.2. date_range 연산 기본구조

[date_range 연산 기본구조]

```
"dateRange": [
  {
    "field": "DATE",
    "startDate": "{start value}",
    "endDate": "{end value}",
    "format": "{format}",
  }
]
```

9.3.12.3. 옵션설명

옵션	설명	필수
dateRange.field	검색을 수행할 필드명을 입력한다. date Type의 필드명만 입력가능하다.	O
dateRange.startDate	문서의 시작 날짜값을 입력한다.	O
dateRange.endDate	문서의 종료 날짜값을 입력한다.	O
dateRange.format	startDate 와 endDate 를 분석할 형식을 입력한다. 만약 format 옵션을 사용하지 않았을 경우, 컬렉션 세팅에서 설정한 시간 형식(format)으로 startDate 와 endDate 값을 분석한다.	X

9.3.12.4. 예시

Format을 입력하지 않았을 경우, default format은 컬렉션 세팅 때 등록한 format을 따르게 되므로 검색 시에도 같은 format으로 입력해야한다. 하단의 예시처럼 새로운 format을 지정해주고 검색하는 것도 가능하다. 주의해야 할 점은 "startDate" 와 "endDate" 에 들어가는 값이 "format" 의 형태와 일치해야 한다는 점이다.

[date_range 연산자 사용 예시]

```
POST : http://127.0.0.1:9001/search

{
  "collection": {
    "alias": {
      "collectionId": "sample",
      "dateRange": [
        {
          "field": "DATE",
          "startDate": "20200411020101",
          "endDate": "20220411020101",
          "format": "yyyyMMddHHmmss"
        }
      ]
    }
  }
}
```

9.3.13. 카테고리 검색 쿼리 (Category Search Query)

카테고리 검색 쿼리는 Category type 으로 설정된 필드를 검색할 수 있는 기능이며, categorySearch 에 입력한 카테고리값을 포함하고 있는 문서를 추출한다. 카테고리 검색 쿼리는 filter 내에서만 사용 가능하다.

9.3.13.1. API 호출

[표 9.11 filterQuery category 검색 API]

HTTP Method	URL	비고
POST	/search/	요청

9.3.13.2. categorySearch 연산 기본구조

[categorySearch 연산 기본구조]

```
"categoryQuery": [
    {
      "field" : "{category field}",
      "category": "{category value}"
   }
]
```

9.3.13.3. 옵션설명

옵션	설명	필수
categoryQuery.field	검색을 수행할 필드명을 입력한다. category Type의 필드명만 입력가능하다.	O
categoryQuery.category	검색하고 싶은 카테고리 값을 입력한다.	O

9.3.13.4. 예시

[Category Query 연산자 사용 예시]

```
POST : http://127.0.0.1:9001/search
{
   "commonQuery": "와이즈넛",
   "collection": {
      "alias": {
         "searchField": [
            "TITLE"
         ],
```

```
        "collectionId": "sample",
        "categoryQuery": [
          {
              "field": "CATEGORY",
              "category" : "웹>검색"
          }
        ]
      }
    }
}
```

주의
✔ 카테고리 검색은 1 depth를 기준으로 검색된다. ✔ 위의 예시에서 커먼 쿼리에서 "와이즈넛" 에 카테고리 검색으로 "웹>검색" 을 추가하여 검색하면, 1 depth에 "웹", 2 depth 에 "검색" 을 포함하는 문서만 추출된다.

9.4. 권한 쿼리 (Authority Query)

9.4.1. 개요

권한 처리(Authority Query)는 문서의 권한 정보를 별도의 컬렉션(Collection)으로 분리하여 검색 결과를 필터링(Filtering)하는 기능이다. 권한 처리 기능이 없는 경우에는 문서 내에 권한 정보를 추가하기 때문에 권한 정보가 변경될 경우 문서 전체를 업데이트하는 경우가 발생된다. 권한 처리 기능을 이용하면 권한 정보가 변경되었을 경우에 권한 컬렉션만 색인하면 되므로 권한 정보를 빠르게 반영할 수있다.

9.4.1.1. 권한 컬렉션 API 호출

[표 9.12 권한 컬렉션 설정 API]

HTTP Method	URL	비고
POST	/authorities/{authority_collection_id}	초기화

POST	/authorities/{authority_collection_id}/update	부분 업데이트
POST	/authorities/{authority_collection_id}/delete	부분 삭제
POST	/authorities/{authority_collection_id}/delete	삭제 (요청 BODY가 비어있는 경우, 전체 데이터를 삭제함)

9.4.1.2. 권한 쿼리 API 호출

[표 9.13 권한 쿼리 API]

HTTP Method	URL	비고
POST	/search/{collection_id}	요청

9.4.1.3. 권한 조회

엔진에 설정되어 있는 권한 컬렉션의 정보를 조회할 때 사용 된다. 삭제 요청과 동일하게 추가 파라미터가 없다.

[표 9.14 권한 리스트 조회 API]

HTTP Method	URL	비고
GET	/authorities	권한 리스트 조회
GET	/authorities/{authority_collection_id}	권한 정보 조회

9.4.2. 권한 컬렉션 (Authority Collection)

권한 쿼리(Authority Query)를 사용하기 위해서는 컬렉션 설정 시, 권한 컬렉션을 따로 설정해주어야한다.

9.4.2.1. 컬렉션 내부 옵션

[표 9.15 권한 컬렉션 내부 옵션]

옵션	설명
key	권한 필드로 설정한 필드의 값을 key값으로 가진다.
accounts	해당 key값을 가진 문서에 주어질 권한계정을 설정한다.

[권한 컬렉션 설정 예]

```
[
  {
    "key": "1",
    "accounts": [
      "user",
      "admin"
    ]
  },
  {
    "key": "2",
    "accounts": [
      "admin"
    ]
  },
  {
    "key": "3",
    "accounts": [
      "user"
    ]
  }
]
```

9.4.3. 권한 쿼리 (Authority Query)

9.4.3.1. 권한 쿼리 기본 구조

```
    "authority": {
      "field": "AUTHORITY_ID",
      "account": "user"
    }
```

9.4.3.2. 쿼리 내부 옵션

[표 9.16 권한 쿼리 내부 옵션]

옵션	설명
authority	field : 권한 필드로 설정한 필드명 account : 권한 계정값

9.4.3.2.1. 예제

권한 쿼리 예시를 위해 sample 컬렉션의 AUTHORITY_ID 필드를 authority type으로 설정하였다.

9.4.3.2.2. 권한 컬렉션 설정

권한 쿼리를 사용하기 위해서는 반드시 해당 컬렉션에 대한 권한 컬렉션을 설정해주어야 한다. 권한 컬렉션은 색인 이후에 설정해주어도 가능하다. 권한 쿼리를 사용하기 위해서는 권한 컬렉션 설정을 완료하여야 한다. 예시는 8.1장 [sample 컬렉션에 대한 authority 컬렉션 설정] 부분과 같다.

9.4.3.2.3. 검색

[Authority 검색 예시]

```
POST : http://127.0.0.1:9001/search
{
  "commonQuery": "와이즈넛",
  "collection": {
    "{alias}": {
      "id": "sample",
      "collectionId": "sample"
      "searchField": [
        "CONTENT"
      ],
      "authority": {
        "field": "AUTHORITY_ID",
        "account": "user"
```

```
        }
      }
    }
  }
```

9.5. 부가 기능

9.5.1. 소개

부가 기능에는 “하이라이팅(Highlighting)”, “요약(Snippet)”, “정렬(Sorting)”, “형태소 분석 조회(Morpheme Analysis)”, “최근 검색어 조회(Recent Query List)”, “오타 교정 키워드 기능(Typo Keyword)”, “중복된 문서 검색(Duplicated Documents)”, “머지 컬렉션(Merge Collection)”, “통합 검색(Integrated Search)”, “거리 검색 (Geodistance)”의 기능이 있다. 각 기능을 수행하기 위해서는 애널라이저 설정, 컬렉션 설정, 문서 색인 과정이 선행되어야 한다. 하단의 예시를 참고해 진행한다.

9.5.2. 정렬 (Sorting)

정렬 (Sorting) 은 ”field” 을 기준으로 결과문서들을 정렬한다. ”field” 에 필드이름 대신 score 를 입력할 경우, 결과 문서들의 score를 기준으로 정렬하게 된다.

9.5.2.1. API 호출

[표 9.17 정렬 API]

HTTP Method	URL	비고
POST	/search/{collection_id}	요청

9.5.2.2. 정렬 기본 구조

[단일 정렬 구조]

```
"sorting": [
  {
    "field": {field},
    "reverse": {reverse}
  }
]
```

[멀티 정렬 구조]

```
"sorting": [
  {
    "field": {field1},
    "reverse": {reverse1}
  },
  {
    "field": {field2},
    "reverse": {reverse2}
  }
]
```

9.5.2.3. 쿼리 내부 옵션

[표 9.18 정렬 쿼리 내부 옵션]

옵션	설명	필수
field	정렬처리할 필드이름을 설정한다. {field} : 해당 필드를 가나다, ABC 순으로 정렬된다. score : score 값을 기준으로 정렬한다.	O
reverse	정렬, 역정렬 여부를 설정한다. (default : false)	X

9.5.2.4. 단일 정렬 예제

[단일 정렬 설정 예시]

```
POST : http://127.0.0.1:9001/search/sample
{
  "commonQuery": "검색",
```

```
  "searchField": [
    "TITLE"
  ],
  "sorting": [
    {
      "field": "score",
      "reverse": true
    }
  ]
}
```

9.5.2.5. 멀티 정렬 예제

멀티정렬은 하단과 같이 배열 형태로 입력가능하다. 하단은 score로 단일정렬 했을 경우와 score, DEPARTMENT로 멀티정렬 했을 경우의 결과값을 비교하여 멀티정렬이 수행되는 과정을 보여준다.

[단일 정렬 설정 예시]

```
POST : http://127.0.0.1:9001/search/sample
{
  "commonQuery": "엔진",
  "searchField": [
    "TITLE"
  ],
  "sorting": [
    {
      "field": "score",
      "reverse": true
    }
  ]
}
```

[멀티 정렬 설정 예시]

```
POST : http://127.0.0.1:9001/search/sample
{
  "commonQuery": "엔진",
```

```
  "searchField": [
    "TITLE"
  ],
  "sorting": [
    {
      "field": "score",
      "reverse": true
    },
    {
      "field": "DEPARTMENT",
      "reverse": true
    }
  ]
}
```

9.5.3. 형태소 분석 조회 기능 (Morpheme Analysis)

형태소 분석 조회는 쿼리식이 어떻게 분석되어 검색요청이 진행되는지 확인할 수 있는 기능이다. 검색 수행 시 입력하는 쿼리식과 문법은 동일하다. 검색필드에 여러 애널라이저가 등록되어 있을 경우, 애널라이저 별로 형태소분석 된 결과값을 확인할 수 있다.

9.5.3.1. API 호출

[표 9.19 형태소 분석 조회 기능 API]

HTTP Method	URL	비고
POST	/search/{collection_id}/query-morpheme	요청

8.8.4.2. 형태소 분석 조회 기능 기본 구조

```
{
  "query": "{sentence}",
  "searchField": [
    "{field-name}"
  ],
  "synonymExpansion": {option},
```

```
  "useSynonym": {option},
  "useLa": {option}
}
```

9.5.3.2. 쿼리 내부 옵션

[표 9.20 형태소 분석 조회 기능 내부 옵션]

옵션	설명	필수
Sentence	형태소 분석할 문장을 입력한다. 문장에 예약어로 사용되는 소괄호, 중괄호, 대괄호를 넣을 수 없다.	O
field-name	형태소 분석할 field를 입력한다.	O
synonymExpansion	동의어확장법을 설정하는 옵션이다. allTerm과 originalTerm 두가지 방식이 있다. allTerm : 0 (default) originalTerm : 1 originalTerm으로 설정할 경우 형태소분석된 term에 대해서는 동의어확장이 수행되지 않는다. 예시) 검색어 : [정부산하에] 일 경우, 동의어에 [정부산하] [정부] 등이 등록되어 있더라도 동의어확장이 되지 않는다. [정부산하에] 라는 originalTerm에 대한 동의어 확장이 수행된다. 자세한 예시는 [8.1.2.4] 쿼리 확장 옵션 예제에서 확인할 수 있다.	X
useSynonym	동의어 사전 적용여부를 설정하는 옵션이다. 사용하지 않음 : 0 동의어사전 적용 : 1 (default) useSynonym의 여부와 관계없이 like, matrix, ngram은 동의어사전이 적용 불가능하다. like, matrix, ngram은 애널라이저 등록 시, 동의어사전을 적용할 수 없으며, 검색 시에도 쿼리식에 useSynonym 옵션을 1으로 설정하여도 동의어확장이 수행되지 않는다. 자세한 예시는 [8.1.2.4] 쿼리 확장 옵션 예제에서 확인할 수 있다.	X
useLa	형태소분석기 사용여부를 설정하는 옵션이다. 사용하지 않음 : 0 형태소분석기 적용 : 1 (default)	X

	useLa 사용하지 않을 경우, useSynonym의 여부와 관계없이 동의어사전 적용이 불가하다. 쿼리는 token과 동일하게 확장되고, useSynonym 옵션을 1으로 설정하여도 동의어확장이 수행되지 않는다. 자세한 예시는 [8.1.2.4] 쿼리 확장 옵션 예제에서 확인할 수 있다.	

9.5.3.3. 예제

예제 실행을 위한 sample 컬렉션 설정에서 "TITLE" 에 kma, token 두개의 애널라이저를 등록하였다. 결과값을 통해 쿼리식이 각 애널라이저를 통해 형태소분석된 결과값이 무엇인지 확인 가능하다.

[형태소 분석 조회 예시]

```
POST : http://127.0.0.1:9001/search/sample/query-morpheme
  {
    "query": "검색엔진 ",
    "searchField": [
      "TITLE"
    ]
  }
```

9.5.4. 최근 검색어 조회 (Recent Query List)

최근 검색어 조회 (Recent Query List) 기능을 사용하기 위해서는 컬렉션 세팅 시, 최근 검색어 기능을 활성화 시키는 옵션을 추가 등록해야한다. 최근 검색어 조회는 match query 에 들어간 검색어를 기준으로 샤드가 있는 엔진에 기록된다.

9.5.4.1. API 호출

[표 9.21 최근 검색어 조회 API]

HTTP Method	URL	비고

POST	/search	요청
POST	/search/{collection_id}	요청

9.5.4.2. 기본 구조

[일반 검색 예시]

```
POST : http://127.0.0.1:9001/search/{collection-id}
{
 "recentQueryInfo": {boolean}
}
```

[통합 검색 예시]

```
POST : http://127.0.0.1:9001/search
{
  "collection": {
    "{alias-id}": {
      "collectionId": "{collection-id}",
      "recentQueryInfo": {boolean}
    }
  }
}
```

9.5.4.3. 쿼리 내부 옵션

[표 9.22 최근 검색어 조회 쿼리 내부 옵션]

옵션	설명
recentQueryInfo	최근 검색어 조회 기능 사용 시, 컬렉션 설정에서 설정한 개수 만큼 반환된다. 최근 검색어를 저장하고 보여주는 지를 설정할 수 있다. 인자 값으로는 true, false(default)가 있으며 false로 설정할 경우 기존에 저장된 최근 검색어들을 지우고 보여주지 않는다.

9.5.4.4. 예제

최근 검색어들을 기록하기 위해, 몇 차례의 검색을 수행한다. Merge Collection 검색에서 "collectionIds": ["sampleA","sampleB"]을 받아와 sampleA, sampleB에 "반팔티"를 각각 저장하고, 일반 검색에서 sampleA에 "블랙'을 저장하였다.

일반 검색에서 sampleA에 검색한 최근 검색어들을 "recentQuery"결과 값에서 조회할 수 있다. 통합 검색에서는 단일 Collection 검색일 경우(collectionId) 최근 검색어들을 조회할 수 있다.

[컬렉션 설정 요청 예시 1]

```
POST : http://127.0.0.1:9001/collections/sampleA
{
  "recentQuery": 5,
   ...
}
```

[검색 요청 예시 1]

```
POST : http://127.0.0.1:9001/search
{
  "commonQuery": "반팔티",
  "collection": {
    "shopping-merge": {
      "collectionIds": [
        "sampleA",
        "sampleB"
      ],
      "searchField": [
        "MERGE_GOODS_NAME"
      ],
      "mergeCollection": {
        "sampleA": {
          "MERGE_DOCID": "DOCID",
          "MERGE_GOODS_NAME": "sampleA_GOODS_NAME",
          "MERGE_PRICE": "sampleA_PRICE"
```

```
        },
        "sampleB": {
          "MERGE_DOCID": "DOCID",
          "MERGE_GOODS_NAME": "sampleB_GOODS_NAME",
          "MERGE_PRICE": "sampleB_PRICE"
        }
      },
      "recentQueryInfo": true
    }
  }
}
```

[검색 요청 예시 2]

```
POST : http://127.0.0.1:9001/search/sampleA
{
  "commonQuery": "블랙",
  "searchField": [
    "TITLE"
  ],
  "recentQueryInfo": true
}
```

[결과값 예시]

```
{
  "recentQuery": [
    "반팔티",
    "블랙"
  ]
}
```

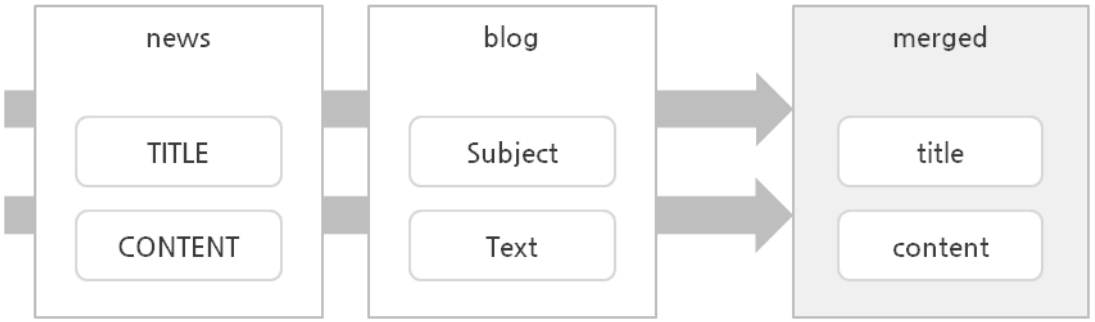
9.5.5. 머지 컬렉션 (Merged Collection)

머지 컬렉션(Merged Collection) 기능은 두 개 이상의 컬렉션(Collection)을 하나의 가상 통합

컬렉션으로 통합해서 검색결과를 보여주는 기능이다. 이 기능은 서로 다른 이종 컬렉션간에 검색을 한번에 해서 같은 화면에 결과를 보여주고자 할 때 편리하게 사용할 수 있다. 예를 들어서 뉴스 컬렉션과 블로그 컬렉션을 동시에 검색하고자 할 때, 두 개의 컬렉션을 하나의 가상 통합 컬렉션에 합쳐서 같이 보여줄 수 있다.

아래의 그림과 같이 각각 다른 컬렉션 (news collection과 blog collection) 이 존재한다. 머지 컬렉션(Merged Collection) 기능은 news 컬렉션과 blog 컬렉션에 검색하여 하나의 머지 컬렉션으로 합쳐 결과가 나타난다. 아래 그림에서 확인 할 수 있듯이 각 컬렉션에서 서로 다른 이름의 필드들을 하나의 머지 컬렉션(Merged Collection)의 필드로 매핑(Mapping) 할 것이다.

[그림 9.1 머지 컬렉션 시나리오]



9.5.5.1. API 호출

[표 9.23 머지 컬렉션 검색 API]

HTTP Method	URL	비고
POST	/search/	요청

9.5.5.2. 머지 컬렉션 검색 기본 구조

```
{
  "collection": {
```

```
    "{alias-id}": {
      "collectionIds": [
        "{collectionA}",
        "{collectionB}"
      ],
      "mergeCollection": {
        "{collectionA}": {
          "{merge-field-1}": "{collectionA-field-name}",
          "{merge-field-2}": "{collectionA-field-name}",
          "{merge-field-3}": "{collectionA-field-name}"
        },
        "{collectionB}": {
          "{merge-field-1}": "{collectionB-field-name}",
          "{merge-field-2}": "{collectionB-field-name}",
          "{merge-field-3}": "{collectionB-figld-name}"
        }
      },
      "documentField": [
        "{merge-field-1}",
        "{merge-field-2}"
      ],
      "collectionQuery": "<{merge-field-1}:contains:{query-keyword}>"
    }
  }
}
```

9.5.5.3. 쿼리 내부 옵션

[표 9.24 머지 컬렉션 검색 쿼리 내부 옵션]

옵션	설명	필수
collectionIds	머지컬렉션으로 합칠 컬렉션의 실제 id 값 리스트를 기입한다.	O
mergeCollection	머지 컬렉션(가상 컬렉션)에 넣을 컬렉션을 설정한다. 머지컬렉션의 필드이름에 매핑할 필드이름을 기입한다.	O

documentField	mergeCollection 에서 매핑한 가상 필드명을 설정한다. default: 전체 필드를 출력하되 mergeCollection 에서 설정한 필드명은 매핑된 필드이름으로 출력된다.	X

9.5.5.4. 예제

Merge할 컬렉션인 sampleA와 sampleB를 각각 설정해준다. 검색결과 비교를 위해 두 컬렉션은 각자 다른 사용자 사전을 적용하도록 sampleA_GOODS_NAME 에는 usernounA 를, sampleB_GOODS_NAME 에는 usernounB 를 사용하는 애널라이저를 설정하였다.

머지컬렉션 검색 시에는 쿼리Id(하단 예시에서 shopping-merge) 내부의 collectionIds 옵션을 통해 머지컬렉션으로 합칠 컬렉션을 구분하고, mergeCollection 옵션을 통해 필드값을 매핑한다. documentField 와 collectionQuery 에는 mergeCollection 에서 정의한 머지컬렉션의 필드값으로 사용한다.

[머지 컬렉션 검색 예시]

```
POST : http://127.0.0.1:9001/search
{
  "collection": {
    "shopping-merge": {
      "collectionIds": [
        "sampleA",
        "sampleB"
      ],
      "mergeCollection": {
        "sampleA": {
          "MERGE_DOCID": "DOCID",
          "MERGE_GOODS_NAME": "sampleA_GOODS_NAME",
          "MERGE_PRICE": "sampleA_PRICE"
        },
```

```
        "sampleB": {
          "MERGE_DOCID": "DOCID",
          "MERGE_GOODS_NAME": "sampleB_GOODS_NAME",
          "MERGE_PRICE": "sampleB_PRICE"
        }
      },
      "collectionQuery": "<MERGE_GOODS_NAME:contains:반 팔티>",
      "documentField": [
        "MERGE_DOCID",
        "MERGE_GOODS_NAME"
      ]
    }
  }
}
```

[결과값 예시]

```
{
  "shopping-merge": {
    "totalHits": 6,
    "pagingInfo": {...},
    "document": [
      {
        "distance": 0.0,
        "score": 0.89087397,
        "weight": 0.0,
        "fields": {
          "MERGE_DOCID": "1",
          "MERGE_GOODS_NAME": "여리여리 반팔티"
        }
      }
    ],
    "typoKeyword": null,
    "elapsedTime": 127.0
  }
}
```

[반팔티] 를 분석할 때, sampleA 컬렉션의 경우, kma-a 애널라이저의 usernounA에 의해 [반팔]과 [티]로 분리된다. sampleB 컬렉션의 경우, kma-b 애널라이저의 usernounB에 의해 [반]과 [팔티]로 분리된다.

그러므로 sampleA, sampleB 에 같은 값인 [여리여리 반팔티]를 색인했더라도, collectionQuery 로 "반 팔티" 를 검색시 sampleB 의 문서만 검색 결과로 출력된다.

9.5.6. 거리 제한 검색 (Geo Distance Search)

거리제한 검색은 검색된 문서들이 특정 위치에서 얼마나 떨어져있는지를 계산하고, 그 거리를 바탕으로 정렬을 하거나 필터링 할 수 있도록 해준다. 이때, 거리를 계산에 사용하는 정보는 문서의 위도(latitude)와 경도(longitude) 정보이다. CommonQuery 와 함께 사용 할 경우 결과 내 문서 안에서 거리 검색을 수행한다.

9.5.6.1. API 호출

[표 9.25 거리 검색 API]

HTTP Method	URL	비고
POST	/search	요청

9.5.6.2. 거리 검색 일반 검색 기본 구조

```
"geoDistance": {
    "latitudeField": {LATITUDE_FIELD},
    "latitude": {LATITUDE_VALUE},
    "longitudeField":{LONGITUDE_FIELD},
    "longitude": {LONGITUDE_VALUE},
    "distance": {DISTANCE_VALUE}
```

```
}
```

9.5.6.3. 쿼리 내부 옵션

[표 9.26 거리 검색 쿼리 내부 옵션]

옵션	설명	필수
latitudeField	collection 에 설정된 위도 필드 명	O
latitude	위도 값	O
longitudeField	collection 에 설정된 경도 필드 명	O
longitude	경도 값	O
distance	거리 (m)	O

9.5.6.4. 예제

[거리 검색 예시]

```
POST : http://127.0.0.1:9001/search/sample
{
  "sorting": [
    {
      "field": "DISTANCE",
      "reverse": false
    }
  ],
  "geoDistance": {
    "latitudeField": "LATITUDE",
    "longitudeField": "LONGITUDE",
    "distance": 100000,
    "latitude": 3.517841E8,
    "longitude": 1.2689179E9
  }
}
```

9.6. 랭킹

랭킹은 검색엔진에서 중요한 요소 중 하나이다. 랭킹이란 사용자가 입력한 검색어로 검색된 문서들을 어떠한 기준으로 정렬하여 제공할 것인가를 결정하는 것이다. 즉, 랭킹 설정에 따라 사용자의 검색 결과에 대한 만족도가 달라질 수 있으며, 데이터의 종류에 따라 랭킹을 다르게 설정할 필요가 있다.

9.6.1. 랭킹 요소

본 제품의 랭킹은 문서의 색인 시점에 추출되는 "시스템 랭킹 요소"(System-Based Ranking Feature)와 사용자가 필드에 부여하는 "사용자 정의 랭킹 요소"(User-Defined Ranking Feature), 문서의 속성에 따른 "문서 속성 랭킹 요소"(Document Property Ranking Feature), 검색에서 질의와 관련된 "다중질의어 랭킹 요소"(Multi-Query Ranking Feature)로 구성되어 있다. 각각의 요소별 세부 항목은 아래의 표와 같다.

[랭킹 요소]

랭킹 요소	설명
System-Based Ranking Feature	문서에서 추출된 색인어의 랭킹 요소 - TF: 색인어 출현 횟수에 대한 가중치 - Position: 색인어가 나타나는 문서의 위치 - Morpheme: 색인어의 형태 정보
User-Defined Ranking Feature	관리자가 정의한 랭킹 요소 - Field Score: 필드별 사용자 가중치 - Term Score: 색인어 사용자 가중치 - Query Score: 질의어 사용자 가중치
Document Property Ranking Feature	문서의 속성에 따른 랭킹 요소 - Length : 문서의 길이 - Date : 문서의 날짜 - User Feedback : 문서의 사용자 피드백
Multi-Query Ranking Feature	다중 질의어로 검색시의 랭킹 요소 - Multi-Keyword: 필드에 포함된 질의어 개수 - Proximity: 인접한 질의어 개수

9.6.2. 기본 구조

```
"similarity": {
        "mode": "{mode}",
        "option": "{option}",
        "percent":{integer},
        "maxRank":{integer},
        "minRank":{integer}
    }
```

9.6.3. 랭킹 내부 옵션

옵션	설명
{mode}	기본 랭킹을 사용할 지 커스텀 랭킹을 사용할 지 설정한다. 지원값 : “basic” or “custom”

{option}	mode에 해당하는 option값으로 뒤에 나올 예시를 참조하면 된다.
percent	퍼센트 랭크 적용 시 최대 Rank 값 (0 보다 큰 경우 적용)
maxRank	최대 랭킹 값
minRank	최소 랭킹 값

percent 랭킹의 경우 각 샤드에서 최고인 점수를 기준으로 백분율 변환을 한다.
백분율 변환을 한 후 maxRank와 minRank로 필터링을 적용할 경우 샤드별로 최고인 점수를 처리하고 필터링을 진행하기 때문에 최종적으로 문서를 확인하였을 경우 살짝 다를 수 있다. 다만 다량의 데이터로 색인을 진행할 경우 점수가 고르게 분포함으로 해결할 수 있다.

9.6.4. 키워드 부스팅 쿼리 주의 사항

키워드 부스팅은 “keyword” 값에 입력된 token과 정확히 일치하는 색인어가 문서에 있는 경우에만 부스팅처리된다. keyword 값을 [검색] 대신 [검색의] 라고 주어진다면, 문서에 [검색의] 라는 색인어가 존재하는 경우에만 해당 문서가 boosting된다. 즉, 키워드부스팅에 들어간 keyword는 형태소분석을 지원하지 않는다.

9.6.5. 기본 랭킹

기본값은 classic으로 설정된다.

sample_bbs 데이터를 이용하여 검색어 “엔진”을 검색했을 때의 각 랭킹 알고리즘의 문서 정렬 예시를 볼 수 있다.

Classic

이 알고리즘은 BM25 와 마찬가지로 TF/IDF 기반으로 문서의 관련성을 평가한다. 문서 내에서 검색어의 빈도와 전체 문서 집합에서 검색어의 역 문서 빈도를 고려하여 검색 결과를 정렬한다. 단어의 희소성을 나타내는 역문서빈도 (IDF) 값이 높을수록 해당 단어가 희귀하다고 간주되며, 검색 결과에서의 가중치가 증가한다.

문서의 길이가 비슷한 경우 빠른 속도로 검색이 가능하다. sample_bbs 데이터에서 [1, 9, 7, 11, 6, 10] 번 문서 순으로 정렬된다.

BM25 (Best Match 25)

Classic과 마찬가지로 TF/IDF 를 기반을 둔 랭킹 알고리즘으로, 기존 TF/IDF 랭킹 모델에서 문서 길이가 주는 영향을 개선하였다. 문서가 길면 BM25 에서 가중치가 감소하며, 문서 길이가 짧은 경우 검색어가 해당 문서의 핵심 내용을 잘 반영하였다고 간주한다.

BM25는 Classic 랭킹 방식에 비해 좀 더 정교하게 검색어와 문서 간의 유사성을 측정하는데, 단어의 등장 빈도를 고려하고 역문서빈도 (IDF) 값의 로그 변환을 적용하여 더 정확한 검색 결과를 제공한다. 역문서빈도 (IDF) 값의 로그 변환 적용은 선형적인 증가보다는 경향성을 좀 더 부드럽게 조정하며 이로 인해 희귀한 단어일수록 큰 가중치를 가지게 된다.

문서의 길이가 다양한 경우 BM25를 사용하면 일관성 있는 결과를 얻을 수 있다. sample_bbs 데이터에서 [9, 1, 7, 11, 6, 10]번 문서 순으로 정렬된다.

Boolean

검색어와 문서 간의 정확한 일치 여부만 고려하는 랭킹 방식이다. 검색어가 문서에 포함되어 있는지 여부와 쿼리의 가중치로 설정하는 값으로만 점수를 계산하며 검색어의 등장 횟수나 문서의 길이는 고려하지 않는 단순한 방식이다.

Boolean 방식은 단순하고 빠른 검색을 지원하며, 사용자가 명확한 검색어를 입력하는 경우에 효과적이지만 검색어의 유사도나 문서의 중요도를 고려하지 않기 때문에 검색 결과의 정확성이 상대적으로 낮다.

DFI(Divergence from independence)

DFI는 검색어와 문서를 이용하여 해당 검색어가 문서에 출현할 것이라 예측하는 예상 출현 빈도를 계산하고 검색어가 문서에 실제로 등장한 실제 출현 빈도를 비교하여 검색 결과를 정렬한다. 예상 출현 빈도보다 문서에 실제로 검색어가 출현한 횟수가 많은 경우 score가 부여된다.

문서에 전문적인 용어가 많이 들어가 있을 경우 효과적인 검색이 가능하다. sample_bbs 데이터에서 [9, 1, 7, 11, 6, 10]번 문서 순으로 정렬된다.

DFR(divergence from randomness)

DFR은 검색어와 문서의 유사도를 측정하여 검색 순위를 결정하는 랭킹 알고리즘이다. 문서 내의 각 단어의 출현 확률을 기반으로 무작위 분포를 생성하고 검색어와 문서 내용을 통해 얻어진 검색어의 출현 빈도와 무작위 분포의 차이를 계산한다. 차이 값이 작을 수록 문서와 검색어의 관련성이 높다고 간주하며 검색어와 문서의 관련성을 평가하는데 효과적이다.

sample_bbs 데이터에서 [9, 1, 7, 11, 10, 6]번 문서 순으로 정렬된다.

LM (Language Modeling)

LM은 검색어와 문서 간의 언어 모델을 사용하여 랭킹을 계산하는 알고리즘이다. LM은 검색어와 문서 간의 확률적인 관계를 모델링하고, 이를 통해 문서의 관련성을 평가한다. LM은 정보 검색에서 통계적 언어 모델링의 개념을 기반으로 하며, 검색어와 문서 사이의 언어적 일치도를 고려하여 랭킹을 결정한다.

문서의 길이가 대체로 길거나 대화형 검색을 사용하는 경우 검색어와 유사한 문서를 검색할 수 있다. sample_bbs 데이터에서 [9, 1, 7, 11, 6]번 문서 순으로 정렬된다.

[bm25 랭킹 예시]

```
POST: http://127.0.0.1:9001/search/sample_bbs
{
  “commonQuery”: “엔진”,
```

```
  "searchField": [
    "CONTENT"
  },
  ⋮
  "similarity": {
    "mode": "basic",
    "option": "bm25"
  }
}
```


Result


```
"document": [
  {
    "distance": -1.0,
    "score": 0.7549048,
    "weight": 0.0,
    "fields": {
      "DOCID": "9",
      "TITLE": "\"의미있는 정보만 가공·분석 검색엔진으로 진화\"",
      "CONTENT": "<HS>엔진<!HE>이다. 검색<HS>엔진<!HE>의 성능을 높이기 위해 연구개발(R&"
    }
  },
  {
    "distance": -1.0,
    "score": 0.7459973,
    "weight": 0.0,
    "fields": {
      "DOCID": "1",
      "TITLE": "와이즈넛, KM&ECM 컨퍼런스 2011 참가 - \"뜨거운 관심 이끌어\"",
      "CONTENT": "<HS>엔진<!HE>전문기업 와이즈넛\"은 지난 17일 삼성동 코엑스에서"
    }
  },
```

```
{
  "distance": -1.0,
  "score": 0.72054684,
  "weight": 0.0,
  "fields": {
    "DOCID": "7",
    "TITLE": "와이즈넛, 축적된 언어분석기술로 일본 · 중국어 형태소분석기 개발",
    "CONTENT": "<HS>엔진<!HE>, 자동분류기, 정보수집기 등 언어처리를 통해 정보"
  }
},
{
  "distance": -1.0,
  "score": 0.6465423,
  "weight": 0.0,
  "fields": {
    "DOCID": "11",
    "TITLE": "와이즈넛, 다양한 검색기술 앞세워 中·日 진출",
    "CONTENT": "<HS>엔진<!HE>인 '서치 포뮬러-1 v5.0'과 기존 솔루션의 업"
  }
},
{
  "distance": -1.0,
  "score": 0.5749877,
  "weight": 0.0,
  "fields": {
    "DOCID": "6",
    "TITLE": "와이즈넛과 피알원,신개념 여론모니터링시스템 개발",
    "CONTENT": "<HS>엔진<!HE>최적화) 컨설팅 분야까지 서비스 영역을 확대할 계획"
  }
},
```

```
  {
    "distance": -1.0,
    "score": 0.5418,
    "weight": 0.0,
    "fields": {
      "DOCID": "10",
      "TITLE": "와이즈넛, G4C 등 공공분야 검색사업 주도",
      "CONTENT": "<HS>엔진<!HE>을 본격 가동하는 원년이 될 것으로 기대하고 있다."
    }
  }
]
```


[DFI 랭킹 예시]

POST: http://127.0.0.1:9001/search/sample_bbs


```
{
  "commonQuery": "엔진",
  "searchField": [
     "CONTENT"
  },
  ⋮
  "similarity": {
     "mode": "basic",
     "option": "dfi"
  }
}
```


Result


```
"document": [
  {
    "distance": -1.0,
    "score": 1.4577628,
```

```
    "weight": 0.0,
    "fields": {
      "DOCID": "9",
      "TITLE": "\"의미있는 정보만 가공·분석 검색엔진으로 진화\"",
      "CONTENT": "<HS>엔진<!HE>이다. 검색<HS>엔진<!HE>의 성능을 높이기 위해 연구개발(R&"
    }
  },
  {
    "distance": -1.0,
    "score": 1.3679293,
    "weight": 0.0,
    "fields": {
      "DOCID": "1",
      "TITLE": "와이즈넛, KM&ECM 컨퍼런스 2011 참가 - \"뜨거운 관심 이끌어\"",
      "CONTENT": "<HS>엔진<!HE>전문기업 와이즈넛\"은 지난 17일 삼성동 코엑스에서"
    }
  },
  {
    "distance": -1.0,
    "score": 1.3679293,
    "weight": 0.0,
    "fields": {
      "DOCID": "7",
      "TITLE": "와이즈넛, 축적된 언어분석기술로 일본 · 중국어 형태소분석기 개발",
      "CONTENT": "<HS>엔진<!HE>, 자동분류기, 정보수집기 등 언어처리를 통해 정보"
    }
  },
  {
    "distance": -1.0,
    "score": 1.2780957,
```

```
    "weight": 0.0,
    "fields": {
      "DOCID": "11",
      "TITLE": "와이즈넛, 다양한 검색기술 앞세워 中·日 진출",
      "CONTENT": "<HS>엔진<!HE>인 '서치 포뮬러-1 v5.0'과 기존 솔루션의 업"
    }
  },
  {
    "distance": -1.0,
    "score": 1.0984286,
    "weight": 0.0,
    "fields": {
      "DOCID": "10",
      "TITLE": "와이즈넛, G4C 등 공공분야 검색사업 주도",
      "CONTENT": "<HS>엔진<!HE>을 본격 가동하는 원년이 될 것으로 기대하고 있다."
    }
  },
  {
    "distance": -1.0,
    "score": 1.0984286,
    "weight": 0.0,
    "fields": {
      "DOCID": "6",
      "TITLE": "와이즈넛과 피알원,신개념 여론모니터링시스템 개발",
      "CONTENT": "<HS>엔진<!HE>최적화) 컨설팅 분야까지 서비스 영역을 확대할 계획"
    }
  }
]
```

9.6.6. 커스텀 랭킹

각각의 랭킹 요소에 대한 가중치는 “;” 로 연결하여 여러 값을 표현할 수 있으며, 지정하지 않은 랭킹 요소 가중치는 0.0 으로 설정되어 랭킹 계산에 포함하지 않는다. weight 요소 합의 값이 0과 1 사이에 있는 값이어야 한다.

속성	설명
rweight	필드 가중치 (Field RankScore)에 대한 가중치 값이다.
fweight	색인어 빈도수 (Term Frequence)에 대한 가중치 값이다.
lweight	검색어와 문서의 길이 비율에 대한 가중치 값이다. [주의] lweight를 사용하기 위해서는 문서의 길이를 저장하기 위해 index field로 저장해야한다.
uweight	사용자 피드백 (User Feedback)에 대한 가중치 값이다. [주의] uweight를 사용하기 위해서는 ufield, ufieldmax를 설정해야 하며, ufield는 Document 필드의 user 타입으로 설정한 필드여야 한다.
dweight	문서의 최신성 (Document Freshness) 에 대한 가중치 값이다. [주의] date 타입으로 색인된 필드만 적용 가능하며, dfield 옵션에 해당 필드를 지정해야한다.
dfield	dweight를 적용할 필드를 지정한다. 필드는 date 타입이어야 한다.
ufield	사용자 피드백 (User Feedback) 필드를 지정한다. 필드는 user 타입이어야 한다.
ufieldmax	사용자 피드백 (User Feedback) 필드의 최대값을 설정한다. 문서의 필드 값이 ufieldmax보다 큰 경우에는 ufieldmax 값으로 제한한다.

[커스텀 랭킹 예시]

```
POST : http://127.0.0.1:9001/search/sample
{
  "commonQuery": "검색",
  "searchField": [
    "TITLE"
  ],
```

```
  "similarity": {
    "mode": "custom",
    "option": "rweight=0.6;uweight=0.4;ufieldmax=1000;ufield={user type field}"
  }
}
```

10. 로그 관리

본 장에서는 로그 설정 및 시스템 로그, 색인 로그, 검색 로그에 대하여 설명한다.

10.1. 로그 설정

10.1.1. 로그 파일 경로

로그 파일의 위치는 추가로 제공하는 설정파일(irag.sh) 에서 변경이 가능하다. 하단의 예시에서 ${LOG_PATH} 에 로그가 저장될 경로를 지정할 수 있으며, 로그 경로를 지정하지 않았을 때 기본값은 ${IRAG_HOME}/log 와 같다.

로그파일 구조는 ${LOG_PATH}/{yyyyMMdd} 경로에 아래와 같은 파일이름으로 로그 레벨에 따라 기록된다.

[표 10.1 로그 파일 구조]

파일 이름	설명
indexer_{level}.log	engine 에 색인하였을 때 색인 모듈과 관련된 로그가 기록된다.
querylog_{level}.log	검색 쿼리와 검색 과정에 대한 트레이스 로그가 기록된다.
system_{level}.log	전체적인 시스템 로그가 기록된다.
isc_{level}.log	사용자 검색 요청을 처리하는 과정에 대한 로그가 기록된다.
sc_{level}.log	검색 모듈과 관련된 로그가 기록된다.
stdout	batch 파일 실행 시 발생하는 로그가 기록된다.

로그는 타입에 따라 정보 로그와 에러 로그로 분리된 파일로 생성되며, 로그 파일의 형식은 아래와 같다.

[표 10.2 로그 타입별 로그 파일 형식]

로그 타입	설명
정보, 경고	동작 현황, 결과 및 경고 로그를 기록한다. 예) ${LOG_PATH}/log/{yyyyMMdd}/{로그 구분}_info.log
에러, 치명	동작 오류에 대한 로그를 기록한다. 예) ${LOG_PATH}/log/{yyyyMMdd}/{로그 구분}_error.log

10.1.2. 로그 언어 지원

로그를 출력하는 언어로는 한국어(ko_KR)와 영어(en_US)를 지원한다. 하단의 예시에서 설정 파일의 ${LOCALE} 에 로그 언어를 설정할 수 있으며, 로그 언어를 지정하지 않았을 때 기본값은 한국어이다.

[로그 언어 설정 예시]

irag.sh
LOCALE=en_US export LOCALE

10.2. 시스템 로그

시스템 로그는 시스템 운영에 필요한 정보나 오류 로그를 기록하게 된다. 시스템 로그의 기본 포맷은 아래와 같다.

[시스템 로그 파일 경로]

${LOG_PATH}/log/{yyyyMMdd}/system_{level}.log

[시스템 로그 포맷]

[yyyy-MM-dd hh:mm:ss:msec][*[module]<code> message*]

[표 10.3 시스템 로그 설명]

구분	설명
yyyy-MM-dd hh:mm:ss:msec	로그가 기록된 날짜 및 시간을 기록한다. 예) 2020-08-10 13:16:38.615
module	해당 로그를 기록한 모듈을 기록한다. 예) [SYS]
code	로그에 대한 로그 번호를 기록한다. 예) <SFL_1002>
message	로그에 대한 메시지를 기록한다.

[시스템 로그 출력 예]

```
[ 00001 ] [ 2020-08-10 13:16:38.610 ] [ [SYS]<SFL_1001> 홈 경로가 설정되었습니다. ( E:\irag-v2 ) ]
[ 00001 ] [ 2020-08-10 13:16:38.610 ] [ [SYS]<SFL_1002> 데이터 경로가 설정되었습니다. ( E:\irag-v2\data ) ]
[ 00001 ] [ 2020-08-10 13:16:38.610 ] [ [SYS]<SFL_1003> 공유데이터 경로가 설정되었습니다. ( E:\irag-v2\shared ) ]
[ 00001 ] [ 2020-08-10 13:16:38.610 ] [ [SYS]<SFL_1004> 지식 경로가 설정되었습니다. ( E:\irag-v2\knowledge ) ]
[ 00001 ] [ 2020-08-10 13:16:38.610 ] [ [SYS]<SFL_1005> 플러그인 경로가 설정되었습니다. ( E:\irag-v2\plugins ) ]
[ 00002 ] [ 2020-08-10 13:16:40.903 ] [ [SYS]<SFL_0751> shared data 초기화 ]
```

10.3. 색인 로그

색인 로그는 색인 모듈에서 발생하는 정보를 기록하게 된다. 색인 로그의 기본 포맷은 아래와 같다.

[색인 로그 파일 경로]

```
${LOG_PATH}/log/{yyyyMMdd}/indexer_{level}.log
```

[색인 로그 포맷]

```
[thread-id][yyyy-MM-dd hh:mm:ss:msec][ [IDX]<irag code> message ]
```

[표 10.4 색인 로그 설명]

구분	설명
[thread-id]	thread id 값을 기록한다.
yyyy-MM-dd hh:mm:ss:msec	로그가 기록된 날짜 및 시간을 기록한다. 예) 2020-08-10 13:16:38.615
IDX	색인 모듈명인 IDX 를 기록한다.
irag code	로그에 대한 로그 번호를 기록한다. 예) <SFL_1002>
message	로그에 대한 메시지를 기록한다. 예) 홈 경로가 설정되었습니다. (E:\irag-v2)

[색인 로그 출력 예]

```
[ 00011 ] [ 2020-07-29 15:12:41.331 ] [ [IDX]<SFL_1801> ( scity ) 컬렉션의 캐시 설정이 완료되었습니다. QueryCacheCount : ( 1,000 ) | QueryCacheSize : ( 500,000 ) byte ]
[ 00010 ] [ 2020-07-29 15:12:52.753 ] [ [IDX]<SFL_1204> Collection ( scity ) 색인 작업이 시작되었습니다. ]
[ 00012 ] [ 2020-07-29 15:13:08.537 ] [ [IDX]<SFL_1214> 색인 작업이 시작되었습니다. ] [ 00010 ] [ 2020-07-29 15:39:34.109 ] [ [IDX]<SFL_1222> 색인 건수: 102,400 ]
[ 00010 ] [ 2020-07-29 15:39:34.109 ] [ [IDX]<SFL_1223> 색인 소요 시간: 986,804.366 ]
```

10.4. 검색 로그

검색 로그는 검색어 및 검색한 컬렉션, 검색된 결과 건수 등이 기록된다.

[검색 로그 파일 경로]

```
${LOG_PATH}/log/{yyyyMMdd}/query.log
```

[검색 로그 포맷]

[thread-id] [QUERY_LOG][yyyy-MM-dd hh:mm:ss:msec][*processing time*][*collection id*][*keyword*][*totalHits*][*session info*]

[표 10.5 검색 로그 설명]

구분	설명
thread-id	thread id 값을 기록한다.
yyyy-MM-dd hh:mm:ss:msec	로그가 기록된 날짜 및 시간을 기록한다. 예) 2020-08-10 13:16:38.615
processing time	검색 소요 시간을 기록한다. 단위는 초이다. 예) 0.01600
collection id	검색한 컬렉션의 id 값을 기록한다. (통합검색시에는 컬렉션 id 리스트가 기록된다.)
keyword	검색한 키워드를 기록한다.
totalHits	검색 결과 총 개수를 기록한다. 검색 실패시 -1 값을 반환한다.
session info	session info 로 설정한 값을 기록한다.

[검색 로그 출력 예]

```
[ 00013 ] [QUERY_LOG][2022-04-20 10:31:37][   0.89900][sampleA sampleB][][3][]
```